SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LAB MANUAL

CSA04

OPERATING SYSTEMS

LIST OF EXPERIMENTS

S. NO.	LAB EXPERIMENTS	CO
1	Implementation of Linux Commands for File and Directory Operations	CO1
2	Development of Shell Scripts for System Information and Resource Monitoring	CO1
3	Implementation of Shell Scripts Using Variables, Operators, and Expressions	CO1
4	Development of Shell Scripts Using Conditional Statements	CO1
5	Development of Shell Scripts Using Loops and Functions	CO1
6	Implementation of File Permissions and Ownership Management in Linux	CO1
7	Implementation of System Calls for File Creation, Reading, Writing, and Deletion	CO1
8	Implementation of Process Information Retrieval Using Linux System Interfaces	CO1
9	Implementation of CPU and Memory Utilization Monitoring Scripts	CO1
10	Development of Shell Scripts for Automated File Backup and Recovery	CO1
11	Implementation of Command Pipelines, Input Redirection, and Output Redirection	CO1
12	Development of Shell Scripts for User Account and Group Management	CO1
13	Implementation of System Calls for File Metadata and Permission Modification	CO1
14	Development of Shell Scripts for Log File Analysis and Error Detection	CO1
15	Implementation of a Menu-Driven Linux System Administration Utility	CO1
16	Implementation of Process Creation Using fork()	CO2
17	Implementation of Program Execution Using the exec() Family	CO2
18	Implementation of Parent–Child Process Synchronization Using wait()	CO2
19	Implementation of Orphan and Zombie Process Identification and Handling	CO2
20	Implementation of FCFS CPU Scheduling Algorithm	CO2
21	Implementation of Non-Preemptive SJF Scheduling Algorithm	CO2
22	Implementation of Preemptive SJF (SRTF) Scheduling Algorithm	CO2
23	Implementation of Non-Preemptive Priority Scheduling Algorithm	CO2
24	Implementation of Preemptive Priority Scheduling Algorithm	CO2
25	Implementation of Round Robin CPU Scheduling Algorithm	CO2
26	Implementation of Multilevel Queue Scheduling Algorithm	CO2
27	Implementation of Multilevel Feedback Queue Scheduling Algorithm	CO2

S. NO.	LAB EXPERIMENTS	CO
28	Performance Comparison of CPU Scheduling Algorithms Using Waiting Time, Turnaround Time, and Response Time	CO2
29	Implementation of Inter-Process Communication Using Unnamed Pipes	CO2
30	Implementation of Inter-Process Communication Using Named Pipes (FIFOs)	CO2
31	Implementation of Shared Memory Communication Between Processes	CO2
32	Implementation of Message Queue Communication Between Processes	CO2
33	Implementation of Signal-Based Process Communication	CO2
34	Implementation of Multithreaded Applications Using POSIX Threads	CO2
35	Implementation of Thread Synchronization Using Mutex Locks	CO2
36	Implementation of Semaphore-Based Synchronization for Shared Resources	CO2
37	Implementation of the Producer–Consumer Problem Using Semaphores	CO2
38	Implementation of the Readers–Writers Problem Using Synchronization Primitives	CO2
39	Implementation of the Dining Philosophers Problem Using Semaphores	CO2
40	Implementation of Condition Variables for Thread Coordination	CO2
41	Implementation of a Critical Section Solution Using Peterson's Algorithm	CO2
42	Implementation of Deadlock Prevention Through Resource Allocation Constraints	CO2
43	Implementation of the Banker's Algorithm for Deadlock Avoidance	CO2
44	Implementation of Deadlock Detection Using a Resource Allocation Graph	CO2
45	Implementation of Deadlock Recovery Through Process Termination and Resource Preemption	CO2
46	Simulation of Resource Allocation and Safe-State Detection	CO2
47	Implementation of Thread Pools for Concurrent Task Execution	CO2
48	Implementation of Producer–Consumer Synchronization Using Bounded Buffers	CO2
49	Implementation of Concurrent File Access Using Locks	CO2
50	Performance Evaluation of Multithreaded and Multiprocess Applications	CO2
51	Implementation of First Fit Memory Allocation Algorithm	CO3
52	Implementation of Best Fit Memory Allocation Algorithm	CO3
53	Implementation of Worst Fit Memory Allocation Algorithm	CO3
54	Implementation of Next Fit Memory Allocation Algorithm	CO3
55	Performance Comparison of Contiguous Memory Allocation Strategies	CO3
56	Simulation of Internal and External Fragmentation	CO3

S. NO.	LAB EXPERIMENTS	CO
57	Implementation of Memory Compaction for Fragmentation Reduction	CO3
58	Implementation of Swapping and Process Memory Transfer Simulation	CO3
59	Implementation of Static and Dynamic Linking Using C Programs	CO3
60	Implementation of Dynamic Library Creation and Runtime Linking in Linux	CO3
61	Implementation of Paging-Based Address Translation	CO3
62	Implementation of Page Table Management and Logical-to-Physical Address Conversion	CO3
63	Simulation of Translation Lookaside Buffer (TLB) Address Translation	CO3
64	Implementation of Segmentation-Based Address Translation	CO3
65	Implementation of Segmentation with Paging	CO3
66	Simulation of Demand Paging and Page Fault Handling	CO3
67	Implementation of FIFO Page Replacement Algorithm	CO3
68	Implementation of Least Recently Used (LRU) Page Replacement Algorithm	CO3
69	Implementation of Optimal Page Replacement Algorithm	CO3
70	Implementation of Least Frequently Used (LFU) Page Replacement Algorithm	CO3
71	Performance Comparison of Page Replacement Algorithms	CO3
72	Implementation of Global and Local Page Replacement Strategies	CO3
73	Simulation of Working Set Management and Thrashing Control	CO3
74	Implementation of File Operations Using open(), read(), write(), and close()	CO4
75	Implementation of File Copying Using Low-Level System Calls	CO4
76	Implementation of Directory Creation, Traversal, and Deletion Using System Calls	CO4
77	Implementation of File Metadata Retrieval and Modification Using Linux System Calls	CO4
78	Implementation of File Locking for Concurrent File Access	CO4
79	Implementation of Contiguous File Allocation	CO4
80	Implementation of Linked File Allocation	CO4
81	Implementation of Indexed File Allocation	CO4
82	Performance Comparison of File Allocation Methods	CO4
83	Implementation of an Inode-Based File Metadata Retrieval Utility	CO4
84	Implementation of File Searching and Indexing for Large Directories	CO4
85	Implementation of File Integrity Checking Using Checksums	CO4
86	Implementation of Incremental File Backup and Restoration	CO4

S. NO.	LAB EXPERIMENTS	CO
87	Implementation of Disk Scheduling Using FCFS	CO4
88	Implementation of Disk Scheduling Using SSTF	CO4
89	Implementation of Disk Scheduling Using SCAN and C-SCAN	CO4
90	Implementation of Disk Scheduling Using LOOK and C-LOOK	CO4
91	Performance Comparison of Disk Scheduling Algorithms Using Total Head Movement	CO4
92	Implementation of an Adaptive Disk Scheduling Algorithm Based on Request Patterns	CO4
93	Implementation of a File Cache Simulator for Repeated File Access	CO4
94	Implementation of Network File Sharing Using NFS in a Linux Environment	CO4
95	Implementation of File Storage and Retrieval Using a Cloud Storage Service	CO4
96	Implementation of File Transfer Between Linux Systems Using Network Protocols	CO4
97	Automation of Linux System Administration Tasks Using Shell Scripts and Cron	CO5
98	Creation, Configuration, and Management of Containers Using Docker or Podman	CO5
99	Creation, Configuration, and Performance Evaluation of Virtual Machines Using KVM/QEMU or VMware	CO5
100	Deployment of a Containerized Application Using Kubernetes and Evaluation of Resource Utilization	CO5

Exp. No: 1

IMPLEMENTATION OF LINUX COMMANDS FOR FILE AND DIRECTORY OPERATIONS

AIM:

To execute basic Linux commands for creating, listing, copying, moving, renaming and deleting files and directories.

PROCEDURE:

1. Open the Linux terminal.
2. Use pwd to display the present working directory.
3. Create directories using mkdir (with -p for nested directories).
4. Create files using touch and write text into a file using echo with redirection.
5. List the contents using ls -l and display a file using cat.
6. Copy a file using cp, move or rename it using mv.
7. Delete files using rm and directories using rmdir or rm -r.
8. Observe the output of each command.

COMMANDS:

```
pwd
mkdir -p college/dept
ls -l
cd college
touch notes.txt
echo "Operating Systems Lab" > notes.txt
cat notes.txt
cp notes.txt copy.txt
mv copy.txt dept/lab.txt
ls -R
mv notes.txt os_notes.txt
ls
rm dept/lab.txt
rmdir dept
ls -l
cd ..
rm -r college
ls
```

OUTPUT:

```
$ pwd
/home/student/oslab/exp1
$ mkdir -p college/dept
$ ls -l
total 4
drwxr-xr-x 3 root root 4096 Oct 21 09:21 college
$ cd college
$ touch notes.txt
$ echo "Operating Systems Lab" > notes.txt
$ cat notes.txt
Operating Systems Lab
$ cp notes.txt copy.txt
$ mv copy.txt dept/lab.txt
$ ls -R
```

```
.:
dept  notes.txt

./dept:
lab.txt
$ mv notes.txt os_notes.txt
$ ls
dept  os_notes.txt
$ rm dept/lab.txt
$ rmdir dept
$ ls -l
total 4
-rw-r--r-- 1 root root 22 Oct 21 09:21 os_notes.txt
$ cd ..
$ rm -r college
$ ls
```

RESULT:

Thus the Linux commands for file and directory operations were executed successfully and the output was verified.

Exp. No: 2

DEVELOPMENT OF SHELL SCRIPTS FOR SYSTEM INFORMATION AND RESOURCE MONITORING

AIM:

To develop a shell script that displays system information and monitors resource usage such as CPU, memory and disk.

PROCEDURE:

1. Start the script with the interpreter line #!/bin/bash.
2. Display the host name, logged-in user and kernel version using hostname, whoami and uname -r.
3. Display the system uptime using uptime -p.
4. Display the number of CPU cores using nproc.
5. Display memory usage using free -m.
6. Display disk usage of the root file system using df -h /.
7. Display the number of running processes using ps.
8. Save the script, give execute permission and run it.

PROGRAM:

```
#!/bin/bash
# System information and resource monitoring report
echo "========== SYSTEM INFORMATION =========="
echo "Host name      : $(hostname)"
echo "Logged-in user : $(whoami)"
echo "Kernel version : $(uname -r)"
echo "Date and time  : $(date '+%d-%m-%Y %H:%M:%S')"
echo "Uptime         : $(uptime -p)"
echo "CPU cores      : $(nproc)"
echo
echo "========== MEMORY USAGE (MB) =========="
free -m
echo
echo "========== DISK USAGE =========="
df -h / | awk '{print $1, $2, $3, $4, $5}'
echo
echo "========== PROCESS COUNT =========="
echo "Running processes : $(ps -e --no-headers | wc -l)"
```

OUTPUT:

```
========== SYSTEM INFORMATION ==========
Host name      : vm
Logged-in user : root
Kernel version : 6.18.44-fc-v139
Date and time  : 21-10-2026 09:21:39
Uptime         : up 20 minutes
CPU cores      : 2

========== MEMORY USAGE (MB) ==========
               total        used        free      shared  buff/cache   available
Mem:            8029         488        7416          14         345        7541
Swap:              0           0           0
```

```
========== DISK USAGE ==========
Filesystem Size Used Avail Use%
/dev/vda 252G 14G 30G 31%

========== PROCESS COUNT ==========
Running processes : 67
```

RESULT:

Thus the shell script to display system information and monitor resource usage was executed successfully.

Exp. No: 3

IMPLEMENTATION OF SHELL SCRIPTS USING VARIABLES, OPERATORS, AND EXPRESSIONS

AIM:

To write a shell script that uses variables, arithmetic operators, relational operators and expressions.

PROCEDURE:

1. Read two numbers from the user using read.
2. Store them in variables a and b.
3. Perform addition, subtraction, multiplication, division and modulus using $(( )).
4. Compare the numbers using relational operators -gt, -lt and -eq.
5. Evaluate a compound expression and display the result.
6. Display all results.

PROGRAM:

```
#!/bin/bash
echo -n "Enter first number : "
read a
echo -n "Enter second number: "
read b
echo "---- Arithmetic Operators ----"
echo "Sum        = $((a + b))"
echo "Difference = $((a - b))"
echo "Product    = $((a * b))"
echo "Quotient   = $((a / b))"
echo "Remainder  = $((a % b))"
echo "---- Relational Operators ----"
[ $a -gt $b ] && echo "$a is greater than $b"
[ $a -lt $b ] && echo "$a is less than $b"
[ $a -eq $b ] && echo "$a is equal to $b"
echo "---- Expression ----"
result=$(( (a + b) * 2 - a / b ))
echo "(a + b) * 2 - a / b = $result"
```

INPUT:

```
25
4
```

OUTPUT:

```
Enter first number : 25
Enter second number: 4
---- Arithmetic Operators ----
Sum        = 29
Difference = 21
Product    = 100
Quotient   = 6
Remainder  = 1
---- Relational Operators ----
25 is greater than 4
---- Expression ----
```

```
(a + b) * 2 - a / b = 52
```

RESULT:

Thus the shell script using variables, operators and expressions was executed successfully.

Exp. No: 4

DEVELOPMENT OF SHELL SCRIPTS USING CONDITIONAL STATEMENTS

AIM:

To write a shell script using conditional statements (if-else, elif and case).

PROCEDURE:

1. Read a mark from the user.
2. Use if-elif-else to find the grade of the student.
3. Read a number and use if-else to check whether it is even or odd.
4. Read a day number and use the case statement to display the day name.
5. Display the results.

PROGRAM:

```
#!/bin/bash
echo -n "Enter the mark (0-100): "
read mark
if [ $mark -ge 90 ]; then
    echo "Grade: S"
elif [ $mark -ge 75 ]; then
    echo "Grade: A"
elif [ $mark -ge 50 ]; then
    echo "Grade: B"
else
    echo "Grade: Fail"
fi

echo -n "Enter a number: "
read n
if [ $((n % 2)) -eq 0 ]; then
    echo "$n is Even"
else
    echo "$n is Odd"
fi

echo -n "Enter day number (1-7): "
read d
case $d in
    1) echo "Sunday" ;;
    2) echo "Monday" ;;
    3) echo "Tuesday" ;;
    4) echo "Wednesday" ;;
    5) echo "Thursday" ;;
    6) echo "Friday" ;;
    7) echo "Saturday" ;;
    *) echo "Invalid day number" ;;
esac
```

INPUT:

```
82
17
4
```

OUTPUT:

```
Enter the mark (0-100): 82
Grade: A
Enter a number: 17
17 is Odd
Enter day number (1-7): 4
Wednesday
```

RESULT:

Thus the shell script using conditional statements was executed successfully.

Exp. No: 5

DEVELOPMENT OF SHELL SCRIPTS USING LOOPS AND FUNCTIONS

AIM:

To write a shell script using loops (for, while) and user-defined functions.

PROCEDURE:

1. Define a function factorial() that computes the factorial using a while loop.
2. Define a function is_prime() that checks whether a number is prime using a for loop.
3. Read a number from the user.
4. Call factorial() and display the result.
5. Use a for loop to display the multiplication table of the number.
6. Use a for loop to call is_prime() and print the prime numbers up to the given number.

PROGRAM:

```
#!/bin/bash
factorial() {
    local n=$1 f=1
    while [ $n -gt 1 ]; do
        f=$((f * n))
        n=$((n - 1))
    done
    echo $f
}

is_prime() {
    local n=$1
    [ $n -lt 2 ] && return 1
    for ((i = 2; i * i <= n; i++)); do
        [ $((n % i)) -eq 0 ] && return 1
    done
    return 0
}

echo -n "Enter a number: "
read num
echo "Factorial of $num = $(factorial $num)"
echo "Multiplication table of $num:"
for i in 1 2 3 4 5; do
    echo "$num x $i = $((num * i))"
done
echo -n "Prime numbers up to $num: "
for ((k = 2; k <= num; k++)); do
    is_prime $k && echo -n "$k "
done
echo
```

INPUT:

```
7
```

OUTPUT:

```
Enter a number: 7
Factorial of 7 = 5040
Multiplication table of 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
Prime numbers up to 7: 2 3 5 7
```

RESULT:

Thus the shell script using loops and functions was executed successfully.

Exp. No: 6

IMPLEMENTATION OF FILE PERMISSIONS AND OWNERSHIP MANAGEMENT IN LINUX

AIM:

To manage file permissions and ownership in Linux using chmod, chown and chgrp.

PROCEDURE:

1. Create a file and a directory.
2. View the current permissions using ls -l.
3. Change permissions using symbolic mode (u, g, o with +, -).
4. Change permissions using octal mode (for example 754 and 640).
5. Create a user and a group, then change the owner using chown and the group using chgrp.
6. Verify the changes with ls -l and stat.

COMMANDS:

```
touch report.txt
mkdir project
ls -l
chmod u+x,g-r,o-r report.txt
ls -l report.txt
chmod 754 report.txt
ls -l report.txt
chmod 640 report.txt
ls -l report.txt
groupadd -f labgroup
id -u student1 >/dev/null 2>&1 || useradd student1
chown student1 report.txt
chgrp labgroup report.txt
ls -l report.txt
chown -R student1:labgroup project
ls -ld project
stat -c "%n  %A  %a  %U  %G" report.txt
```

OUTPUT:

```
$ touch report.txt
$ mkdir project
$ ls -l
total 4
drwxr-xr-x 2 root root 4096 Oct 21 09:21 project
-rw-r--r-- 1 root root    0 Oct 21 09:21 report.txt
$ chmod u+x,g-r,o-r report.txt
$ ls -l report.txt
-rwx------ 1 root root 0 Oct 21 09:21 report.txt
$ chmod 754 report.txt
$ ls -l report.txt
-rwxr-xr-- 1 root root 0 Oct 21 09:21 report.txt
$ chmod 640 report.txt
$ ls -l report.txt
-rw-r----- 1 root root 0 Oct 21 09:21 report.txt
$ groupadd -f labgroup
$ id -u student1 >/dev/null 2>&1 || useradd student1
$ chown student1 report.txt
$ chgrp labgroup report.txt
$ ls -l report.txt
```

```
-rw-r----- 1 student1 labgroup 0 Oct 21 09:21 report.txt
$ chown -R student1:labgroup project
$ ls -ld project
drwxr-xr-x 2 student1 labgroup 4096 Oct 21 09:21 project
$ stat -c "%n  %A  %a  %U  %G" report.txt
report.txt  -rw-r-----  640  student1  labgroup
```

RESULT:

Thus the file permissions and ownership were modified using chmod, chown and chgrp and verified successfully.

Exp. No: 7

IMPLEMENTATION OF SYSTEM CALLS FOR FILE CREATION, READING, WRITING, AND DELETION

AIM:

To write a C program that creates, writes, reads and deletes a file using the system calls open(), write(), read(), close() and unlink().

PROCEDURE:

1. Include <fcntl.h> and <unistd.h>.
2. Create a file using open() with O_CREAT | O_WRONLY | O_TRUNC and permission 0644.
3. Write a string into the file using write() and close it.
4. Open the file again in read-only mode and read the contents into a buffer using read().
5. Display the contents and close the file.
6. Delete the file using unlink() and confirm that it no longer exists using access().

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    char *fname = "sample.txt";
    char data[] = "Welcome to Operating Systems Lab\n";
    char buf[100];
    int fd, n;

    fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    n = write(fd, data, strlen(data));
    printf("File created and %d bytes written\n", n);
    close(fd);

    fd = open(fname, O_RDONLY);
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("Bytes read : %d\n", n);
    printf("Contents   : %s", buf);
    close(fd);

    if (unlink(fname) == 0)
        printf("File %s deleted\n", fname);
    if (access(fname, F_OK) != 0)
        printf("Verified: file no longer exists\n");
    return 0;
}
```

OUTPUT:

```
File created and 33 bytes written
Bytes read : 33
```

```
Contents   : Welcome to Operating Systems Lab
File sample.txt deleted
Verified: file no longer exists
```

RESULT:

Thus the C program to create, write, read and delete a file using system calls was executed successfully.

Exp. No: 8

IMPLEMENTATION OF PROCESS INFORMATION RETRIEVAL USING LINUX SYSTEM INTERFACES

AIM:

To write a C program that retrieves process information using system calls and the /proc interface.

PROCEDURE:

1. Get the process ID and parent process ID using getpid() and getppid().
2. Get the user ID and group ID using getuid() and getgid().
3. Open the file /proc/self/status using fopen().
4. Read it line by line and print the lines Name, State, PPid, Threads and VmRSS.
5. Close the file.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char line[256];
    FILE *fp;

    printf("Process ID        : %d\n", getpid());
    printf("Parent Process ID : %d\n", getppid());
    printf("User ID           : %d\n", getuid());
    printf("Group ID          : %d\n", getgid());

    fp = fopen("/proc/self/status", "r");
    if (fp == NULL) { perror("fopen"); return 1; }
    printf("\nDetails from /proc/self/status:\n");
    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "Name:", 5) == 0 || strncmp(line, "State:", 6) == 0 ||
            strncmp(line, "PPid:", 5) == 0 || strncmp(line, "Threads:", 8) == 0 ||
            strncmp(line, "VmRSS:", 6) == 0)
            printf("%s", line);
    }
    fclose(fp);
    return 0;
}
```

OUTPUT:

```
Process ID        : 1668
Parent Process ID : 1606
User ID           : 0
Group ID          : 0

Details from /proc/self/status:
Name:   prog
State:  R (running)
PPid:   1606
VmRSS:      1736 kB
```

```
Threads:        1
```

RESULT:

Thus the C program to retrieve process information using system calls and the /proc interface was executed successfully.

Exp. No: 9

IMPLEMENTATION OF CPU AND MEMORY UTILIZATION MONITORING SCRIPTS

AIM:

To write a shell script that monitors CPU and memory utilisation and displays a warning when usage crosses a threshold.

PROCEDURE:

1. Set the threshold values for CPU and memory usage.
2. Repeat the check three times with an interval of one second.
3. Calculate CPU usage from the first line of /proc/stat taken at two instants.
4. Calculate memory usage from the total and used values given by free.
5. Display the usage and print a WARNING if it crosses the threshold, otherwise print OK.

PROGRAM:

```
#!/bin/bash
CPU_LIMIT=80
MEM_LIMIT=80

cpu_usage() {
    read -r _ u1 n1 s1 i1 _ < /proc/stat
    sleep 1
    read -r _ u2 n2 s2 i2 _ < /proc/stat
    busy=$(( (u2 + n2 + s2) - (u1 + n1 + s1) ))
    total=$(( busy + (i2 - i1) ))
    [ $total -eq 0 ] && total=1
    echo $(( 100 * busy / total ))
}

for check in 1 2 3; do
    cpu=$(cpu_usage)
    mem=$(free | awk '/Mem:/ {printf "%d", $3 * 100 / $2}')
    echo "Check $check: CPU = ${cpu}%  Memory = ${mem}%"
    if [ $cpu -gt $CPU_LIMIT ]; then echo "  WARNING: High CPU usage"; else echo "  CPU
usage OK"; fi
    if [ $mem -gt $MEM_LIMIT ]; then echo "  WARNING: High memory usage"; else echo "
Memory usage OK"; fi
done
```

OUTPUT:

```
Check 1: CPU = 0%  Memory = 6%
  CPU usage OK
  Memory usage OK
Check 2: CPU = 0%  Memory = 6%
  CPU usage OK
  Memory usage OK
Check 3: CPU = 0%  Memory = 6%
  CPU usage OK
  Memory usage OK
```

RESULT:

Thus the shell script to monitor CPU and memory utilisation was executed successfully.

Exp. No: 10

DEVELOPMENT OF SHELL SCRIPTS FOR AUTOMATED FILE BACKUP AND RECOVERY

AIM:

To write a shell script that takes a backup of a directory as a compressed archive and restores it when required.

PROCEDURE:

1. Create a sample directory with files to be backed up.
2. Create a backup file name with the current date and time.
3. Create the backup archive using tar -czf.
4. List the contents of the archive using tar -tzf.
5. Simulate data loss by deleting the original directory.
6. Restore the directory by extracting the archive using tar -xzf and verify the files.

PROGRAM:

```
#!/bin/bash
SRC=project
BACKUP_DIR=backup
mkdir -p $SRC $BACKUP_DIR
echo "main program" > $SRC/main.c
echo "notes" > $SRC/notes.txt

FILE=$BACKUP_DIR/${SRC}_$(date +%Y%m%d_%H%M%S).tar.gz
tar -czf $FILE $SRC
echo "Backup created : $FILE"
echo "Archive contents:"
tar -tzf $FILE

rm -rf $SRC
echo "Original directory deleted (simulated data loss)"
ls $SRC 2>/dev/null || echo "$SRC not found"

tar -xzf $FILE
echo "Restored from backup. Files:"
ls -l $SRC
```

OUTPUT:

```
Backup created : backup/project_20261021_092145.tar.gz
Archive contents:
project/
project/notes.txt
project/main.c
Original directory deleted (simulated data loss)
project not found
Restored from backup. Files:
total 8
-rw-r--r-- 1 root root 13 Oct 21 09:21 main.c
-rw-r--r-- 1 root root  6 Oct 21 09:21 notes.txt
```

RESULT:

Thus the shell script for automated file backup and recovery was executed successfully.

Exp. No: 11

IMPLEMENTATION OF COMMAND PIPELINES, INPUT REDIRECTION, AND OUTPUT REDIRECTION

AIM:

To use command pipelines, input redirection and output redirection in Linux.

PROCEDURE:

1. Create a text file of student marks using output redirection (>).
2. Append data to the file using >>.
3. Use input redirection (<) to give the file as input to sort and wc.
4. Use pipelines (|) to combine commands such as sort, head, grep, cut and wc.
5. Redirect error messages to a file using 2>.

COMMANDS:

```
echo "Arun 78" > marks.txt
echo "Bala 92" >> marks.txt
echo "Kavi 65" >> marks.txt
echo "Divya 88" >> marks.txt
cat marks.txt
wc -l < marks.txt
sort -k2 -n -r < marks.txt
sort -k2 -n -r marks.txt | head -2
grep "a" marks.txt | wc -l
cut -d' ' -f1 marks.txt | sort > names.txt
cat names.txt
ls nofile.txt 2> error.log
cat error.log
```

OUTPUT:

```
$ echo "Arun 78" > marks.txt
$ echo "Bala 92" >> marks.txt
$ echo "Kavi 65" >> marks.txt
$ echo "Divya 88" >> marks.txt
$ cat marks.txt
Arun 78
Bala 92
Kavi 65
Divya 88
$ wc -l < marks.txt
4
$ sort -k2 -n -r < marks.txt
Bala 92
Divya 88
Arun 78
Kavi 65
$ sort -k2 -n -r marks.txt | head -2
Bala 92
Divya 88
$ grep "a" marks.txt | wc -l
3
$ cut -d' ' -f1 marks.txt | sort > names.txt
$ cat names.txt
Arun
Bala
Divya
```

```
Kavi
$ ls nofile.txt 2> error.log
$ cat error.log
ls: cannot access 'nofile.txt': No such file or directory
```

RESULT:

Thus the command pipelines, input redirection and output redirection were executed successfully.

Exp. No: 12

DEVELOPMENT OF SHELL SCRIPTS FOR USER ACCOUNT AND GROUP MANAGEMENT

AIM:

To write a shell script that creates and manages user accounts and groups in Linux.

PROCEDURE:

1. Run the script as the root user.
2. Create a group using groupadd.
3. Create a user using useradd and add the user to the group using usermod -aG.
4. Display the user details using id and the group entry from /etc/group.
5. Lock the user account using usermod -L and check the status using passwd -S.
6. Delete the user and the group using userdel and groupdel.

PROGRAM:

```
#!/bin/bash
GROUP=oslab
USER=student2

groupadd $GROUP && echo "Group $GROUP created"
useradd -m $USER && echo "User $USER created"
usermod -aG $GROUP $USER && echo "User $USER added to group $GROUP"

echo "User details:"
id $USER
grep "^$GROUP:" /etc/group

usermod -L $USER && echo "Account $USER locked"
passwd -S $USER

userdel -r $USER 2>/dev/null && echo "User $USER deleted"
groupdel $GROUP && echo "Group $GROUP deleted"
id $USER 2>&1
```

OUTPUT:

```
Group oslab created
User student2 created
User student2 added to group oslab
User details:
uid=30034(student2) gid=30035(student2) groups=30035(student2),30034(oslab)
oslab:x:30034:student2
Account student2 locked
student2 L 2026-10-21 0 99999 7 -1
User student2 deleted
Group oslab deleted
id: ‘student2’: no such user
```

RESULT:

Thus the shell script for user account and group management was executed successfully.

Exp. No: 13

IMPLEMENTATION OF SYSTEM CALLS FOR FILE METADATA AND PERMISSION MODIFICATION

AIM:

To write a C program that retrieves file metadata using stat() and modifies file permissions using chmod().

PROCEDURE:

1. Create a sample file.
2. Call stat() to get the file metadata.
3. Display the inode number, size, number of links, owner ID and permissions.
4. Change the permission of the file to 0754 using chmod().
5. Call stat() again and display the new permissions.

PROGRAM:

```
#include <stdio.h>
#include <sys/stat.h>

void show(char *name)
{
    struct stat st;
    if (stat(name, &st) < 0) { perror("stat"); return; }
    printf("File name   : %s\n", name);
    printf("Inode       : %lu\n", (unsigned long)st.st_ino);
    printf("Size        : %ld bytes\n", (long)st.st_size);
    printf("Links       : %lu\n", (unsigned long)st.st_nlink);
    printf("Owner UID   : %d\n", st.st_uid);
    printf("Permissions : %o\n\n", st.st_mode & 0777);
}

int main()
{
    FILE *fp = fopen("data.txt", "w");
    fprintf(fp, "file metadata demo\n");
    fclose(fp);

    printf("Before chmod:\n");
    show("data.txt");
    if (chmod("data.txt", 0754) == 0)
        printf("Permissions changed to 754\n\n");
    printf("After chmod:\n");
    show("data.txt");
    return 0;
}
```

OUTPUT:

```
Before chmod:
File name   : data.txt
Inode       : 886358
Size        : 19 bytes
Links       : 1
Owner UID   : 0
Permissions : 644
```

```
Permissions changed to 754

After chmod:
File name   : data.txt
Inode       : 886358
Size        : 19 bytes
Links       : 1
Owner UID   : 0
Permissions : 754
```

RESULT:

Thus the C program to retrieve file metadata and modify permissions using system calls was executed successfully.

Exp. No: 14

DEVELOPMENT OF SHELL SCRIPTS FOR LOG FILE ANALYSIS AND ERROR DETECTION

AIM:

To write a shell script that analyses a log file and reports the errors and warnings found in it.

PROCEDURE:

1. Create a sample log file containing INFO, WARNING and ERROR lines.
2. Count the total number of lines using wc -l.
3. Count the INFO, WARNING and ERROR lines using grep -c.
4. Display all the ERROR lines with line numbers using grep -n.
5. Find the most frequent error message using cut, sort and uniq -c.
6. Display a summary report.

PROGRAM:

```
#!/bin/bash
LOG=app.log
cat > $LOG << EOF
2025-08-01 10:00:01 INFO  Server started
2025-08-01 10:02:15 WARNING Disk usage 85%
2025-08-01 10:05:40 ERROR Database connection failed
2025-08-01 10:06:02 INFO  Retrying connection
2025-08-01 10:06:30 ERROR Database connection failed
2025-08-01 10:10:11 ERROR File not found: config.ini
2025-08-01 10:12:45 INFO  Request served
EOF

echo "Log file      : $LOG"
echo "Total lines   : $(wc -l < $LOG)"
echo "INFO lines    : $(grep -c 'INFO' $LOG)"
echo "WARNING lines : $(grep -c 'WARNING' $LOG)"
echo "ERROR lines   : $(grep -c 'ERROR' $LOG)"
echo
echo "Error lines with line numbers:"
grep -n "ERROR" $LOG
echo
echo "Most frequent error:"
grep "ERROR" $LOG | cut -d' ' -f4- | sort | uniq -c | sort -nr | head -1
```

OUTPUT:

```
Log file      : app.log
Total lines   : 7
INFO lines    : 3
WARNING lines : 1
ERROR lines   : 3

Error lines with line numbers:
3:2025-08-01 10:05:40 ERROR Database connection failed
5:2025-08-01 10:06:30 ERROR Database connection failed
6:2025-08-01 10:10:11 ERROR File not found: config.ini

Most frequent error:
      2 Database connection failed
```

RESULT:

Thus the shell script for log file analysis and error detection was executed successfully.

Exp. No: 15

IMPLEMENTATION OF A MENU-DRIVEN LINUX SYSTEM ADMINISTRATION UTILITY

AIM:

To develop a menu-driven shell script that performs common Linux system administration tasks.

PROCEDURE:

1. Display a menu with options: date, disk usage, memory usage, logged-in users, running processes and exit.
2. Read the choice of the user.
3. Use a case statement to execute the command for the chosen option.
4. Repeat the menu using a while loop until the user selects Exit.

PROGRAM:

```
#!/bin/bash
while true; do
    echo "===== SYSTEM ADMIN MENU ====="
    echo "1. Show date and time"
    echo "2. Show disk usage"
    echo "3. Show memory usage"
    echo "4. Show logged-in user"
    echo "5. Show process count"
    echo "6. Exit"
    echo -n "Enter your choice: "
    read ch
    case $ch in
        1) date ;;
        2) df -h / | tail -1 ;;
        3) free -m | grep Mem ;;
        4) whoami ;;
        5) echo "Running processes: $(ps -e --no-headers | wc -l)" ;;
        6) echo "Exiting..."; break ;;
        *) echo "Invalid choice" ;;
    esac
    echo
done
```

INPUT:

```
1
3
4
6
```

OUTPUT:

```
===== SYSTEM ADMIN MENU =====
1. Show date and time
2. Show disk usage
3. Show memory usage
4. Show logged-in user
5. Show process count
6. Exit
```

```
Enter your choice: 1
Wed Oct 21 09:21:45 IST 2026

===== SYSTEM ADMIN MENU =====
1. Show date and time
2. Show disk usage
3. Show memory usage
4. Show logged-in user
5. Show process count
6. Exit
Enter your choice: 3
Mem:            8029         494        7406          14         348        7535

===== SYSTEM ADMIN MENU =====
1. Show date and time
2. Show disk usage
3. Show memory usage
4. Show logged-in user
5. Show process count
6. Exit
Enter your choice: 4
root

===== SYSTEM ADMIN MENU =====
1. Show date and time
2. Show disk usage
3. Show memory usage
4. Show logged-in user
5. Show process count
6. Exit
Enter your choice: 6
Exiting...
```

RESULT:

Thus the menu-driven Linux system administration utility was executed successfully.

Exp. No: 16

IMPLEMENTATION OF PROCESS CREATION USING FORK()

AIM:

To write a C program that creates a new process using the fork() system call and displays the process IDs of the parent and the child.

PROCEDURE:

1. Include <unistd.h> and <sys/wait.h>.
2. Call fork() to create a child process.
3. If fork() returns a negative value, display an error.
4. If fork() returns 0, the code is running in the child: display the child's PID and its parent's PID.
5. Otherwise the code is running in the parent: display the parent's PID and the child's PID returned by fork().
6. The parent waits for the child to finish using wait().

PROGRAM:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    printf("Before fork: PID = %d\n", getpid());
    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
    } else if (pid == 0) {
        printf("Child  : PID = %d, Parent PID = %d\n", getpid(), getppid());
    } else {
        wait(NULL);
        printf("Parent : PID = %d, Child PID = %d\n", getpid(), pid);
    }
    return 0;
}
```

OUTPUT:

```
Before fork: PID = 1788
Child  : PID = 1789, Parent PID = 1788
Parent : PID = 1788, Child PID = 1789
```

RESULT:

Thus the C program to create a process using fork() was executed successfully.

Exp. No: 17

IMPLEMENTATION OF PROGRAM EXECUTION USING THE EXEC() FAMILY

AIM:

To write a C program that executes other programs from a child process using the exec() family of system calls.

PROCEDURE:

1. Create three child processes one after another using fork().
2. In the first child, run the echo command using execl() with the full path /bin/echo.
3. In the second child, run the ls command using execlp(), which searches the PATH.
4. In the third child, run the wc command using execvp() with an argument array.
5. If exec() returns, print an error because exec() returns only on failure.
6. In the parent, wait for each child before creating the next one.

PROGRAM:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char *args[] = {"wc", "-l", "names.txt", NULL};

    if (fork() == 0) {
        printf("Child 1 using execl():\n");
        fflush(stdout);
        execl("/bin/echo", "echo", "Hello from execl", NULL);
        perror("execl");
    }
    wait(NULL);

    if (fork() == 0) {
        printf("Child 2 using execlp():\n");
        fflush(stdout);
        execlp("ls", "ls", "-1", NULL);
        perror("execlp");
    }
    wait(NULL);

    if (fork() == 0) {
        printf("Child 3 using execvp():\n");
        fflush(stdout);
        execvp(args[0], args);
        perror("execvp");
    }
    wait(NULL);
    printf("Parent: all children finished\n");
    return 0;
}
```

OUTPUT:

```
Child 1 using execl():
Hello from execl
Child 2 using execlp():
names.txt
prog
prog.c
Child 3 using execvp():
3 names.txt
Parent: all children finished
```

RESULT:

Thus the C program to execute programs using the exec() family of system calls was executed successfully.

Exp. No: 18

IMPLEMENTATION OF PARENT–CHILD PROCESS SYNCHRONIZATION USING WAIT()

AIM:

To write a C program that synchronises a parent and a child process using the wait() system call.

PROCEDURE:

1. Create a child process using fork().
2. In the child, print the numbers 1 to 5 and exit with status 7 using exit().
3. In the parent, call wait() so that the parent is blocked until the child terminates.
4. After wait() returns, check WIFEXITED() and display the exit status using WEXITSTATUS().
5. Display a message from the parent after the child has finished.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int status, i;
    pid_t pid = fork();

    if (pid == 0) {
        for (i = 1; i <= 5; i++)
            printf("Child printing %d\n", i);
        exit(7);
    } else {
        printf("Parent waiting for child %d...\n", pid);
        fflush(stdout);
        pid_t done = wait(&status);
        if (WIFEXITED(status))
            printf("Child %d finished with exit status %d\n", done,
WEXITSTATUS(status));
        printf("Parent continues after the child\n");
    }
    return 0;
}
```

OUTPUT:

```
Parent waiting for child 1806...
Child printing 1
Child printing 2
Child printing 3
Child printing 4
Child printing 5
Child 1806 finished with exit status 7
Parent continues after the child
```

RESULT:

Thus the C program to synchronise parent and child processes using wait() was executed successfully.

Exp. No: 19

IMPLEMENTATION OF ORPHAN AND ZOMBIE PROCESS IDENTIFICATION AND HANDLING

AIM:

To write a C program that creates a zombie process and an orphan process, identifies them and handles them.

PROCEDURE:

1. Zombie: create a child that exits immediately while the parent sleeps without calling wait().
2. Display the process table using ps; the child is shown with state Z (zombie).
3. Handle the zombie by calling wait() in the parent, which removes its entry.
4. Orphan: create a process whose parent exits before it.
5. The orphan child sleeps, then prints its new parent PID, showing that it was adopted by another process (init or a sub-reaper).

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char cmd[100];

    /* ----- Zombie process ----- */
    pid = fork();
    if (pid == 0) {
        printf("Zombie demo: child %d exits now\n", getpid());
        exit(0);
    }
    sleep(1);
    sprintf(cmd, "ps -o pid,ppid,stat,comm -p %d", pid);
    printf("Process status before wait():\n");
    fflush(stdout);
    system(cmd);
    wait(NULL);
    printf("Parent called wait(): zombie %d removed\n\n", pid);

    /* ----- Orphan process ----- */
    pid = fork();
    if (pid == 0) {
        if (fork() == 0) {
            printf("Orphan demo: child %d, parent %d\n", getpid(), getppid());
            fflush(stdout);
            sleep(2);
            printf("After parent exit: child %d adopted by %d\n", getpid(), getppid());
            exit(0);
        }
        sleep(1);
        printf("Parent %d exits, leaving its child as an orphan\n", getpid());
        exit(0);
    }
```

```
    wait(NULL);
    sleep(3);
    return 0;
}
```

OUTPUT:

```
Zombie demo: child 1813 exits now
Process status before wait():
  PID  PPID STAT COMMAND
 1813  1812 Z+   prog
Parent called wait(): zombie 1813 removed

Orphan demo: child 1817, parent 1816
Parent 1816 exits, leaving its child as an orphan
After parent exit: child 1817 adopted by 1
```

RESULT:

Thus the zombie and orphan processes were created, identified and handled successfully.

Exp. No: 20

IMPLEMENTATION OF FCFS CPU SCHEDULING ALGORITHM

AIM:

To write a C program to implement the First Come First Served (FCFS) CPU scheduling algorithm and find the average waiting time and turnaround time.

PROCEDURE:

1. Read the number of processes and the arrival time and burst time of each process.
2. Sort the processes in the order of their arrival time.
3. Start with time = 0. For each process, if the CPU is idle before it arrives, move the time to its arrival time.
4. Completion time = current time + burst time.
5. Turnaround time = completion time - arrival time; waiting time = turnaround time - burst time.
6. Display the Gantt chart, the table and the average waiting and turnaround times.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, j, at[10], bt[10], ct[10], tat[10], wt[10], p[10], t;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time of P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        p[i] = i + 1;
    }
    /* sort by arrival time */
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (at[j] > at[j + 1]) {
                t = at[j]; at[j] = at[j + 1]; at[j + 1] = t;
                t = bt[j]; bt[j] = bt[j + 1]; bt[j + 1] = t;
                t = p[j];  p[j]  = p[j + 1];  p[j + 1]  = t;
            }
    t = 0;
    printf("\nGantt Chart: ");
    for (i = 0; i < n; i++) {
        if (t < at[i]) t = at[i];
        printf("| P%d (%d-%d) ", p[i], t, t + bt[i]);
        t += bt[i];
        ct[i] = t;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        twt += wt[i];
        ttat += tat[i];
    }
    printf("|\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
```

```
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
4
0 5
1 3
2 8
3 6
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time and burst time of P1: 0 5
Enter arrival time and burst time of P2: 1 3
Enter arrival time and burst time of P3: 2 8
Enter arrival time and burst time of P4: 3 6

Gantt Chart: | P1 (0-5) | P2 (5-8) | P3 (8-16) | P4 (16-22) |

Process AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      1       3       8       7       4
P3      2       8       16      14      6
P4      3       6       22      19      13

Average Waiting Time    = 5.75
Average Turnaround Time = 11.25
```

RESULT:

Thus the C program to implement FCFS CPU scheduling was executed successfully and the average waiting time and turnaround time were calculated.

Exp. No: 21

IMPLEMENTATION OF NON-PREEMPTIVE SJF SCHEDULING ALGORITHM

AIM:

To write a C program to implement the non-preemptive Shortest Job First (SJF) CPU scheduling algorithm.

PROCEDURE:

1. Read the number of processes and the arrival time and burst time of each process.
2. Set time = 0 and mark all processes as not completed.
3. Among the processes that have arrived and are not completed, select the one with the smallest burst time.
4. If no process has arrived, increase the time by 1.
5. Run the selected process to completion and compute its completion, turnaround and waiting times.
6. Repeat until all processes are completed, then display the results and averages.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, at[10], bt[10], ct[10], tat[10], wt[10], done[10] = {0};
    int t = 0, completed = 0, k;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time of P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    printf("\nGantt Chart: ");
    while (completed < n) {
        k = -1;
        for (i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (k == -1 || bt[i] < bt[k]))
                k = i;
        if (k == -1) { t++; continue; }
        printf("| P%d (%d-%d) ", k + 1, t, t + bt[k]);
        t += bt[k];
        ct[k] = t;
        tat[k] = ct[k] - at[k];
        wt[k] = tat[k] - bt[k];
        done[k] = 1;
        completed++;
    }
    printf("|\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        twt += wt[i];
        ttat += tat[i];
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);
```

```
    return 0;
}
```

INPUT:

```
4
0 7
2 4
4 1
5 4
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time and burst time of P1: 0 7
Enter arrival time and burst time of P2: 2 4
Enter arrival time and burst time of P3: 4 1
Enter arrival time and burst time of P4: 5 4

Gantt Chart: | P1 (0-7) | P3 (7-8) | P2 (8-12) | P4 (12-16) |

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      2       4       12      10      6
P3      4       1       8       4       3
P4      5       4       16      11      7

Average Waiting Time    = 4.00
Average Turnaround Time = 8.00
```

RESULT:

Thus the C program to implement non-preemptive SJF scheduling was executed successfully.

Exp. No: 22

IMPLEMENTATION OF PREEMPTIVE SJF (SRTF) SCHEDULING ALGORITHM

AIM:

To write a C program to implement the preemptive Shortest Job First, also called Shortest Remaining Time First (SRTF), scheduling algorithm.

PROCEDURE:

1. Read the number of processes and the arrival time and burst time of each process.
2. Copy the burst times into a remaining-time array.
3. At every time unit, select the arrived process with the smallest remaining time.
4. Execute the selected process for one time unit and decrease its remaining time.
5. When the remaining time becomes 0, record the completion time.
6. Repeat until all processes complete; compute turnaround and waiting times and the averages.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, at[10], bt[10], rt[10], ct[10], tat[10], wt[10];
    int t = 0, completed = 0, k, prev = -1;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time of P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("\nExecution order: ");
    while (completed < n) {
        k = -1;
        for (i = 0; i < n; i++)
            if (at[i] <= t && rt[i] > 0 && (k == -1 || rt[i] < rt[k]))
                k = i;
        if (k == -1) { t++; continue; }
        if (k != prev) { printf("P%d(t=%d) ", k + 1, t); prev = k; }
        rt[k]--;
        t++;
        if (rt[k] == 0) {
            ct[k] = t;
            completed++;
        }
    }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        twt += wt[i];
        ttat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
```

```
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
4
0 8
1 4
2 9
3 5
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time and burst time of P1: 0 8
Enter arrival time and burst time of P2: 1 4
Enter arrival time and burst time of P3: 2 9
Enter arrival time and burst time of P4: 3 5

Execution order: P1(t=0) P2(t=1) P4(t=5) P1(t=10) P3(t=17)

Process AT      BT      CT      TAT     WT
P1      0       8       17      17      9
P2      1       4       5       4       0
P3      2       9       26      24      15
P4      3       5       10      7       2

Average Waiting Time    = 6.50
Average Turnaround Time = 13.00
```

RESULT:

Thus the C program to implement SRTF (preemptive SJF) scheduling was executed successfully.

Exp. No: 23

IMPLEMENTATION OF NON-PREEMPTIVE PRIORITY SCHEDULING ALGORITHM

AIM:

To write a C program to implement the non-preemptive priority CPU scheduling algorithm (lower number means higher priority).

PROCEDURE:

1. Read the number of processes and the arrival time, burst time and priority of each process.
2. Set time = 0.
3. Among the arrived processes that are not completed, select the one with the highest priority (smallest priority number).
4. Run the selected process until it completes and record its completion time.
5. Repeat until all processes are completed.
6. Compute turnaround and waiting times, display the table and the averages.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, at[10], bt[10], pr[10], ct[10], tat[10], wt[10], done[10] = {0};
    int t = 0, completed = 0, k;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time, burst time and priority of P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }
    printf("\nGantt Chart: ");
    while (completed < n) {
        k = -1;
        for (i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (k == -1 || pr[i] < pr[k]))
                k = i;
        if (k == -1) { t++; continue; }
        printf("| P%d (%d-%d) ", k + 1, t, t + bt[k]);
        t += bt[k];
        ct[k] = t;
        done[k] = 1;
        completed++;
    }
    printf("|\n\nProcess\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        twt += wt[i];
        ttat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], ct[i],
tat[i], wt[i]);
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);
```

```
    return 0;
}
```

INPUT:

```
4
0 5 3
1 3 1
2 8 4
3 6 2
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time, burst time and priority of P1: 0 5 3
Enter arrival time, burst time and priority of P2: 1 3 1
Enter arrival time, burst time and priority of P3: 2 8 4
Enter arrival time, burst time and priority of P4: 3 6 2

Gantt Chart: | P1 (0-5) | P2 (5-8) | P4 (8-14) | P3 (14-22) |

Process AT      BT      PRI     CT      TAT     WT
P1      0       5       3       5       5       0
P2      1       3       1       8       7       4
P3      2       8       4       22      20      12
P4      3       6       2       14      11      5

Average Waiting Time    = 5.25
Average Turnaround Time = 10.75
```

RESULT:

Thus the C program to implement non-preemptive priority scheduling was executed successfully.

Exp. No: 24

IMPLEMENTATION OF PREEMPTIVE PRIORITY SCHEDULING ALGORITHM

AIM:

To write a C program to implement the preemptive priority CPU scheduling algorithm (lower number means higher priority).

PROCEDURE:

1. Read the number of processes and the arrival time, burst time and priority of each process.
2. Copy the burst times into a remaining-time array.
3. At every time unit, select the arrived process with the highest priority that still has remaining time.
4. Run it for one time unit; a newly arrived process with higher priority preempts the running process.
5. When the remaining time of a process becomes 0, record its completion time.
6. Compute turnaround and waiting times, display the table and the averages.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, at[10], bt[10], pr[10], rt[10], ct[10], tat, wt;
    int t = 0, completed = 0, k, prev = -1;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time, burst time and priority of P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
    }
    printf("\nExecution order: ");
    while (completed < n) {
        k = -1;
        for (i = 0; i < n; i++)
            if (at[i] <= t && rt[i] > 0 && (k == -1 || pr[i] < pr[k]))
                k = i;
        if (k == -1) { t++; continue; }
        if (k != prev) { printf("P%d(t=%d) ", k + 1, t); prev = k; }
        rt[k]--;
        t++;
        if (rt[k] == 0) { ct[k] = t; completed++; }
    }
    printf("\n\nProcess\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        tat = ct[i] - at[i];
        wt = tat - bt[i];
        twt += wt;
        ttat += tat;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], ct[i], tat,
wt);
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
```

```
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
4
0 5 3
1 3 1
2 8 4
3 6 2
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time, burst time and priority of P1: 0 5 3
Enter arrival time, burst time and priority of P2: 1 3 1
Enter arrival time, burst time and priority of P3: 2 8 4
Enter arrival time, burst time and priority of P4: 3 6 2

Execution order: P1(t=0) P2(t=1) P4(t=4) P1(t=10) P3(t=14)

Process AT      BT      PRI     CT      TAT     WT
P1      0       5       3       14      14      9
P2      1       3       1       4       3       0
P3      2       8       4       22      20      12
P4      3       6       2       10      7       1

Average Waiting Time    = 5.50
Average Turnaround Time = 11.00
```

RESULT:

Thus the C program to implement preemptive priority scheduling was executed successfully.

Exp. No: 25

IMPLEMENTATION OF ROUND ROBIN CPU SCHEDULING ALGORITHM

AIM:

To write a C program to implement the Round Robin CPU scheduling algorithm with a given time quantum.

PROCEDURE:

1. Read the number of processes, the burst time of each process and the time quantum (all processes arrive at time 0).
2. Copy the burst times into a remaining-time array.
3. Visit the processes in circular order. If the remaining time is more than the quantum, run the process for one quantum.
4. Otherwise run it for its remaining time and record its completion time.
5. Repeat until all processes are completed.
6. Waiting time = completion time - burst time; display the table and the averages.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, q, bt[10], rt[10], ct[10], t = 0, left;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d", &q);

    left = n;
    printf("\nGantt Chart: ");
    while (left > 0) {
        for (i = 0; i < n; i++) {
            if (rt[i] == 0) continue;
            int run = rt[i] > q ? q : rt[i];
            printf("| P%d (%d-%d) ", i + 1, t, t + run);
            t += run;
            rt[i] -= run;
            if (rt[i] == 0) { ct[i] = t; left--; }
        }
    }
    printf("|\n\nProcess\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        int tat = ct[i], wt = ct[i] - bt[i];
        twt += wt;
        ttat += tat;
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
```

```
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
4
10
4
5
3
2
```

OUTPUT:

```
Enter number of processes: 4
Enter burst time of P1: 10
Enter burst time of P2: 4
Enter burst time of P3: 5
Enter burst time of P4: 3
Enter time quantum: 2

Gantt Chart: | P1 (0-2) | P2 (2-4) | P3 (4-6) | P4 (6-8) | P1 (8-10) | P2 (10-12) | P3
(12-14) | P4 (14-15) | P1 (15-17) | P3 (17-18) | P1 (18-20) | P1 (20-22) |

Process BT      CT      TAT     WT
P1      10      22      22      12
P2      4       12      12      8
P3      5       18      18      13
P4      3       15      15      12

Average Waiting Time    = 11.25
Average Turnaround Time = 16.75
```

RESULT:

Thus the C program to implement Round Robin scheduling was executed successfully.

Exp. No: 26

IMPLEMENTATION OF MULTILEVEL QUEUE SCHEDULING ALGORITHM

AIM:

To write a C program to implement Multilevel Queue scheduling with a system (foreground) queue and a user (background) queue.

PROCEDURE:

1. Read the number of processes, the burst time of each process and its queue (0 = system, 1 = user).
2. All processes arrive at time 0.
3. The system queue has absolute priority over the user queue.
4. Schedule all processes of the system queue first in FCFS order.
5. Then schedule the processes of the user queue in FCFS order.
6. Compute waiting and turnaround times and display the results.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, level, bt[10], qn[10], wt[10], tat[10], t = 0;
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time and queue (0-system, 1-user) of P%d: ", i + 1);
        scanf("%d %d", &bt[i], &qn[i]);
    }
    printf("\nGantt Chart: ");
    for (level = 0; level <= 1; level++) {
        for (i = 0; i < n; i++) {
            if (qn[i] != level) continue;
            wt[i] = t;
            printf("| P%d (%d-%d) ", i + 1, t, t + bt[i]);
            t += bt[i];
            tat[i] = t;
        }
    }
    printf("|\n\nProcess\tQueue\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%s\t%d\t%d\t%d\n", i + 1, qn[i] == 0 ? "System" : "User", bt[i],
wt[i], tat[i]);
        twt += wt[i];
        ttat += tat[i];
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
5
4 1
3 0
5 1
2 0
6 0
```

OUTPUT:

```
Enter number of processes: 5
Enter burst time and queue (0-system, 1-user) of P1: 4 1
Enter burst time and queue (0-system, 1-user) of P2: 3 0
Enter burst time and queue (0-system, 1-user) of P3: 5 1
Enter burst time and queue (0-system, 1-user) of P4: 2 0
Enter burst time and queue (0-system, 1-user) of P5: 6 0

Gantt Chart: | P2 (0-3) | P4 (3-5) | P5 (5-11) | P1 (11-15) | P3 (15-20) |

Process Queue   BT      WT      TAT
P1      User    4       11      15
P2      System  3       0       3
P3      User    5       15      20
P4      System  2       3       5
P5      System  6       5       11

Average Waiting Time    = 6.80
Average Turnaround Time = 10.80
```

RESULT:

Thus the C program to implement Multilevel Queue scheduling was executed successfully.

Exp. No: 27

IMPLEMENTATION OF MULTILEVEL FEEDBACK QUEUE SCHEDULING ALGORITHM

AIM:

To write a C program to implement Multilevel Feedback Queue scheduling with three queues (Q0: RR quantum 2, Q1: RR quantum 4, Q2: FCFS).

PROCEDURE:

1. Read the number of processes and the burst time of each process (all arrive at time 0).
2. Place all processes in Q0.
3. Run each process in Q0 for at most 2 units; if it does not finish, move it down to Q1.
4. Run each process in Q1 for at most 4 units; if it does not finish, move it down to Q2.
5. Run the processes in Q2 in FCFS order until completion.
6. Record the completion times and compute waiting and turnaround times.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i, level, bt[10], rt[10], ct[10], t = 0;
    int quantum[3] = {2, 4, 1000};
    float twt = 0, ttat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    for (level = 0; level < 3; level++) {
        printf("\nQueue Q%d: ", level);
        for (i = 0; i < n; i++) {
            if (rt[i] == 0) continue;
            int run = rt[i] < quantum[level] ? rt[i] : quantum[level];
            printf("P%d(%d-%d) ", i + 1, t, t + run);
            t += run;
            rt[i] -= run;
            if (rt[i] == 0) ct[i] = t;
        }
    }
    printf("\n\nProcess\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, bt[i], ct[i], ct[i], ct[i] - bt[i]);
        twt += ct[i] - bt[i];
        ttat += ct[i];
    }
    printf("\nAverage Waiting Time    = %.2f\n", twt / n);
    printf("Average Turnaround Time = %.2f\n", ttat / n);
    return 0;
}
```

INPUT:

```
4
3
7
12
5
```

OUTPUT:

```
Enter number of processes: 4
Enter burst time of P1: 3
Enter burst time of P2: 7
Enter burst time of P3: 12
Enter burst time of P4: 5

Queue Q0: P1(0-2) P2(2-4) P3(4-6) P4(6-8)
Queue Q1: P1(8-9) P2(9-13) P3(13-17) P4(17-20)
Queue Q2: P2(20-21) P3(21-27)

Process BT      CT      TAT     WT
P1      3       9       9       6
P2      7       21      21      14
P3      12      27      27      15
P4      5       20      20      15

Average Waiting Time    = 12.50
Average Turnaround Time = 19.25
```

RESULT:

Thus the C program to implement Multilevel Feedback Queue scheduling was executed successfully.

Exp. No: 28

PERFORMANCE COMPARISON OF CPU SCHEDULING ALGORITHMS USING WAITING TIME, TURNAROUND TIME, AND RESPONSE TIME

AIM:

To write a C program that compares FCFS, SJF and Round Robin scheduling using average waiting time, turnaround time and response time.

PROCEDURE:

1. Read the number of processes, their burst times (arrival time 0) and the time quantum.
2. FCFS: run the processes in the given order and compute WT, TAT and RT.
3. SJF: sort the processes by burst time and compute WT, TAT and RT.
4. Round Robin: simulate the time quantum and compute WT, TAT and RT (RT is the time of the first run).
5. Display the averages of the three algorithms in a table.
6. Identify the algorithm with the lowest average waiting time.

PROGRAM:

```
#include <stdio.h>

int n, bt[10];

void fcfs(int order[], float *wt, float *tat, float *rt)
{
    int i, t = 0;
    *wt = *tat = *rt = 0;
    for (i = 0; i < n; i++) {
        int p = order[i];
        *wt += t;
        *rt += t;
        t += bt[p];
        *tat += t;
    }
    *wt /= n; *tat /= n; *rt /= n;
}

void rr(int q, float *wt, float *tat, float *rt)
{
    int rem[10], first[10], i, t = 0, left = n;
    *wt = *tat = *rt = 0;
    for (i = 0; i < n; i++) { rem[i] = bt[i]; first[i] = -1; }
    while (left > 0)
        for (i = 0; i < n; i++) {
            if (rem[i] == 0) continue;
            if (first[i] == -1) first[i] = t;
            int run = rem[i] > q ? q : rem[i];
            t += run;
            rem[i] -= run;
            if (rem[i] == 0) {
                left--;
                *tat += t;
                *wt += t - bt[i];
                *rt += first[i];
            }
        }
```

```
    *wt /= n; *tat /= n; *rt /= n;
}

int main()
{
    int i, j, q, order[10], t;
    float w[3], ta[3], r[3];
    char *name[3] = {"FCFS", "SJF", "Round Robin"};

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
        order[i] = i;
    }
    printf("Enter time quantum for Round Robin: ");
    scanf("%d", &q);

    fcfs(order, &w[0], &ta[0], &r[0]);
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (bt[order[j]] > bt[order[j + 1]]) {
                t = order[j]; order[j] = order[j + 1]; order[j + 1] = t;
            }
    fcfs(order, &w[1], &ta[1], &r[1]);
    rr(q, &w[2], &ta[2], &r[2]);

    printf("\nAlgorithm\tAvg WT\tAvg TAT\tAvg RT\n");
    for (i = 0; i < 3; i++)
        printf("%-12s\t%.2f\t%.2f\t%.2f\n", name[i], w[i], ta[i], r[i]);
    j = 0;
    for (i = 1; i < 3; i++)
        if (w[i] < w[j]) j = i;
    printf("\nLowest average waiting time : %s\n", name[j]);
    j = 0;
    for (i = 1; i < 3; i++)
        if (r[i] < r[j]) j = i;
    printf("Lowest average response time: %s\n", name[j]);
    return 0;
}
```

INPUT:

```
4
10
4
6
2
3
```

OUTPUT:

```
Enter number of processes: 4
Enter burst time of P1: 10
Enter burst time of P2: 4
Enter burst time of P3: 6
Enter burst time of P4: 2
Enter time quantum for Round Robin: 3

Algorithm       Avg WT  Avg TAT Avg RT
FCFS            11.00   16.50   11.00
```

```
SJF             5.00    10.50   5.00
Round Robin     11.00   16.50   4.50

Lowest average waiting time : SJF
Lowest average response time: Round Robin
```

RESULT:

Thus the C program to compare the performance of FCFS, SJF and Round Robin scheduling was executed successfully.

Exp. No: 29

IMPLEMENTATION OF INTER-PROCESS COMMUNICATION USING UNNAMED PIPES

AIM:

To write a C program to implement inter-process communication between a parent and a child process using an unnamed pipe.

PROCEDURE:

1. Create a pipe using pipe(fd); fd[0] is the read end and fd[1] is the write end.
2. Create a child process using fork().
3. In the parent, close the read end, write a message into fd[1] and close it.
4. In the child, close the write end, read the message from fd[0] and display it.
5. The parent waits for the child to finish.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    char msg[] = "Hello child, this is your parent";
    char buf[100];

    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    if (fork() == 0) {
        close(fd[1]);
        int n = read(fd[0], buf, sizeof(buf));
        buf[n] = '\0';
        printf("Child received  : %s\n", buf);
        close(fd[0]);
    } else {
        close(fd[0]);
        printf("Parent sending  : %s\n", msg);
        fflush(stdout);
        write(fd[1], msg, strlen(msg));
        close(fd[1]);
        wait(NULL);
        printf("Communication through pipe completed\n");
    }
    return 0;
}
```

OUTPUT:

```
Parent sending  : Hello child, this is your parent
Child received  : Hello child, this is your parent
Communication through pipe completed
```

RESULT:

Thus inter-process communication using an unnamed pipe was implemented successfully.

Exp. No: 30

IMPLEMENTATION OF INTER-PROCESS COMMUNICATION USING NAMED PIPES (FIFOS)

AIM:

To write a C program to implement inter-process communication between two unrelated processes using a named pipe (FIFO).

PROCEDURE:

1. Create a named pipe in the file system using mkfifo().
2. Create a child process using fork() to act as the reader.
3. The writer (parent) opens the FIFO in write mode and writes messages into it.
4. The reader (child) opens the FIFO in read mode, reads the messages and displays them.
5. Close the FIFO in both processes and remove it using unlink().

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main()
{
    char *fifo = "myfifo";
    char buf[100];

    mkfifo(fifo, 0666);
    printf("Named pipe '%s' created\n", fifo);
    fflush(stdout);

    if (fork() == 0) {               /* reader */
        int fd = open(fifo, O_RDONLY);
        int n = read(fd, buf, sizeof(buf) - 1);
        buf[n] = '\0';
        printf("Reader received: %s\n", buf);
        close(fd);
        return 0;
    }
    int fd = open(fifo, O_WRONLY);     /* writer */
    char msg[] = "Data sent through FIFO";
    printf("Writer sending : %s\n", msg);
    fflush(stdout);
    write(fd, msg, strlen(msg));
    close(fd);
    wait(NULL);
    unlink(fifo);
    printf("FIFO removed\n");
    return 0;
}
```

OUTPUT:

```
Named pipe 'myfifo' created
```

```
Writer sending : Data sent through FIFO
Reader received: Data sent through FIFO
FIFO removed
```

RESULT:

Thus inter-process communication using a named pipe (FIFO) was implemented successfully.

Exp. No: 31

IMPLEMENTATION OF SHARED MEMORY COMMUNICATION BETWEEN PROCESSES

AIM:

To write a C program to implement communication between processes using shared memory.

PROCEDURE:

1. Create a shared memory segment using shmget() with IPC_PRIVATE.
2. Create a child process using fork().
3. The child attaches the segment using shmat(), writes a message into it and detaches.
4. The parent waits for the child, attaches the segment and reads the message.
5. The parent detaches the segment using shmdt() and removes it using shmctl() with IPC_RMID.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>

int main()
{
    int shmid = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget"); return 1; }
    printf("Shared memory created with id %d\n", shmid);
    fflush(stdout);

    if (fork() == 0) {
        char *mem = (char *)shmat(shmid, NULL, 0);
        strcpy(mem, "Hello from child through shared memory");
        printf("Child wrote : %s\n", mem);
        shmdt(mem);
        return 0;
    }
    wait(NULL);
    char *mem = (char *)shmat(shmid, NULL, 0);
    printf("Parent read : %s\n", mem);
    shmdt(mem);
    shmctl(shmid, IPC_RMID, NULL);
    printf("Shared memory removed\n");
    return 0;
}
```

OUTPUT:

```
Shared memory created with id 2
Child wrote : Hello from child through shared memory
Parent read : Hello from child through shared memory
Shared memory removed
```

RESULT:

Thus communication between processes using shared memory was implemented successfully.

Exp. No: 32

IMPLEMENTATION OF MESSAGE QUEUE COMMUNICATION BETWEEN PROCESSES

AIM:

To write a C program to implement communication between processes using a message queue.

PROCEDURE:

1. Define a message structure with a message type and a text field.
2. Create a message queue using msgget().
3. Create a child process using fork(). The child (sender) sends three messages using msgsnd().
4. The parent (receiver) receives the messages using msgrcv() and displays them.
5. Remove the message queue using msgctl() with IPC_RMID.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/wait.h>

struct message {
    long type;
    char text[100];
};

int main()
{
    struct message m;
    int i, qid = msgget(IPC_PRIVATE, IPC_CREAT | 0666);
    char *data[3] = {"Message one", "Message two", "Message three"};

    printf("Message queue created with id %d\n", qid);
    fflush(stdout);
    if (fork() == 0) {
        for (i = 0; i < 3; i++) {
            m.type = 1;
            strcpy(m.text, data[i]);
            msgsnd(qid, &m, sizeof(m.text), 0);
            printf("Sender   : sent \"%s\"\n", m.text);
            fflush(stdout);
        }
        return 0;
    }
    wait(NULL);
    for (i = 0; i < 3; i++) {
        msgrcv(qid, &m, sizeof(m.text), 1, 0);
        printf("Receiver : got  \"%s\"\n", m.text);
    }
    msgctl(qid, IPC_RMID, NULL);
    printf("Message queue removed\n");
    return 0;
}
```

OUTPUT:

```
Message queue created with id 2
Sender   : sent "Message one"
Sender   : sent "Message two"
Sender   : sent "Message three"
Receiver : got  "Message one"
Receiver : got  "Message two"
Receiver : got  "Message three"
Message queue removed
```

RESULT:

Thus communication between processes using a message queue was implemented successfully.

Exp. No: 33

IMPLEMENTATION OF SIGNAL-BASED PROCESS COMMUNICATION

AIM:

To write a C program to implement communication between processes using signals.

PROCEDURE:

1. Write a signal handler function for SIGUSR1 and SIGUSR2.
2. Register the handler using signal().
3. Create a child process using fork(); the child waits for signals using pause().
4. The parent sends SIGUSR1 and SIGUSR2 to the child using kill().
5. The child displays a message for each signal received.
6. Finally the parent sends SIGTERM to terminate the child and waits for it.

PROGRAM:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void handler(int sig)
{
    if (sig == SIGUSR1)
        printf("Child received SIGUSR1: start the task\n");
    else if (sig == SIGUSR2)
        printf("Child received SIGUSR2: stop the task\n");
    fflush(stdout);
}

int main()
{
    int status;
    signal(SIGUSR1, handler);
    signal(SIGUSR2, handler);

    pid_t pid = fork();
    if (pid == 0) {
        while (1)
            pause();
    }
    sleep(1);
    printf("Parent sending SIGUSR1 to child %d\n", pid);
    fflush(stdout);
    kill(pid, SIGUSR1);
    sleep(1);
    printf("Parent sending SIGUSR2 to child %d\n", pid);
    fflush(stdout);
    kill(pid, SIGUSR2);
    sleep(1);
    printf("Parent sending SIGTERM to child %d\n", pid);
    kill(pid, SIGTERM);
    wait(&status);
    if (WIFSIGNALED(status))
        printf("Child terminated by signal %d\n", WTERMSIG(status));
    return 0;
}
```

OUTPUT:

```
Parent sending SIGUSR1 to child 1908
Child received SIGUSR1: start the task
Parent sending SIGUSR2 to child 1908
Child received SIGUSR2: stop the task
Parent sending SIGTERM to child 1908
Child terminated by signal 15
```

RESULT:

Thus communication between processes using signals was implemented successfully.

Exp. No: 34

IMPLEMENTATION OF MULTITHREADED APPLICATIONS USING POSIX THREADS

AIM:

To write a multithreaded C program using POSIX threads that finds the sum of an array by dividing the work among threads.

PROCEDURE:

1. Declare a global array of 12 numbers and an array to hold the partial sums.
2. Write a thread function that adds the elements of its own part of the array.
3. Create 3 threads using pthread_create(), passing each thread its index.
4. Wait for all threads using pthread_join().
5. Add the partial sums to get the total and display it.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>

#define N 12
#define T 3
int a[N] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
int partial[T];

void *sum_part(void *arg)
{
    int id = *(int *)arg, i;
    int start = id * (N / T), end = start + N / T;
    partial[id] = 0;
    for (i = start; i < end; i++)
        partial[id] += a[i];
    printf("Thread %d: sum of a[%d..%d] = %d\n", id, start, end - 1, partial[id]);
    return NULL;
}

int main()
{
    pthread_t th[T];
    int id[T], i, total = 0;

    for (i = 0; i < T; i++) {
        id[i] = i;
        pthread_create(&th[i], NULL, sum_part, &id[i]);
    }
    for (i = 0; i < T; i++)
        pthread_join(th[i], NULL);
    for (i = 0; i < T; i++)
        total += partial[i];
    printf("Total sum = %d\n", total);
    return 0;
}
```

OUTPUT:

```
Thread 1: sum of a[4..7] = 26
```

```
Thread 2: sum of a[8..11] = 42
Thread 0: sum of a[0..3] = 10
Total sum = 78
```

RESULT:

Thus the multithreaded application using POSIX threads was implemented successfully.

Exp. No: 35

IMPLEMENTATION OF THREAD SYNCHRONIZATION USING MUTEX LOCKS

AIM:

To write a C program that synchronises threads using mutex locks and shows the race condition that occurs without them.

PROCEDURE:

1. Declare a shared counter and a mutex.
2. Write a thread function that increments the counter 1,000,000 times without locking.
3. Write a second thread function that locks the mutex before each increment and unlocks it after.
4. Run two threads with the unsynchronised function and display the final counter value.
5. Reset the counter, run two threads with the mutex-protected function and display the result.
6. Compare the two results with the expected value 2,000,000.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>

#define LOOPS 1000000
long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *no_lock(void *arg)
{
    for (int i = 0; i < LOOPS; i++)
        counter++;
    return NULL;
}

void *with_lock(void *arg)
{
    for (int i = 0; i < LOOPS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void run(void *(*fn)(void *), char *title)
{
    pthread_t t1, t2;
    counter = 0;
    pthread_create(&t1, NULL, fn, NULL);
    pthread_create(&t2, NULL, fn, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("%-15s: counter = %ld (expected %d)\n", title, counter, 2 * LOOPS);
}

int main()
```

```
{
    run(no_lock, "Without mutex");
    run(with_lock, "With mutex");
    pthread_mutex_destroy(&lock);
    return 0;
}
```

OUTPUT:

```
Without mutex  : counter = 1242896 (expected 2000000)
With mutex     : counter = 2000000 (expected 2000000)
```

RESULT:

Thus thread synchronisation using mutex locks was implemented successfully. Without the mutex the counter value was wrong because of the race condition; with the mutex it was correct.

Exp. No: 36

IMPLEMENTATION OF SEMAPHORE-BASED SYNCHRONIZATION FOR SHARED RESOURCES

AIM:

To write a C program that uses a counting semaphore to control access to a shared resource that has a limited number of instances.

PROCEDURE:

1. Assume there are 2 printers (resource instances) and 5 users (threads).
2. Initialise a counting semaphore to 2 using sem_init().
3. Each thread calls sem_wait() before using a printer; if both printers are busy it waits.
4. The thread uses the printer for a short time and then calls sem_post() to release it.
5. Create 5 threads and wait for them to finish; destroy the semaphore.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t printers;

void *user(void *arg)
{
    int id = *(int *)arg;
    printf("User %d waiting for a printer\n", id);
    sem_wait(&printers);
    printf("User %d is PRINTING\n", id);
    usleep(300000);
    printf("User %d finished printing\n", id);
    sem_post(&printers);
    return NULL;
}

int main()
{
    pthread_t t[5];
    int id[5], i;
    sem_init(&printers, 0, 2);
    for (i = 0; i < 5; i++) {
        id[i] = i + 1;
        pthread_create(&t[i], NULL, user, &id[i]);
        usleep(50000);
    }
    for (i = 0; i < 5; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&printers);
    printf("All users served using 2 printers\n");
    return 0;
}
```

OUTPUT:

```
User 1 waiting for a printer
```

```
User 1 is PRINTING
User 2 waiting for a printer
User 2 is PRINTING
User 3 waiting for a printer
User 4 waiting for a printer
User 5 waiting for a printer
User 1 finished printing
User 3 is PRINTING
User 2 finished printing
User 4 is PRINTING
User 3 finished printing
User 5 is PRINTING
User 4 finished printing
User 5 finished printing
All users served using 2 printers
```

RESULT:

Thus semaphore-based synchronisation for a shared resource was implemented successfully; at most two users printed at the same time.

Exp. No: 37

IMPLEMENTATION OF THE PRODUCER–CONSUMER PROBLEM USING SEMAPHORES

AIM:

To write a C program to solve the producer-consumer problem using semaphores.

PROCEDURE:

1. Declare a buffer of size 5 with in and out indices.
2. Initialise semaphore empty = 5, semaphore full = 0 and a mutex for the buffer.
3. Producer: wait(empty), lock mutex, put an item into the buffer, unlock mutex, signal(full).
4. Consumer: wait(full), lock mutex, remove an item from the buffer, unlock mutex, signal(empty).
5. Create one producer thread and one consumer thread that produce and consume 8 items.
6. Join the threads and display the result.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
#define ITEMS 8
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg)
{
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer produced item %d at position %d\n", item, in);
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer consumed item %d from position %d\n", item, out);
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(250000);
    }
```

```
    return NULL;
}

int main()
{
    pthread_t p, c;
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("All %d items produced and consumed\n", ITEMS);
    return 0;
}
```

OUTPUT:

```
Producer produced item 1 at position 0
Consumer consumed item 1 from position 0
Producer produced item 2 at position 1
Producer produced item 3 at position 2
Consumer consumed item 2 from position 1
Producer produced item 4 at position 3
Producer produced item 5 at position 4
Consumer consumed item 3 from position 2
Producer produced item 6 at position 0
Producer produced item 7 at position 1
Producer produced item 8 at position 2
Consumer consumed item 4 from position 3
Consumer consumed item 5 from position 4
Consumer consumed item 6 from position 0
Consumer consumed item 7 from position 1
Consumer consumed item 8 from position 2
All 8 items produced and consumed
```

RESULT:

Thus the producer-consumer problem was solved using semaphores successfully.

Exp. No: 38

IMPLEMENTATION OF THE READERS–WRITERS PROBLEM USING SYNCHRONIZATION PRIMITIVES

AIM:

To write a C program to solve the readers-writers problem using semaphores and a mutex, so that many readers can read together but a writer gets exclusive access.

PROCEDURE:

1. Declare a shared variable data, a counter read_count, a mutex for read_count and a semaphore wrt initialised to 1.
2. Reader: lock mutex, increment read_count; if it is the first reader, wait(wrt); unlock mutex.
3. Reader: read the data, then lock mutex, decrement read_count; if it is the last reader, signal(wrt); unlock mutex.
4. Writer: wait(wrt), modify the data, signal(wrt).
5. Create 3 reader threads and 2 writer threads and join them.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int data = 0, read_count = 0;
sem_t wrt;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *reader(void *arg)
{
    int id = *(int *)arg;
    pthread_mutex_lock(&mutex);
    read_count++;
    if (read_count == 1) sem_wait(&wrt);
    pthread_mutex_unlock(&mutex);

    printf("Reader %d reads data = %d (readers inside = %d)\n", id, data, read_count);
    usleep(200000);

    pthread_mutex_lock(&mutex);
    read_count--;
    if (read_count == 0) sem_post(&wrt);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&wrt);
    data += 10;
    printf("Writer %d writes data = %d\n", id, data);
    usleep(200000);
    sem_post(&wrt);
    return NULL;
```

```
}

int main()
{
    pthread_t r[3], w[2];
    int rid[3] = {1, 2, 3}, wid[2] = {1, 2}, i;
    sem_init(&wrt, 0, 1);

    pthread_create(&w[0], NULL, writer, &wid[0]);
    usleep(50000);
    for (i = 0; i < 3; i++)
        pthread_create(&r[i], NULL, reader, &rid[i]);
    usleep(50000);
    pthread_create(&w[1], NULL, writer, &wid[1]);

    for (i = 0; i < 3; i++) pthread_join(r[i], NULL);
    for (i = 0; i < 2; i++) pthread_join(w[i], NULL);
    printf("Final data = %d\n", data);
    return 0;
}
```

OUTPUT:

```
Writer 1 writes data = 10
Reader 2 reads data = 10 (readers inside = 1)
Reader 3 reads data = 10 (readers inside = 2)
Reader 1 reads data = 10 (readers inside = 3)
Writer 2 writes data = 20
Final data = 20
```

RESULT:

Thus the readers-writers problem was solved using synchronisation primitives successfully.

Exp. No: 39

IMPLEMENTATION OF THE DINING PHILOSOPHERS PROBLEM USING SEMAPHORES

AIM:

To write a C program to solve the dining philosophers problem using semaphores without deadlock.

PROCEDURE:

1. Represent each of the 5 chopsticks by a binary semaphore initialised to 1.
2. Each philosopher thinks, picks up two chopsticks, eats and puts them down.
3. To avoid deadlock, the even-numbered philosophers pick up the left chopstick first and the odd-numbered philosophers pick up the right chopstick first.
4. Each philosopher eats twice.
5. Create 5 philosopher threads and join them.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
sem_t chopstick[N];

void *philosopher(void *arg)
{
    int id = *(int *)arg;
    int left = id, right = (id + 1) % N;
    int first = (id % 2 == 0) ? left : right;
    int second = (id % 2 == 0) ? right : left;

    for (int meal = 1; meal <= 2; meal++) {
        printf("Philosopher %d is thinking\n", id);
        usleep(100000);
        sem_wait(&chopstick[first]);
        sem_wait(&chopstick[second]);
        printf("Philosopher %d is eating (meal %d) with chopsticks %d and %d\n", id,
meal, left, right);
        usleep(150000);
        sem_post(&chopstick[second]);
        sem_post(&chopstick[first]);
        printf("Philosopher %d finished eating\n", id);
    }
    return NULL;
}

int main()
{
    pthread_t t[N];
    int id[N], i;
    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);
    for (i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&t[i], NULL, philosopher, &id[i]);
```

```
    }
    for (i = 0; i < N; i++)
        pthread_join(t[i], NULL);
    printf("All philosophers have eaten. No deadlock.\n");
    return 0;
}
```

OUTPUT:

```
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 4 is thinking
Philosopher 0 is eating (meal 1) with chopsticks 0 and 1
Philosopher 2 is eating (meal 1) with chopsticks 2 and 3
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 3 is eating (meal 1) with chopsticks 3 and 4
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 1 is eating (meal 1) with chopsticks 1 and 2
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 0 is eating (meal 2) with chopsticks 0 and 1
Philosopher 2 is eating (meal 2) with chopsticks 2 and 3
Philosopher 0 finished eating
Philosopher 4 is eating (meal 1) with chopsticks 4 and 0
Philosopher 2 finished eating
Philosopher 1 is eating (meal 2) with chopsticks 1 and 2
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 1 finished eating
Philosopher 3 is eating (meal 2) with chopsticks 3 and 4
Philosopher 3 finished eating
Philosopher 4 is eating (meal 2) with chopsticks 4 and 0
Philosopher 4 finished eating
All philosophers have eaten. No deadlock.
```

RESULT:

Thus the dining philosophers problem was solved using semaphores without deadlock.

Exp. No: 40

IMPLEMENTATION OF CONDITION VARIABLES FOR THREAD COORDINATION

AIM:

To write a C program that coordinates threads using condition variables.

PROCEDURE:

1. Declare a mutex, a condition variable and a shared flag ready = 0.
2. Create three worker threads. Each worker locks the mutex and waits on the condition variable while ready is 0.
3. The main thread prepares the data, sets ready = 1 and wakes all workers using pthread_cond_broadcast().
4. Each worker, after waking up, processes the data and unlocks the mutex.
5. Join all threads.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int ready = 0, data = 0;

void *worker(void *arg)
{
    int id = *(int *)arg;
    pthread_mutex_lock(&m);
    while (ready == 0) {
        printf("Worker %d waiting for data\n", id);
        pthread_cond_wait(&cond, &m);
    }
    printf("Worker %d woke up and processed data = %d\n", id, data * id);
    pthread_mutex_unlock(&m);
    return NULL;
}

int main()
{
    pthread_t t[3];
    int id[3] = {1, 2, 3}, i;
    for (i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, worker, &id[i]);
    sleep(1);

    pthread_mutex_lock(&m);
    data = 100;
    ready = 1;
    printf("Main thread: data is ready, signalling all workers\n");
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&m);

    for (i = 0; i < 3; i++)
        pthread_join(t[i], NULL);
    return 0;
```

```
}
```

OUTPUT:

```
Worker 2 waiting for data
Worker 3 waiting for data
Worker 1 waiting for data
Main thread: data is ready, signalling all workers
Worker 3 woke up and processed data = 300
Worker 1 woke up and processed data = 100
Worker 2 woke up and processed data = 200
```

RESULT:

Thus thread coordination using condition variables was implemented successfully.

Exp. No: 41

IMPLEMENTATION OF A CRITICAL SECTION SOLUTION USING PETERSON'S ALGORITHM

AIM:

To write a C program that implements a solution to the critical-section problem for two threads using Peterson's algorithm.

PROCEDURE:

1. Declare the shared variables flag[2] and turn.
2. Entry section of thread i: flag[i] = 1; turn = j; wait while (flag[j] == 1 and turn == j).
3. Critical section: increment a shared counter.
4. Exit section: flag[i] = 0.
5. Run two threads, each entering the critical section 100,000 times.
6. Display the final counter value and check it with the expected value.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>

#define LOOPS 100000
volatile int flag[2] = {0, 0};
volatile int turn = 0;
long counter = 0;

void *process(void *arg)
{
    int i = *(int *)arg, j = 1 - i;
    for (int k = 0; k < LOOPS; k++) {
        flag[i] = 1;                    /* entry section */
        turn = j;
        __sync_synchronize();           /* memory barrier */
        while (flag[j] == 1 && turn == j)
            ;
        counter++;                      /* critical section */
        __sync_synchronize();
        flag[i] = 0;                    /* exit section */
    }
    printf("Thread %d completed %d entries into the critical section\n", i, LOOPS);
    return NULL;
}

int main()
{
    pthread_t t0, t1;
    int id0 = 0, id1 = 1;
    pthread_create(&t0, NULL, process, &id0);
    pthread_create(&t1, NULL, process, &id1);
    pthread_join(t0, NULL);
    pthread_join(t1, NULL);
    printf("Final counter = %ld (expected %d)\n", counter, 2 * LOOPS);
    if (counter == 2 * LOOPS)
        printf("Mutual exclusion achieved using Peterson's algorithm\n");
    return 0;
}
```

OUTPUT:

```
Thread 1 completed 100000 entries into the critical section
Thread 0 completed 100000 entries into the critical section
Final counter = 200000 (expected 200000)
Mutual exclusion achieved using Peterson's algorithm
```

RESULT:

Thus the critical-section problem was solved using Peterson's algorithm and mutual exclusion was verified.

Exp. No: 42

IMPLEMENTATION OF DEADLOCK PREVENTION THROUGH RESOURCE ALLOCATION CONSTRAINTS

AIM:

To write a C program that prevents deadlock by breaking the circular-wait condition, making every thread acquire resources in a fixed global order.

PROCEDURE:

1. Create two resources R1 and R2 represented by mutex locks, numbered 1 and 2.
2. Thread A needs R1 then R2; thread B needs R2 then R1 (this order can cause deadlock).
3. Write a function that always locks the lower-numbered resource first, whatever order the thread asks for.
4. Both threads request their resources through this function, so a circular wait cannot form.
5. Each thread uses the resources, releases them and repeats three times.
6. Display that both threads finished without deadlock.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t res[3];    /* res[1] = R1, res[2] = R2 */

void acquire(int a, int b)
{
    int first = a < b ? a : b, second = a < b ? b : a;   /* global ordering */
    pthread_mutex_lock(&res[first]);
    pthread_mutex_lock(&res[second]);
}

void release(int a, int b)
{
    pthread_mutex_unlock(&res[a]);
    pthread_mutex_unlock(&res[b]);
}

void *threadA(void *arg)
{
    for (int i = 1; i <= 3; i++) {
        acquire(1, 2);
        printf("Thread A (needs R1, R2) holds R1 and R2 - round %d\n", i);
        usleep(50000);
        release(1, 2);
    }
    return NULL;
}

void *threadB(void *arg)
{
    for (int i = 1; i <= 3; i++) {
        acquire(2, 1);
        printf("Thread B (needs R2, R1) holds R1 and R2 - round %d\n", i);
        usleep(50000);
        release(2, 1);
```

```
    }
    return NULL;
}

int main()
{
    pthread_t a, b;
    pthread_mutex_init(&res[1], NULL);
    pthread_mutex_init(&res[2], NULL);
    printf("Resources are always acquired in the order R1 -> R2\n");
    pthread_create(&a, NULL, threadA, NULL);
    pthread_create(&b, NULL, threadB, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("Both threads completed. Deadlock prevented (no circular wait).\n");
    return 0;
}
```

OUTPUT:

```
Resources are always acquired in the order R1 -> R2
Thread A (needs R1, R2) holds R1 and R2 - round 1
Thread A (needs R1, R2) holds R1 and R2 - round 2
Thread A (needs R1, R2) holds R1 and R2 - round 3
Thread B (needs R2, R1) holds R1 and R2 - round 1
Thread B (needs R2, R1) holds R1 and R2 - round 2
Thread B (needs R2, R1) holds R1 and R2 - round 3
Both threads completed. Deadlock prevented (no circular wait).
```

RESULT:

Thus deadlock prevention by resource ordering was implemented successfully.

Exp. No: 43

IMPLEMENTATION OF THE BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE

AIM:

To write a C program to implement the Banker's algorithm for deadlock avoidance and find a safe sequence.

PROCEDURE:

1. Read the number of processes and resource types.
2. Read the Allocation matrix, the Maximum matrix and the Available vector.
3. Calculate Need = Maximum - Allocation.
4. Set Work = Available and Finish[i] = false for all processes.
5. Find a process with Finish = false and Need <= Work; add its allocation to Work and set Finish = true.
6. Repeat until no such process is found. If all processes finish, the system is in a safe state; display the safe sequence.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k, count = 0;
    int alloc[10][10], max[10][10], need[10][10], avail[10], work[10];
    int finish[10] = {0}, safe[10];

    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Maximum matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) scanf("%d", &max[i][j]);
    printf("Enter Available resources: ");
    for (j = 0; j < m; j++) { scanf("%d", &avail[j]); work[j] = avail[j]; }

    printf("\nNeed matrix:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            if (finish[i]) continue;
            for (j = 0; j < m; j++)
                if (need[i][j] > work[j]) break;
            if (j == m) {
                for (j = 0; j < m; j++) work[j] += alloc[i][j];
                finish[i] = 1;
                safe[count++] = i;
```

```
            }
        }
    }
    if (count == n) {
        printf("\nSystem is in a SAFE state.\nSafe sequence: ");
        for (i = 0; i < n; i++) printf("P%d%s", safe[i], i < n - 1 ? " -> " : "\n");
    } else
        printf("\nSystem is in an UNSAFE state.\n");
    return 0;
}
```

INPUT:

```
5 3
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
3 3 2
```

OUTPUT:

```
Enter number of processes and resources: 5 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available resources: 3 3 2

Need matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in a SAFE state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2
```

RESULT:

Thus the Banker's algorithm was implemented and the safe sequence was found successfully.

Exp. No: 44

IMPLEMENTATION OF DEADLOCK DETECTION USING A RESOURCE ALLOCATION GRAPH

AIM:

To write a C program that detects deadlock by finding a cycle in a resource allocation graph (reduced to a wait-for graph).

PROCEDURE:

1. Read the number of processes.
2. Read the wait-for graph as an adjacency matrix: wait[i][j] = 1 if process Pi is waiting for a resource held by Pj.
3. Perform a depth-first search from every process, keeping track of the processes on the current path.
4. If the search reaches a process that is already on the current path, a cycle exists.
5. A cycle in the wait-for graph means the system is deadlocked; display the processes in the cycle.
6. If no cycle is found, report that there is no deadlock.

PROGRAM:

```
#include <stdio.h>

int n, g[10][10], visited[10], onpath[10], parent[10];
int cycle_start = -1, cycle_end = -1;

int dfs(int u)
{
    visited[u] = onpath[u] = 1;
    for (int v = 0; v < n; v++) {
        if (!g[u][v]) continue;
        if (onpath[v]) { cycle_start = v; cycle_end = u; return 1; }
        if (!visited[v]) {
            parent[v] = u;
            if (dfs(v)) return 1;
        }
    }
    onpath[u] = 0;
    return 0;
}

int main()
{
    int i, j, found = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter wait-for matrix (1 if Pi waits for Pj):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) scanf("%d", &g[i][j]);

    for (i = 0; i < n && !found; i++)
        if (!visited[i]) found = dfs(i);

    if (found) {
        int path[10], len = 0, v = cycle_end;
```

```
        while (v != cycle_start) { path[len++] = v; v = parent[v]; }
        path[len++] = cycle_start;
        printf("\nDeadlock detected! Cycle: ");
        for (i = len - 1; i >= 0; i--) printf("P%d -> ", path[i]);
        printf("P%d\n", cycle_start);
    } else
        printf("\nNo cycle in the graph. No deadlock.\n");
    return 0;
}
```

INPUT:

```
4
0 1 0 0
0 0 1 0
0 0 0 1
0 1 0 0
```

OUTPUT:

```
Enter number of processes: 4
Enter wait-for matrix (1 if Pi waits for Pj):
0 1 0 0
0 0 1 0
0 0 0 1
0 1 0 0

Deadlock detected! Cycle: P1 -> P2 -> P3 -> P1
```

RESULT:

Thus deadlock detection using the resource allocation (wait-for) graph was implemented successfully.

Exp. No: 45

IMPLEMENTATION OF DEADLOCK RECOVERY THROUGH PROCESS TERMINATION AND RESOURCE PREEMPTION

AIM:

To write a C program that detects deadlock using the detection algorithm and recovers by terminating processes (pre-empting their resources) one at a time.

PROCEDURE:

1. Read the number of processes and resources, the Allocation matrix, the Request matrix and the Available vector.
2. Run the detection algorithm: Work = Available; a process can finish if Request <= Work, and then Work += Allocation.
3. Processes that cannot finish are deadlocked.
4. Recovery: choose the deadlocked process with the largest allocation as the victim and terminate it.
5. Pre-empt its resources by adding its allocation to Available and setting its allocation and request to 0.
6. Run the detection algorithm again; repeat until no deadlock remains.

PROGRAM:

```
#include <stdio.h>

int n, m, alloc[10][10], req[10][10], avail[10], killed[10];

int detect(int dead[])
{
    int work[10], finish[10], i, j, changed = 1, count = 0;
    for (j = 0; j < m; j++) work[j] = avail[j];
    for (i = 0; i < n; i++) finish[i] = killed[i];
    while (changed) {
        changed = 0;
        for (i = 0; i < n; i++) {
            if (finish[i]) continue;
            for (j = 0; j < m; j++)
                if (req[i][j] > work[j]) break;
            if (j == m) {
                for (j = 0; j < m; j++) work[j] += alloc[i][j];
                finish[i] = 1;
                changed = 1;
            }
        }
    }
    for (i = 0; i < n; i++) {
        dead[i] = !finish[i];
        count += dead[i];
    }
    return count;
}

int main()
{
    int i, j, dead[10];
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);
```

```
    printf("Enter Allocation matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Request matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &req[i][j]);
    printf("Enter Available resources: ");
    for (j = 0; j < m; j++) scanf("%d", &avail[j]);

    while (detect(dead) > 0) {
        int victim = -1, best = -1;
        printf("\nDeadlocked processes: ");
        for (i = 0; i < n; i++)
            if (dead[i]) {
                int sum = 0;
                printf("P%d ", i);
                for (j = 0; j < m; j++) sum += alloc[i][j];
                if (sum > best) { best = sum; victim = i; }
            }
        printf("\nRecovery: terminating P%d and pre-empting its resources\n", victim);
        for (j = 0; j < m; j++) {
            avail[j] += alloc[victim][j];
            alloc[victim][j] = req[victim][j] = 0;
        }
        killed[victim] = 1;
        printf("Available now: ");
        for (j = 0; j < m; j++) printf("%d ", avail[j]);
        printf("\n");
    }
    printf("\nNo deadlock. Remaining processes can complete.\n");
    return 0;
}
```

INPUT:

```
4 3
1 0 1
1 1 0
0 1 1
1 0 0
0 1 1
1 0 1
1 1 0
0 0 0
0 0 0
```

OUTPUT:

```
Enter number of processes and resources: 4 3
Enter Allocation matrix:
1 0 1
1 1 0
0 1 1
1 0 0
Enter Request matrix:
0 1 1
1 0 1
1 1 0
0 0 0
Enter Available resources: 0 0 0

Deadlocked processes: P0 P1 P2
Recovery: terminating P0 and pre-empting its resources
Available now: 1 0 1
```

```
No deadlock. Remaining processes can complete.
```

RESULT:

Thus deadlock detection and recovery through process termination and resource pre-emption were implemented successfully.

Exp. No: 46

SIMULATION OF RESOURCE ALLOCATION AND SAFE-STATE DETECTION

AIM:

To write a C program that simulates resource allocation requests and grants a request only if the system remains in a safe state.

PROCEDURE:

1. Read the number of processes and resources, the Allocation matrix, the Maximum matrix and the Available vector.
2. Compute Need = Maximum - Allocation.
3. Read a request (process number and request vector).
4. If Request > Need, report an error; if Request > Available, the process must wait.
5. Otherwise pretend to allocate: Available -= Request, Allocation += Request, Need -= Request.
6. Run the safety algorithm. If the state is safe, grant the request; otherwise roll back and deny it.
7. Repeat for each request.

PROGRAM:

```
#include <stdio.h>

int n, m, alloc[10][10], max[10][10], need[10][10], avail[10];

int is_safe()
{
    int work[10], finish[10] = {0}, i, j, k, count = 0;
    for (j = 0; j < m; j++) work[j] = avail[j];
    for (k = 0; k < n; k++)
        for (i = 0; i < n; i++) {
            if (finish[i]) continue;
            for (j = 0; j < m; j++) if (need[i][j] > work[j]) break;
            if (j == m) {
                for (j = 0; j < m; j++) work[j] += alloc[i][j];
                finish[i] = 1;
                count++;
            }
        }
    return count == n;
}

int main()
{
    int i, j, p, r[10], nreq;
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Maximum matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &max[i][j]);
    printf("Enter Available resources: ");
    for (j = 0; j < m; j++) scanf("%d", &avail[j]);
```

```
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) need[i][j] = max[i][j] -
alloc[i][j];
    printf("Initial state is %s\n", is_safe() ? "SAFE" : "UNSAFE");

    printf("Enter number of requests: ");
    scanf("%d", &nreq);
    while (nreq--) {
        printf("\nEnter process number and request vector: ");
        scanf("%d", &p);
        for (j = 0; j < m; j++) scanf("%d", &r[j]);
        int ok = 1;
        for (j = 0; j < m; j++) if (r[j] > need[p][j]) ok = 0;
        if (!ok) { printf("P%d: request exceeds its maximum claim - ERROR\n", p);
continue; }
        for (j = 0; j < m; j++) if (r[j] > avail[j]) ok = 0;
        if (!ok) { printf("P%d: resources not available - process must WAIT\n", p);
continue; }
        for (j = 0; j < m; j++) { avail[j] -= r[j]; alloc[p][j] += r[j]; need[p][j] -=
r[j]; }
        if (is_safe())
            printf("P%d: request GRANTED (system remains safe)\n", p);
        else {
            for (j = 0; j < m; j++) { avail[j] += r[j]; alloc[p][j] -= r[j]; need[p][j]
+= r[j]; }
            printf("P%d: request DENIED (would lead to unsafe state)\n", p);
        }
    }
    return 0;
}
```

INPUT:

```
5 3
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
3 3 2
3
1 1 0 2
4 3 3 0
0 0 2 0
```

OUTPUT:

```
Enter number of processes and resources: 5 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum matrix:
7 5 3
3 2 2
9 0 2
2 2 2
```

```
4 3 3
Enter Available resources: 3 3 2
Initial state is SAFE
Enter number of requests: 3

Enter process number and request vector: 1 1 0 2
P1: request GRANTED (system remains safe)

Enter process number and request vector: 4 3 3 0
P4: resources not available - process must WAIT

Enter process number and request vector: 0 0 2 0
P0: request DENIED (would lead to unsafe state)
```

RESULT:

Thus resource allocation with safe-state detection was simulated successfully.

Exp. No: 47

IMPLEMENTATION OF THREAD POOLS FOR CONCURRENT TASK EXECUTION

AIM:

To write a C program that implements a thread pool in which a fixed number of worker threads execute tasks taken from a shared task queue.

PROCEDURE:

1. Create a task queue (array) protected by a mutex and a condition variable.
2. Create a pool of 3 worker threads.
3. Each worker waits on the condition variable until a task is available, removes it from the queue and executes it.
4. The main thread adds 8 tasks (each task computes the square of a number) to the queue and signals the workers.
5. After all tasks are added, the main thread sets a shutdown flag and wakes all workers.
6. Workers exit when the queue is empty and shutdown is set; the main thread joins them.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define WORKERS 3
#define MAXQ 20
int queue[MAXQ], front = 0, rear = 0, shutdown_flag = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

void *worker(void *arg)
{
    int id = *(int *)arg;
    while (1) {
        pthread_mutex_lock(&m);
        while (front == rear && !shutdown_flag)
            pthread_cond_wait(&cv, &m);
        if (front == rear && shutdown_flag) {
            pthread_mutex_unlock(&m);
            break;
        }
        int task = queue[front++];
        pthread_mutex_unlock(&m);
        printf("Worker %d executing task %d: %d x %d = %d\n", id, task, task, task, task
* task);
        usleep(100000);
    }
    printf("Worker %d exiting\n", id);
    return NULL;
}

int main()
{
    pthread_t t[WORKERS];
    int id[WORKERS], i;
    for (i = 0; i < WORKERS; i++) {
        id[i] = i + 1;
```

```
        pthread_create(&t[i], NULL, worker, &id[i]);
    }
    for (i = 1; i <= 8; i++) {
        pthread_mutex_lock(&m);
        queue[rear++] = i;
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    pthread_mutex_lock(&m);
    shutdown_flag = 1;
    pthread_cond_broadcast(&cv);
    pthread_mutex_unlock(&m);
    for (i = 0; i < WORKERS; i++)
        pthread_join(t[i], NULL);
    printf("All tasks completed by the thread pool\n");
    return 0;
}
```

OUTPUT:

```
Worker 2 executing task 1: 1 x 1 = 1
Worker 3 executing task 3: 3 x 3 = 9
Worker 1 executing task 2: 2 x 2 = 4
Worker 2 executing task 4: 4 x 4 = 16
Worker 3 executing task 5: 5 x 5 = 25
Worker 1 executing task 6: 6 x 6 = 36
Worker 2 executing task 7: 7 x 7 = 49
Worker 1 executing task 8: 8 x 8 = 64
Worker 3 exiting
Worker 2 exiting
Worker 1 exiting
All tasks completed by the thread pool
```

RESULT:

Thus a thread pool for concurrent task execution was implemented successfully.

Exp. No: 48

IMPLEMENTATION OF PRODUCER–CONSUMER SYNCHRONIZATION USING BOUNDED BUFFERS

AIM:

To write a C program that synchronises producers and consumers on a bounded buffer using a mutex and condition variables.

PROCEDURE:

1. Declare a circular buffer of size 3 with count, in and out.
2. Declare a mutex and two condition variables not_full and not_empty.
3. Producer: lock; while count == size wait on not_full; insert item; signal not_empty; unlock.
4. Consumer: lock; while count == 0 wait on not_empty; remove item; signal not_full; unlock.
5. Create 2 producers (4 items each) and 2 consumers (4 items each) and join them.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define SIZE 3
int buf[SIZE], count = 0, in = 0, out = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg)
{
    int id = *(int *)arg;
    for (int i = 1; i <= 4; i++) {
        int item = id * 100 + i;
        pthread_mutex_lock(&m);
        while (count == SIZE) {
            printf("Producer %d: buffer FULL, waiting\n", id);
            pthread_cond_wait(&not_full, &m);
        }
        buf[in] = item; in = (in + 1) % SIZE; count++;
        printf("Producer %d put %d   (count = %d)\n", id, item, count);
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg)
{
    int id = *(int *)arg;
    for (int i = 1; i <= 4; i++) {
        pthread_mutex_lock(&m);
        while (count == 0)
            pthread_cond_wait(&not_empty, &m);
        int item = buf[out]; out = (out + 1) % SIZE; count--;
        printf("Consumer %d took %d  (count = %d)\n", id, item, count);
        pthread_cond_signal(&not_full);
```

```
        pthread_mutex_unlock(&m);
        usleep(100000);
    }
    return NULL;
}

int main()
{
    pthread_t p[2], c[2];
    int id[2] = {1, 2}, i;
    for (i = 0; i < 2; i++) pthread_create(&p[i], NULL, producer, &id[i]);
    for (i = 0; i < 2; i++) pthread_create(&c[i], NULL, consumer, &id[i]);
    for (i = 0; i < 2; i++) { pthread_join(p[i], NULL); pthread_join(c[i], NULL); }
    printf("Bounded buffer: all items produced and consumed\n");
    return 0;
}
```

OUTPUT:

```
Producer 2 put 201   (count = 1)
Producer 2 put 202   (count = 2)
Producer 2 put 203   (count = 3)
Producer 2: buffer FULL, waiting
Consumer 2 took 201  (count = 2)
Producer 2 put 204   (count = 3)
Consumer 1 took 202  (count = 2)
Producer 1 put 101   (count = 3)
Producer 1: buffer FULL, waiting
Consumer 2 took 203  (count = 2)
Producer 1 put 102   (count = 3)
Producer 1: buffer FULL, waiting
Consumer 1 took 204  (count = 2)
Producer 1 put 103   (count = 3)
Producer 1: buffer FULL, waiting
Consumer 2 took 101  (count = 2)
Producer 1 put 104   (count = 3)
Consumer 1 took 102  (count = 2)
Consumer 2 took 103  (count = 1)
Consumer 1 took 104  (count = 0)
Bounded buffer: all items produced and consumed
```

RESULT:

Thus producer-consumer synchronisation using a bounded buffer was implemented successfully.

Exp. No: 49

IMPLEMENTATION OF CONCURRENT FILE ACCESS USING LOCKS

AIM:

To write a C program in which several processes update the same file safely, using file locks (fcntl) for concurrent file access.

PROCEDURE:

1. Create a file counter.txt containing the value 0.
2. Create 3 child processes using fork().
3. Each child repeats 5 times: open the file, lock it with fcntl() F_SETLKW (write lock), read the value, increment it, write it back and unlock it.
4. The lock ensures that only one process updates the file at a time.
5. The parent waits for all children and displays the final value (expected 15).

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

void update(int id)
{
    struct flock lk = {0};
    char buf[20];
    for (int i = 0; i < 5; i++) {
        int fd = open("counter.txt", O_RDWR);
        lk.l_type = F_WRLCK;  lk.l_whence = SEEK_SET;
        fcntl(fd, F_SETLKW, &lk);                  /* lock */
        int n = read(fd, buf, sizeof(buf) - 1);
        buf[n] = '\0';
        int value = atoi(buf) + 1;
        lseek(fd, 0, SEEK_SET);
        n = sprintf(buf, "%d\n", value);
        write(fd, buf, n);
        printf("Process %d updated counter to %d\n", id, value);
        fflush(stdout);
        lk.l_type = F_UNLCK;
        fcntl(fd, F_SETLK, &lk);                   /* unlock */
        close(fd);
        usleep(20000);
    }
}

int main()
{
    FILE *fp = fopen("counter.txt", "w");
    fprintf(fp, "0\n");
    fclose(fp);
    for (int i = 1; i <= 3; i++)
        if (fork() == 0) { update(i); exit(0); }
    for (int i = 0; i < 3; i++) wait(NULL);

    char buf[20];
    fp = fopen("counter.txt", "r");
```

```
    fgets(buf, sizeof(buf), fp);
    fclose(fp);
    printf("Final counter value in file = %s", buf);
    return 0;
}
```

OUTPUT:

```
Process 1 updated counter to 1
Process 3 updated counter to 2
Process 2 updated counter to 3
Process 1 updated counter to 4
Process 2 updated counter to 5
Process 3 updated counter to 6
Process 1 updated counter to 7
Process 3 updated counter to 8
Process 2 updated counter to 9
Process 1 updated counter to 10
Process 3 updated counter to 11
Process 2 updated counter to 12
Process 1 updated counter to 13
Process 2 updated counter to 14
Process 3 updated counter to 15
Final counter value in file = 15
```

RESULT:

Thus concurrent file access using file locks was implemented successfully and the final value was correct.

Exp. No: 50

PERFORMANCE EVALUATION OF MULTITHREADED AND MULTIPROCESS APPLICATIONS

AIM:

To write a C program that compares the performance of a multithreaded and a multiprocess solution of the same task (summing a large array).

PROCEDURE:

1. Fill an array of 40,000,000 integers.
2. Multithreaded version: create 4 threads; each sums one quarter of the array; join them and add the results.
3. Multiprocess version: create 4 child processes with fork(); each sums one quarter and returns the partial sum through a pipe.
4. Measure the time taken by each version using clock_gettime().
5. Display both sums and times and compare them.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define N 40000000
#define W 4
int *a;
long part[W];

double now()
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

void *tsum(void *arg)
{
    int id = *(int *)arg;
    long s = 0;
    for (int i = id * (N / W); i < (id + 1) * (N / W); i++) s += a[i];
    part[id] = s;
    return NULL;
}

int main()
{
    int i, id[W], fd[2];
    long total;
    double t1, t2;
    pthread_t th[W];

    a = malloc(sizeof(int) * N);
    for (i = 0; i < N; i++) a[i] = i % 10;
```

```
    t1 = now();
    for (i = 0; i < W; i++) { id[i] = i; pthread_create(&th[i], NULL, tsum, &id[i]); }
    total = 0;
    for (i = 0; i < W; i++) { pthread_join(th[i], NULL); total += part[i]; }
    t2 = now();
    printf("Multithreaded  (%d threads)  : sum = %ld, time = %.4f s\n", W, total, t2 -
t1);

    pipe(fd);
    t1 = now();
    for (i = 0; i < W; i++)
        if (fork() == 0) {
            long s = 0;
            for (int k = i * (N / W); k < (i + 1) * (N / W); k++) s += a[k];
            write(fd[1], &s, sizeof(s));
            exit(0);
        }
    total = 0;
    for (i = 0; i < W; i++) {
        long s;
        read(fd[0], &s, sizeof(s));
        total += s;
        wait(NULL);
    }
    t2 = now();
    printf("Multiprocess   (%d processes): sum = %ld, time = %.4f s\n", W, total, t2 -
t1);
    printf("Threads share memory and are cheaper to create than processes.\n");
    free(a);
    return 0;
}
```

OUTPUT:

```
Multithreaded  (4 threads)  : sum = 180000000, time = 0.0565 s
Multiprocess   (4 processes): sum = 180000000, time = 0.0600 s
Threads share memory and are cheaper to create than processes.
```

RESULT:

Thus the performance of multithreaded and multiprocess applications was evaluated and compared successfully.

Exp. No: 51

IMPLEMENTATION OF FIRST FIT MEMORY ALLOCATION ALGORITHM

AIM:

To write a C program to implement the First Fit memory allocation algorithm.

PROCEDURE:

1. Read the number of memory blocks and the size of each block.
2. Read the number of processes and the size of each process.
3. For each process, search the blocks from the beginning and select the first block whose free size is greater than or equal to the process size.
4. If a block is found, allocate it and reduce the free size of that block by the process size.
5. If no block is large enough, mark the process as not allocated.
6. Display the allocation table and the remaining free space in each block.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nb, np, i, j, k, blk[10], prc[10], alloc[10], orig[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (j = 0; j < nb; j++) { scanf("%d", &blk[j]); orig[j] = blk[j]; }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &prc[i]);

    for (i = 0; i < np; i++) {
        k = -1;
        for (j = 0; j < nb; j++)
            if (blk[j] >= prc[i]) { k = j; break; }
        alloc[i] = k;
        if (k != -1) blk[k] -= prc[i];
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i] != -1)
            printf("P%d\t%d\t%d (%d KB)\n", i + 1, prc[i], alloc[i] + 1,
orig[alloc[i]]);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, prc[i]);
    }
    printf("\nRemaining free space in blocks: ");
    for (j = 0; j < nb; j++) printf("%d ", blk[j]);
    printf("\n");
    return 0;
}
```

INPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 4
Enter size of each process: 212 417 112 426

Process Size    Block
P1      212     2 (500 KB)
P2      417     5 (600 KB)
P3      112     2 (500 KB)
P4      426     Not Allocated

Remaining free space in blocks: 100 176 200 300 183
```

RESULT:

Thus the First Fit memory allocation algorithm was implemented successfully.

Exp. No: 52

IMPLEMENTATION OF BEST FIT MEMORY ALLOCATION ALGORITHM

AIM:

To write a C program to implement the Best Fit memory allocation algorithm.

PROCEDURE:

1. Read the number of memory blocks and the size of each block.
2. Read the number of processes and the size of each process.
3. For each process, search all the blocks and select the smallest block whose free size is greater than or equal to the process size.
4. If a block is found, allocate it and reduce the free size of that block by the process size.
5. If no block is large enough, mark the process as not allocated.
6. Display the allocation table and the remaining free space in each block.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nb, np, i, j, k, blk[10], prc[10], alloc[10], orig[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (j = 0; j < nb; j++) { scanf("%d", &blk[j]); orig[j] = blk[j]; }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &prc[i]);

    for (i = 0; i < np; i++) {
        k = -1;
        for (j = 0; j < nb; j++)
            if (blk[j] >= prc[i] && (k == -1 || blk[j] < blk[k])) k = j;
        alloc[i] = k;
        if (k != -1) blk[k] -= prc[i];
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i] != -1)
            printf("P%d\t%d\t%d (%d KB)\n", i + 1, prc[i], alloc[i] + 1,
orig[alloc[i]]);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, prc[i]);
    }
    printf("\nRemaining free space in blocks: ");
    for (j = 0; j < nb; j++) printf("%d ", blk[j]);
    printf("\n");
    return 0;
}
```

INPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 4
Enter size of each process: 212 417 112 426

Process Size    Block
P1      212     4 (300 KB)
P2      417     2 (500 KB)
P3      112     3 (200 KB)
P4      426     5 (600 KB)

Remaining free space in blocks: 100 83 88 88 174
```

RESULT:

Thus the Best Fit memory allocation algorithm was implemented successfully.

Exp. No: 53

IMPLEMENTATION OF WORST FIT MEMORY ALLOCATION ALGORITHM

AIM:

To write a C program to implement the Worst Fit memory allocation algorithm.

PROCEDURE:

1. Read the number of memory blocks and the size of each block.
2. Read the number of processes and the size of each process.
3. For each process, search all the blocks and select the largest block whose free size is greater than or equal to the process size.
4. If a block is found, allocate it and reduce the free size of that block by the process size.
5. If no block is large enough, mark the process as not allocated.
6. Display the allocation table and the remaining free space in each block.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nb, np, i, j, k, blk[10], prc[10], alloc[10], orig[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (j = 0; j < nb; j++) { scanf("%d", &blk[j]); orig[j] = blk[j]; }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &prc[i]);

    for (i = 0; i < np; i++) {
        k = -1;
        for (j = 0; j < nb; j++)
            if (blk[j] >= prc[i] && (k == -1 || blk[j] > blk[k])) k = j;
        alloc[i] = k;
        if (k != -1) blk[k] -= prc[i];
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i] != -1)
            printf("P%d\t%d\t%d (%d KB)\n", i + 1, prc[i], alloc[i] + 1,
orig[alloc[i]]);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, prc[i]);
    }
    printf("\nRemaining free space in blocks: ");
    for (j = 0; j < nb; j++) printf("%d ", blk[j]);
    printf("\n");
    return 0;
}
```

INPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 4
Enter size of each process: 212 417 112 426

Process Size    Block
P1      212     5 (600 KB)
P2      417     2 (500 KB)
P3      112     5 (600 KB)
P4      426     Not Allocated

Remaining free space in blocks: 100 83 200 300 276
```

RESULT:

Thus the Worst Fit memory allocation algorithm was implemented successfully.

Exp. No: 54

IMPLEMENTATION OF NEXT FIT MEMORY ALLOCATION ALGORITHM

AIM:

To write a C program to implement the Next Fit memory allocation algorithm.

PROCEDURE:

1. Read the number of memory blocks and the size of each block.
2. Read the number of processes and the size of each process.
3. For each process, start searching from the block where the previous allocation ended (not from the beginning), wrapping around to the first block, and select the first block that is large enough.
4. If a block is found, allocate it and reduce the free size of that block by the process size.
5. If no block is large enough, mark the process as not allocated.
6. Display the allocation table and the remaining free space in each block.
7. Remember the position of the last allocation for the next process.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nb, np, i, j, c, last = 0, blk[10], prc[10], alloc[10], orig[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (j = 0; j < nb; j++) { scanf("%d", &blk[j]); orig[j] = blk[j]; }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &prc[i]);

    for (i = 0; i < np; i++) {
        alloc[i] = -1;
        for (c = 0; c < nb; c++) {
            j = (last + c) % nb;
            if (blk[j] >= prc[i]) {
                alloc[i] = j;
                blk[j] -= prc[i];
                last = j;
                break;
            }
        }
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        if (alloc[i] != -1)
            printf("P%d\t%d\t%d (%d KB)\n", i + 1, prc[i], alloc[i] + 1,
orig[alloc[i]]);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, prc[i]);
    }
    printf("\nRemaining free space in blocks: ");
```

```
    for (j = 0; j < nb; j++) printf("%d ", blk[j]);
    printf("\n");
    return 0;
}
```

INPUT:

```
5
100 500 200 300 600
5
212 417 112 426 90
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 5
Enter size of each process: 212 417 112 426 90

Process Size    Block
P1      212     2 (500 KB)
P2      417     5 (600 KB)
P3      112     5 (600 KB)
P4      426     Not Allocated
P5      90      1 (100 KB)

Remaining free space in blocks: 10 288 200 300 71
```

RESULT:

Thus the Next Fit memory allocation algorithm was implemented successfully.

Exp. No: 55

PERFORMANCE COMPARISON OF CONTIGUOUS MEMORY ALLOCATION STRATEGIES

AIM:

To write a C program that compares the First Fit, Best Fit, Worst Fit and Next Fit contiguous memory allocation strategies.

PROCEDURE:

1. Read the block sizes and process sizes.
2. Write one function that allocates the processes using a given strategy (1 = first, 2 = best, 3 = worst, 4 = next) on a fresh copy of the blocks.
3. For each strategy count the number of processes allocated and the total free memory left.
4. Also find the largest free hole left, which shows how well large future requests can be served.
5. Display a comparison table and identify the strategy that allocated the most processes.

PROGRAM:

```
#include <stdio.h>

int nb, np, blk[10], prc[10];

void simulate(int s, int *done, int *freemem, int *largest)
{
    int b[10], i, j, c, k, last = 0;
    for (j = 0; j < nb; j++) b[j] = blk[j];
    *done = 0;
    for (i = 0; i < np; i++) {
        k = -1;
        for (c = 0; c < nb; c++) {
            j = (s == 4) ? (last + c) % nb : c;
            if (b[j] < prc[i]) continue;
            if (s == 1 || s == 4) { k = j; break; }
            if (k == -1 || (s == 2 && b[j] < b[k]) || (s == 3 && b[j] > b[k])) k = j;
        }
        if (k != -1) { b[k] -= prc[i]; (*done)++; last = k; }
    }
    *freemem = *largest = 0;
    for (j = 0; j < nb; j++) {
        *freemem += b[j];
        if (b[j] > *largest) *largest = b[j];
    }
}

int main()
{
    int i, s, d, f, l, best = 1, bestd = -1;
    char *name[5] = {"", "First Fit", "Best Fit", "Worst Fit", "Next Fit"};

    printf("Enter number of blocks: ");
    scanf("%d", &nb);
    printf("Enter block sizes: ");
    for (i = 0; i < nb; i++) scanf("%d", &blk[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter process sizes: ");
```

```
    for (i = 0; i < np; i++) scanf("%d", &prc[i]);

    printf("\nStrategy\tAllocated\tFree memory\tLargest hole\n");
    for (s = 1; s <= 4; s++) {
        simulate(s, &d, &f, &l);
        printf("%-10s\t%d of %d\t\t%d\t\t%d\n", name[s], d, np, f, l);
        if (d > bestd) { bestd = d; best = s; }
    }
    printf("\nBest strategy for this input: %s\n", name[best]);
    return 0;
}
```

INPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

OUTPUT:

```
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 417 112 426

Strategy        Allocated       Free memory     Largest hole
First Fit       3 of 4          959             300
Best Fit        4 of 4          533             174
Worst Fit       3 of 4          959             300
Next Fit        3 of 4          959             300

Best strategy for this input: Best Fit
```

RESULT:

Thus the performance of the contiguous memory allocation strategies was compared successfully.

Exp. No: 56

SIMULATION OF INTERNAL AND EXTERNAL FRAGMENTATION

AIM:

To write a C program that simulates internal fragmentation in fixed partitioning and external fragmentation in variable partitioning.

PROCEDURE:

1. Internal fragmentation: read the fixed partition size and the process sizes.
2. Allocate one process per partition; wasted space inside a partition = partition size - process size.
3. Display the internal fragmentation of each partition and the total.
4. External fragmentation: read the sizes of the free holes in memory and the size of a new process.
5. If no single hole is large enough but the total free space is enough, the request fails because of external fragmentation.
6. Display the result.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int part, n, i, p, total = 0, holes, h[10], need, sum = 0, maxh = 0;

    printf("--- Internal Fragmentation (fixed partitions) ---\n");
    printf("Enter partition size: ");
    scanf("%d", &part);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter size of P%d: ", i + 1);
        scanf("%d", &p);
        if (p > part)
            printf("  P%d too large for a partition\n", i + 1);
        else {
            printf("  P%d in partition %d, internal fragmentation = %d KB\n", i + 1, i +
1, part - p);
            total += part - p;
        }
    }
    printf("Total internal fragmentation = %d KB\n", total);

    printf("\n--- External Fragmentation (variable partitions) ---\n");
    printf("Enter number of free holes: ");
    scanf("%d", &holes);
    printf("Enter hole sizes: ");
    for (i = 0; i < holes; i++) {
        scanf("%d", &h[i]);
        sum += h[i];
        if (h[i] > maxh) maxh = h[i];
    }
    printf("Enter size of new process: ");
    scanf("%d", &need);
    printf("Total free memory = %d KB, largest hole = %d KB\n", sum, maxh);
    if (maxh >= need)
```

```
        printf("Process can be allocated. No external fragmentation problem.\n");
    else if (sum >= need)
        printf("Process CANNOT be allocated although %d KB is free: EXTERNAL
FRAGMENTATION\n", sum);
    else
        printf("Not enough total memory.\n");
    return 0;
}
```

INPUT:

```
100
4
75
40
100
18
4
50 30 60 40
120
```

OUTPUT:

```
--- Internal Fragmentation (fixed partitions) ---
Enter partition size: 100
Enter number of processes: 4
Enter size of P1: 75
  P1 in partition 1, internal fragmentation = 25 KB
Enter size of P2: 40
  P2 in partition 2, internal fragmentation = 60 KB
Enter size of P3: 100
  P3 in partition 3, internal fragmentation = 0 KB
Enter size of P4: 18
  P4 in partition 4, internal fragmentation = 82 KB
Total internal fragmentation = 167 KB

--- External Fragmentation (variable partitions) ---
Enter number of free holes: 4
Enter hole sizes: 50 30 60 40
Enter size of new process: 120
Total free memory = 180 KB, largest hole = 60 KB
Process CANNOT be allocated although 180 KB is free: EXTERNAL FRAGMENTATION
```

RESULT:

Thus internal and external fragmentation were simulated successfully.

Exp. No: 57

IMPLEMENTATION OF MEMORY COMPACTION FOR FRAGMENTATION REDUCTION

AIM:

To write a C program that reduces external fragmentation using memory compaction.

PROCEDURE:

1. Represent memory as a list of blocks; each block has a start address, a size and an owner (process name or FREE).
2. Read the memory map and display it with the free holes scattered.
3. Read the size of a new process and show that it cannot be allocated in any single hole.
4. Compaction: move all allocated blocks towards address 0 one after another and update their start addresses.
5. Combine all free space into one large hole at the end of memory.
6. Display the memory map after compaction and allocate the new process.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct block { char name[10]; int start, size; } mem[20], newmem[20];

void show(struct block m[], int n)
{
    printf("Start\tSize\tOwner\n");
    for (int i = 0; i < n; i++)
        printf("%d\t%d\t%s\n", m[i].start, m[i].size, m[i].name);
}

int main()
{
    int n, i, k = 0, addr = 0, freesz = 0, req, maxh = 0;
    printf("Enter number of memory blocks: ");
    scanf("%d", &n);
    printf("Enter owner (or FREE) and size of each block:\n");
    for (i = 0; i < n; i++) {
        scanf("%s %d", mem[i].name, &mem[i].size);
        mem[i].start = addr;
        addr += mem[i].size;
        if (strcmp(mem[i].name, "FREE") == 0 && mem[i].size > maxh) maxh = mem[i].size;
    }
    printf("\nMemory before compaction:\n");
    show(mem, n);
    printf("Enter size of new process P5: ");
    scanf("%d", &req);
    printf("Largest hole = %d KB, so P5 (%d KB) cannot be allocated.\n", maxh, req);

    addr = 0;
    for (i = 0; i < n; i++) {
        if (strcmp(mem[i].name, "FREE") == 0) { freesz += mem[i].size; continue; }
        newmem[k] = mem[i];
        newmem[k].start = addr;
        addr += mem[i].size;
        k++;
    }
```

```
    strcpy(newmem[k].name, "FREE");
    newmem[k].start = addr;
    newmem[k].size = freesz;
    k++;
    printf("\nMemory after compaction:\n");
    show(newmem, k);
    if (freesz >= req)
        printf("P5 allocated at address %d after compaction.\n", addr);
    return 0;
}
```

INPUT:

```
6
P1 100
FREE 50
P2 200
FREE 80
P3 150
FREE 70
180
```

OUTPUT:

```
Enter number of memory blocks: 6
Enter owner (or FREE) and size of each block:
P1 100
FREE 50
P2 200
FREE 80
P3 150
FREE 70

Memory before compaction:
Start   Size    Owner
0       100     P1
100     50      FREE
150     200     P2
350     80      FREE
430     150     P3
580     70      FREE
Enter size of new process P5: 180
Largest hole = 80 KB, so P5 (180 KB) cannot be allocated.

Memory after compaction:
Start   Size    Owner
0       100     P1
100     200     P2
300     150     P3
450     200     FREE
P5 allocated at address 450 after compaction.
```

RESULT:

Thus memory compaction was implemented successfully and the scattered free holes were combined into one large hole.

Exp. No: 58

IMPLEMENTATION OF SWAPPING AND PROCESS MEMORY TRANSFER SIMULATION

AIM:

To write a C program that simulates swapping, in which processes are moved between main memory and the backing store when memory is insufficient.

PROCEDURE:

1. Read the size of main memory and the sizes of the incoming processes.
2. Load each process into main memory if enough free memory is available.
3. If memory is not sufficient, swap out the oldest process in memory (FIFO) to the backing store until there is enough space.
4. Load the new process (swap in) and display the contents of memory and the backing store after each step.
5. Display the total number of swap-outs.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int cap, n, i, j, size[10], inmem[10], cnt = 0, used = 0, disk[10], dcnt = 0, swaps
= 0;

    printf("Enter main memory size (KB): ");
    scanf("%d", &cap);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter size of each process: ");
    for (i = 0; i < n; i++) scanf("%d", &size[i]);

    for (i = 0; i < n; i++) {
        printf("\nLoading P%d (%d KB)\n", i + 1, size[i]);
        while (used + size[i] > cap && cnt > 0) {
            int out = inmem[0];
            printf("  Not enough memory: swap OUT P%d (%d KB) to backing store\n", out +
1, size[out]);
            used -= size[out];
            disk[dcnt++] = out;
            for (j = 0; j < cnt - 1; j++) inmem[j] = inmem[j + 1];
            cnt--;
            swaps++;
        }
        inmem[cnt++] = i;
        used += size[i];
        printf("  Swap IN P%d. Memory: ", i + 1);
        for (j = 0; j < cnt; j++) printf("P%d ", inmem[j] + 1);
        printf("(used %d/%d KB)  Backing store: ", used, cap);
        for (j = 0; j < dcnt; j++) printf("P%d ", disk[j] + 1);
        printf("\n");
    }
    printf("\nTotal swap-outs = %d\n", swaps);
    return 0;
}
```

INPUT:

```
1000
5
300 400 200 500 250
```

OUTPUT:

```
Enter main memory size (KB): 1000
Enter number of processes: 5
Enter size of each process: 300 400 200 500 250

Loading P1 (300 KB)
  Swap IN P1. Memory: P1 (used 300/1000 KB)  Backing store:

Loading P2 (400 KB)
  Swap IN P2. Memory: P1 P2 (used 700/1000 KB)  Backing store:

Loading P3 (200 KB)
  Swap IN P3. Memory: P1 P2 P3 (used 900/1000 KB)  Backing store:

Loading P4 (500 KB)
  Not enough memory: swap OUT P1 (300 KB) to backing store
  Not enough memory: swap OUT P2 (400 KB) to backing store
  Swap IN P4. Memory: P3 P4 (used 700/1000 KB)  Backing store: P1 P2

Loading P5 (250 KB)
  Swap IN P5. Memory: P3 P4 P5 (used 950/1000 KB)  Backing store: P1 P2

Total swap-outs = 2
```

RESULT:

Thus swapping of processes between main memory and the backing store was simulated successfully.

Exp. No: 59

IMPLEMENTATION OF STATIC AND DYNAMIC LINKING USING C PROGRAMS

AIM:

To create a static library and a shared (dynamic) library from a C program, link a program with each, and compare static and dynamic linking.

PROCEDURE:

1. Write the library functions add() and square() in mathlib.c and the main program in main.c.
2. Static linking: compile mathlib.c to an object file and create the archive libmathlib.a using ar rcs.
3. Link main.c with the static library using gcc -static and run the program.
4. Dynamic linking: compile mathlib.c with -fPIC and create the shared library libmathlib.so using gcc -shared.
5. Link main.c with the shared library and run it after setting LD_LIBRARY_PATH.
6. Compare the sizes of the two executables and check their library dependencies using ldd.

PROGRAM:

mathlib.c

```
/* mathlib.c - library functions */
int add(int a, int b)
{
    return a + b;
}

int square(int x)
{
    return x * x;
}
```

main.c

```
/* main.c - uses the library */
#include <stdio.h>

int add(int a, int b);
int square(int x);

int main()
{
    printf("add(4, 5)  = %d\n", add(4, 5));
    printf("square(6)  = %d\n", square(6));
    return 0;
}
```

Commands:

```
gcc -c mathlib.c -o mathlib.o
ar rcs libmathlib.a mathlib.o
gcc -static main.c -L. -lmathlib -o static_prog
```

```
./static_prog
gcc -fPIC -c mathlib.c -o mathlib_pic.o
gcc -shared -o libmathlib.so mathlib_pic.o
gcc main.c -L. -lmathlib -o dynamic_prog
export LD_LIBRARY_PATH=.
./dynamic_prog
ls -l static_prog dynamic_prog
ldd static_prog
ldd dynamic_prog | grep -E "mathlib|libc\."
```

OUTPUT:

```
$ gcc -c mathlib.c -o mathlib.o
$ ar rcs libmathlib.a mathlib.o
$ gcc -static main.c -L. -lmathlib -o static_prog
$ ./static_prog
add(4, 5)  = 9
square(6)  = 36
$ gcc -fPIC -c mathlib.c -o mathlib_pic.o
$ gcc -shared -o libmathlib.so mathlib_pic.o
$ gcc main.c -L. -lmathlib -o dynamic_prog
$ export LD_LIBRARY_PATH=.
$ ./dynamic_prog
add(4, 5)  = 9
square(6)  = 36
$ ls -l static_prog dynamic_prog
-rwxr-xr-x 1 root root  16024 Oct 21 09:22 dynamic_prog
-rwxr-xr-x 1 root root 785416 Oct 21 09:22 static_prog
$ ldd static_prog
        not a dynamic executable
$ ldd dynamic_prog | grep -E "mathlib|libc\."
        libmathlib.so => ./libmathlib.so (0x00007f3d503fe000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f3d50000000)
```

RESULT:

Thus static and dynamic linking were implemented. The statically linked program is much larger because the library code is copied into it, while the dynamically linked program loads libmathlib.so at run time.

Exp. No: 60

IMPLEMENTATION OF DYNAMIC LIBRARY CREATION AND RUNTIME LINKING IN LINUX

AIM:

To create a dynamic (shared) library in Linux and load it at run time from a C program using dlopen(), dlsym() and dlclose().

PROCEDURE:

1. Write the functions area_circle() and area_square() in shapes.c.
2. Compile with -fPIC and create the shared library libshapes.so using gcc -shared.
3. Write loader.c, which opens the library at run time using dlopen().
4. Get the address of each function using dlsym() and call it through a function pointer.
5. Close the library using dlclose().
6. Compile loader.c with -ldl and run it; use nm -D to list the symbols exported by the library.

PROGRAM:

shapes.c

```
/* shapes.c - shared library */
double area_circle(double r)
{
    return 3.14159 * r * r;
}

double area_square(double s)
{
    return s * s;
}
```

loader.c

```
/* loader.c - loads the library at run time */
#include <stdio.h>
#include <dlfcn.h>

int main()
{
    void *handle;
    double (*fn)(double);
    char *name[2] = {"area_circle", "area_square"};

    handle = dlopen("./libshapes.so", RTLD_LAZY);
    if (!handle) {
        printf("dlopen error: %s\n", dlerror());
        return 1;
    }
    printf("Library libshapes.so loaded at run time\n");
    for (int i = 0; i < 2; i++) {
        fn = (double (*)(double))dlsym(handle, name[i]);
        if (fn)
            printf("%s(5.0) = %.2f\n", name[i], fn(5.0));
    }
    dlclose(handle);
```

```
    printf("Library unloaded\n");
    return 0;
}
```

Commands:

```
gcc -fPIC -c shapes.c -o shapes.o
gcc -shared -o libshapes.so shapes.o
nm -D libshapes.so | grep area
gcc loader.c -o loader -ldl
./loader
```

OUTPUT:

```
$ gcc -fPIC -c shapes.c -o shapes.o
$ gcc -shared -o libshapes.so shapes.o
$ nm -D libshapes.so | grep area
00000000000010f9 T area_circle
000000000000111e T area_square
$ gcc loader.c -o loader -ldl
$ ./loader
Library libshapes.so loaded at run time
area_circle(5.0) = 78.54
area_square(5.0) = 25.00
Library unloaded
```

RESULT:

Thus a dynamic library was created and linked at run time using dlopen() and dlsym() successfully.

Exp. No: 61

IMPLEMENTATION OF PAGING-BASED ADDRESS TRANSLATION

AIM:

To write a C program that translates logical addresses into physical addresses using paging.

PROCEDURE:

1. Read the page size and the number of pages of the process.
2. Read the page table, which gives the frame number for each page.
3. Read a logical address.
4. Page number = logical address / page size; offset = logical address % page size.
5. If the page number is invalid, report an error.
6. Physical address = frame number x page size + offset; display it.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int ps, np, pt[20], i, n, la, p, d;

    printf("Enter page size (bytes): ");
    scanf("%d", &ps);
    printf("Enter number of pages: ");
    scanf("%d", &np);
    printf("Enter frame number for each page: ");
    for (i = 0; i < np; i++) scanf("%d", &pt[i]);

    printf("Enter number of logical addresses: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nEnter logical address: ");
        scanf("%d", &la);
        p = la / ps;
        d = la % ps;
        if (p >= np) {
            printf("Page %d is invalid - address out of range\n", p);
            continue;
        }
        printf("Page = %d, Offset = %d, Frame = %d\n", p, d, pt[p]);
        printf("Physical address = %d x %d + %d = %d\n", pt[p], ps, d, pt[p] * ps + d);
    }
    return 0;
}
```

INPUT:

```
1024
4
5 2 7 1
3
2100
3000
5000
```

OUTPUT:

```
Enter page size (bytes): 1024
Enter number of pages: 4
Enter frame number for each page: 5 2 7 1
Enter number of logical addresses: 3

Enter logical address: 2100
Page = 2, Offset = 52, Frame = 7
Physical address = 7 x 1024 + 52 = 7220

Enter logical address: 3000
Page = 2, Offset = 952, Frame = 7
Physical address = 7 x 1024 + 952 = 8120

Enter logical address: 5000
Page 4 is invalid - address out of range
```

RESULT:

Thus logical addresses were translated into physical addresses using paging successfully.

Exp. No: 62

IMPLEMENTATION OF PAGE TABLE MANAGEMENT AND LOGICAL-TO-PHYSICAL ADDRESS CONVERSION

AIM:

To write a C program that manages a page table: allocate free frames to the pages of a process, maintain valid bits and convert logical addresses to physical addresses.

PROCEDURE:

1. Read the page size, the number of frames in memory and the list of free frames.
2. Read the number of pages of the process and the pages to be loaded into memory.
3. For each page to be loaded, take the next free frame, store it in the page table and set the valid bit to 1. Other pages remain invalid (0).
4. Display the page table with frame numbers and valid bits.
5. For each logical address, find the page and offset. If the page is valid, compute the physical address; otherwise report a page fault.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int ps, nf, free_fr[20], np, nl, i, p, k = 0, la, frame[20], valid[20] = {0};

    printf("Enter page size: ");
    scanf("%d", &ps);
    printf("Enter number of free frames: ");
    scanf("%d", &nf);
    printf("Enter free frame numbers: ");
    for (i = 0; i < nf; i++) scanf("%d", &free_fr[i]);
    printf("Enter number of pages in the process: ");
    scanf("%d", &np);
    printf("Enter number of pages to load: ");
    scanf("%d", &nl);
    printf("Enter page numbers to load: ");
    for (i = 0; i < nl; i++) {
        scanf("%d", &p);
        if (k < nf) {
            frame[p] = free_fr[k++];
            valid[p] = 1;
        }
    }
    printf("\nPAGE TABLE\nPage\tFrame\tValid\n");
    for (i = 0; i < np; i++) {
        if (valid[i]) printf("%d\t%d\t1\n", i, frame[i]);
        else          printf("%d\t-\t0\n", i);
    }
    printf("\nEnter logical addresses (-1 to stop):\n");
    while (scanf("%d", &la) == 1 && la != -1) {
        p = la / ps;
        if (p >= np)
            printf("%d -> invalid page %d (trap)\n", la, p);
        else if (!valid[p])
            printf("%d -> page %d not in memory: PAGE FAULT\n", la, p);
        else
            printf("%d -> page %d, offset %d -> physical %d\n", la, p, la % ps, frame[p]
* ps + la % ps);
```

```
    }
    return 0;
}
```

INPUT:

```
100
4
9 3 6 12
5
3
0 2 4
45
130
250
470
600
-1
```

OUTPUT:

```
Enter page size: 100
Enter number of free frames: 4
Enter free frame numbers: 9 3 6 12
Enter number of pages in the process: 5
Enter number of pages to load: 3
Enter page numbers to load: 0 2 4

PAGE TABLE
Page    Frame   Valid
0       9       1
1       -       0
2       3       1
3       -       0
4       6       1

Enter logical addresses (-1 to stop):
45
45 -> page 0, offset 45 -> physical 945
130
130 -> page 1 not in memory: PAGE FAULT
250
250 -> page 2, offset 50 -> physical 350
470
470 -> page 4, offset 70 -> physical 670
600
600 -> invalid page 6 (trap)
-1
```

RESULT:

Thus page table management and logical-to-physical address conversion were implemented successfully.

Exp. No: 63

SIMULATION OF TRANSLATION LOOKASIDE BUFFER (TLB) ADDRESS TRANSLATION

AIM:

To write a C program that simulates address translation using a Translation Look-aside Buffer (TLB) and calculates the hit ratio and the effective access time.

PROCEDURE:

1. Read the page table (frame for each page) and the TLB size.
2. Read the sequence of page numbers referenced by the CPU.
3. For each reference, search the TLB. If found, it is a TLB hit.
4. If not found (TLB miss), get the frame from the page table and insert the entry into the TLB, replacing the oldest entry (FIFO) if the TLB is full.
5. Count hits and misses and compute the hit ratio.
6. Effective access time = h x (TLB time + memory time) + (1 - h) x (TLB time + 2 x memory time).

PROGRAM:

```
#include <stdio.h>

int main()
{
    int np, pt[20], tsize, n, ref, i, j, hits = 0, next = 0, count = 0;
    int tlb_page[10], tlb_frame[10];
    float tt, mt, h, eat;

    printf("Enter number of pages: ");
    scanf("%d", &np);
    printf("Enter frame for each page: ");
    for (i = 0; i < np; i++) scanf("%d", &pt[i]);
    printf("Enter TLB size: ");
    scanf("%d", &tsize);
    printf("Enter TLB access time and memory access time (ns): ");
    scanf("%f %f", &tt, &mt);
    printf("Enter number of page references: ");
    scanf("%d", &n);
    printf("Enter page references: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &ref);
        for (j = 0; j < count; j++)
            if (tlb_page[j] == ref) break;
        if (j < count) {
            hits++;
            printf("Page %d: TLB HIT  -> frame %d\n", ref, tlb_frame[j]);
        } else {
            printf("Page %d: TLB MISS -> frame %d from page table\n", ref, pt[ref]);
            if (count < tsize) j = count++;
            else { j = next; next = (next + 1) % tsize; }
            tlb_page[j] = ref;
            tlb_frame[j] = pt[ref];
        }
    }
    h = (float)hits / n;
    eat = h * (tt + mt) + (1 - h) * (tt + 2 * mt);
    printf("\nHits = %d, Misses = %d, Hit ratio = %.2f\n", hits, n - hits, h);
```

```
    printf("Effective access time = %.2f ns\n", eat);
    return 0;
}
```

INPUT:

```
6
3 8 1 6 2 9
3
20 100
10
0 1 2 0 1 3 0 4 1 0
```

OUTPUT:

```
Enter number of pages: 6
Enter frame for each page: 3 8 1 6 2 9
Enter TLB size: 3
Enter TLB access time and memory access time (ns): 20 100
Enter number of page references: 10
Enter page references: 0 1 2 0 1 3 0 4 1 0
Page 0: TLB MISS -> frame 3 from page table
Page 1: TLB MISS -> frame 8 from page table
Page 2: TLB MISS -> frame 1 from page table
Page 0: TLB HIT  -> frame 3
Page 1: TLB HIT  -> frame 8
Page 3: TLB MISS -> frame 6 from page table
Page 0: TLB MISS -> frame 3 from page table
Page 4: TLB MISS -> frame 2 from page table
Page 1: TLB MISS -> frame 8 from page table
Page 0: TLB HIT  -> frame 3

Hits = 3, Misses = 7, Hit ratio = 0.30
Effective access time = 190.00 ns
```

RESULT:

Thus address translation using a TLB was simulated and the hit ratio and effective access time were calculated successfully.

Exp. No: 64

IMPLEMENTATION OF SEGMENTATION-BASED ADDRESS TRANSLATION

AIM:

To write a C program that translates logical addresses into physical addresses using segmentation.

PROCEDURE:

1. Read the number of segments and the base and limit of each segment (segment table).
2. Read a logical address as a pair (segment number, offset).
3. If the segment number is invalid, report an error.
4. If offset >= limit, generate a trap (addressing error).
5. Otherwise physical address = base + offset; display it.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int ns, base[10], limit[10], n, s, d, i;

    printf("Enter number of segments: ");
    scanf("%d", &ns);
    for (i = 0; i < ns; i++) {
        printf("Enter base and limit of segment %d: ", i);
        scanf("%d %d", &base[i], &limit[i]);
    }
    printf("\nSEGMENT TABLE\nSeg\tBase\tLimit\n");
    for (i = 0; i < ns; i++) printf("%d\t%d\t%d\n", i, base[i], limit[i]);

    printf("\nEnter number of logical addresses: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter segment number and offset: ");
        scanf("%d %d", &s, &d);
        if (s >= ns)
            printf("  Invalid segment number\n");
        else if (d >= limit[s])
            printf("  TRAP: offset %d exceeds limit %d of segment %d\n", d, limit[s],
s);
        else
            printf("  Physical address = %d + %d = %d\n", base[s], d, base[s] + d);
    }
    return 0;
}
```

INPUT:

```
3
1400 1000
6300 400
4300 1100
3
0 53
```

```
1 450
2 1000
```

OUTPUT:

```
Enter number of segments: 3
Enter base and limit of segment 0: 1400 1000
Enter base and limit of segment 1: 6300 400
Enter base and limit of segment 2: 4300 1100

SEGMENT TABLE
Seg     Base    Limit
0       1400    1000
1       6300    400
2       4300    1100

Enter number of logical addresses: 3
Enter segment number and offset: 0 53
  Physical address = 1400 + 53 = 1453
Enter segment number and offset: 1 450
  TRAP: offset 450 exceeds limit 400 of segment 1
Enter segment number and offset: 2 1000
  Physical address = 4300 + 1000 = 5300
```

RESULT:

Thus segmentation-based address translation was implemented successfully.

Exp. No: 65

IMPLEMENTATION OF SEGMENTATION WITH PAGING

AIM:

To write a C program that translates logical addresses using segmentation with paging.

PROCEDURE:

1. Each segment is divided into pages; each segment has its own page table.
2. Read the page size and, for each segment, the number of pages and the frame number of each page.
3. Read a logical address as (segment number, offset within the segment).
4. Check the offset against the segment length (number of pages x page size); if it is larger, generate a trap.
5. Page number = offset / page size and page offset = offset % page size.
6. Get the frame from the page table of that segment; physical address = frame x page size + page offset.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int ps, ns, npages[5], pt[5][10], i, j, n, s, off, p, d;

    printf("Enter page size: ");
    scanf("%d", &ps);
    printf("Enter number of segments: ");
    scanf("%d", &ns);
    for (i = 0; i < ns; i++) {
        printf("Enter number of pages in segment %d: ", i);
        scanf("%d", &npages[i]);
        printf("Enter frames of segment %d: ", i);
        for (j = 0; j < npages[i]; j++) scanf("%d", &pt[i][j]);
    }
    printf("\nEnter number of logical addresses: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter segment number and offset: ");
        scanf("%d %d", &s, &off);
        if (s >= ns || off >= npages[s] * ps) {
            printf("  TRAP: invalid address\n");
            continue;
        }
        p = off / ps;
        d = off % ps;
        printf("  Segment %d -> page %d, offset %d -> frame %d -> physical address
%d\n",
               s, p, d, pt[s][p], pt[s][p] * ps + d);
    }
    return 0;
}
```

INPUT:

```
100
```

```
2
3
4 7 2
2
9 5
3
0 150
1 85
1 230
```

OUTPUT:

```
Enter page size: 100
Enter number of segments: 2
Enter number of pages in segment 0: 3
Enter frames of segment 0: 4 7 2
Enter number of pages in segment 1: 2
Enter frames of segment 1: 9 5

Enter number of logical addresses: 3
Enter segment number and offset: 0 150
  Segment 0 -> page 1, offset 50 -> frame 7 -> physical address 750
Enter segment number and offset: 1 85
  Segment 1 -> page 0, offset 85 -> frame 9 -> physical address 985
Enter segment number and offset: 1 230
  TRAP: invalid address
```

RESULT:

Thus address translation using segmentation with paging was implemented successfully.

Exp. No: 66

SIMULATION OF DEMAND PAGING AND PAGE FAULT HANDLING

AIM:

To write a C program that simulates demand paging and the steps of page-fault handling.

PROCEDURE:

1. Read the number of frames and the page reference string.
2. Initially all pages are invalid (not in memory).
3. For each reference, check the valid bit. If valid, the access continues normally.
4. If invalid, a page fault occurs: trap to the OS, find a free frame (or replace the oldest page using FIFO), read the page from disk into the frame, update the page table and restart the instruction.
5. Display the steps and the frames after each reference.
6. Display the total number of page faults and the page-fault rate.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nf, n, ref[30], frame[10], i, j, k, faults = 0, next = 0, used = 0, found;

    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &ref[i]);

    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < used; j++)
            if (frame[j] == ref[i]) found = 1;
        if (found) {
            printf("Page %d: valid bit = 1, access memory directly\n", ref[i]);
            continue;
        }
        faults++;
        printf("Page %d: valid bit = 0 -> PAGE FAULT, trap to OS\n", ref[i]);
        if (used < nf) {
            k = used++;
            printf("   free frame %d found\n", k);
        } else {
            k = next;
            next = (next + 1) % nf;
            printf("   no free frame, replace page %d in frame %d\n", frame[k], k);
        }
        frame[k] = ref[i];
        printf("   page %d read from disk into frame %d, page table updated, instruction
restarted\n", ref[i], k);
        printf("   Frames: ");
        for (j = 0; j < used; j++) printf("%d ", frame[j]);
        printf("\n");
    }
    printf("\nTotal page faults = %d, Page fault rate = %.2f\n", faults, (float)faults /
n);
```

```
    return 0;
}
```

INPUT:

```
3
8
1 2 3 1 4 2 5 1
```

OUTPUT:

```
Enter number of frames: 3
Enter length of reference string: 8
Enter reference string: 1 2 3 1 4 2 5 1
Page 1: valid bit = 0 -> PAGE FAULT, trap to OS
   free frame 0 found
   page 1 read from disk into frame 0, page table updated, instruction restarted
   Frames: 1
Page 2: valid bit = 0 -> PAGE FAULT, trap to OS
   free frame 1 found
   page 2 read from disk into frame 1, page table updated, instruction restarted
   Frames: 1 2
Page 3: valid bit = 0 -> PAGE FAULT, trap to OS
   free frame 2 found
   page 3 read from disk into frame 2, page table updated, instruction restarted
   Frames: 1 2 3
Page 1: valid bit = 1, access memory directly
Page 4: valid bit = 0 -> PAGE FAULT, trap to OS
   no free frame, replace page 1 in frame 0
   page 4 read from disk into frame 0, page table updated, instruction restarted
   Frames: 4 2 3
Page 2: valid bit = 1, access memory directly
Page 5: valid bit = 0 -> PAGE FAULT, trap to OS
   no free frame, replace page 2 in frame 1
   page 5 read from disk into frame 1, page table updated, instruction restarted
   Frames: 4 5 3
Page 1: valid bit = 0 -> PAGE FAULT, trap to OS
   no free frame, replace page 3 in frame 2
   page 1 read from disk into frame 2, page table updated, instruction restarted
   Frames: 4 5 1

Total page faults = 6, Page fault rate = 0.75
```

RESULT:

Thus demand paging and page-fault handling were simulated successfully.

Exp. No: 67

IMPLEMENTATION OF FIFO PAGE REPLACEMENT ALGORITHM

AIM:

To write a C program to implement the First In First Out (FIFO) page replacement algorithm.

PROCEDURE:

1. Read the number of frames and the page reference string.
2. Initialise all frames to -1 (empty) and a pointer to the oldest frame.
3. For each page, check whether it is already in a frame (hit).
4. On a miss, place the page in the frame pointed to by the pointer (the oldest page) and advance the pointer circularly; count a page fault.
5. Display the frames after each reference.
6. Display the total number of page faults.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nf, n, ref[30], fr[10], i, j, k, faults = 0, hit;
    int next = 0;

    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &ref[i]);
    for (j = 0; j < nf; j++) fr[j] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < nf; j++)
            if (fr[j] == ref[i]) hit = 1;
        if (!hit) {
            fr[next] = ref[i];
            next = (next + 1) % nf;
            faults++;
        }
        printf("%d\t", ref[i]);
        for (k = 0; k < nf; k++) fr[k] == -1 ? printf("- ") : printf("%d ", fr[k]);
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults (FIFO) = %d\n", faults);
    return 0;
}
```

INPUT:

```
3
20
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

OUTPUT:

```
Enter number of frames: 3
Enter length of reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref     Frames          Status
7       7 - -           Fault
0       7 0 -           Fault
1       7 0 1           Fault
2       2 0 1           Fault
0       2 0 1           Hit
3       2 3 1           Fault
0       2 3 0           Fault
4       4 3 0           Fault
2       4 2 0           Fault
3       4 2 3           Fault
0       0 2 3           Fault
3       0 2 3           Hit
2       0 2 3           Hit
1       0 1 3           Fault
2       0 1 2           Fault
0       0 1 2           Hit
1       0 1 2           Hit
7       7 1 2           Fault
0       7 0 2           Fault
1       7 0 1           Fault

Total page faults (FIFO) = 15
```

RESULT:

Thus the FIFO page replacement algorithm was implemented successfully.

Exp. No: 68

IMPLEMENTATION OF LEAST RECENTLY USED (LRU) PAGE REPLACEMENT ALGORITHM

AIM:

To write a C program to implement the Least Recently Used (LRU) page replacement algorithm.

PROCEDURE:

1. Read the number of frames and the page reference string.
2. Keep, for each frame, the time at which its page was last used.
3. For each page, if it is in a frame (hit), update its last-used time.
4. On a miss, use an empty frame if available; otherwise replace the page with the smallest last-used time (least recently used). Count a page fault.
5. Display the frames after each reference.
6. Display the total number of page faults.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nf, n, ref[30], fr[10], i, j, k, faults = 0, hit;
    int last[10], t;

    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &ref[i]);
    for (j = 0; j < nf; j++) { fr[j] = -1; last[j] = -1; }

    printf("\nRef\tFrames\t\tStatus\n");
    for (t = 0; t < n; t++) {
        hit = 0;
        for (j = 0; j < nf; j++)
            if (fr[j] == ref[t]) { hit = 1; last[j] = t; }
        if (!hit) {
            int lru = 0;
            for (j = 0; j < nf; j++) {
                if (fr[j] == -1) { lru = j; break; }
                if (last[j] < last[lru]) lru = j;
            }
            fr[lru] = ref[t];
            last[lru] = t;
            faults++;
        }
        printf("%d\t", ref[t]);
        for (k = 0; k < nf; k++) fr[k] == -1 ? printf("- ") : printf("%d ", fr[k]);
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults (LRU) = %d\n", faults);
    return 0;
}
```

INPUT:

```
3
20
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

OUTPUT:

```
Enter number of frames: 3
Enter length of reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref     Frames          Status
7       7 - -           Fault
0       7 0 -           Fault
1       7 0 1           Fault
2       2 0 1           Fault
0       2 0 1           Hit
3       2 0 3           Fault
0       2 0 3           Hit
4       4 0 3           Fault
2       4 0 2           Fault
3       4 3 2           Fault
0       0 3 2           Fault
3       0 3 2           Hit
2       0 3 2           Hit
1       1 3 2           Fault
2       1 3 2           Hit
0       1 0 2           Fault
1       1 0 2           Hit
7       1 0 7           Fault
0       1 0 7           Hit
1       1 0 7           Hit

Total page faults (LRU) = 12
```

RESULT:

Thus the LRU page replacement algorithm was implemented successfully.

Exp. No: 69

IMPLEMENTATION OF OPTIMAL PAGE REPLACEMENT ALGORITHM

AIM:

To write a C program to implement the Optimal page replacement algorithm.

PROCEDURE:

1. Read the number of frames and the page reference string.
2. For each page, check whether it is in a frame (hit).
3. On a miss, use an empty frame if available.
4. Otherwise, for every page in the frames, find when it will be used next in the future reference string.
5. Replace the page that will not be used for the longest time (or never again). Count a page fault.
6. Display the frames after each reference and the total number of page faults.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nf, n, ref[30], fr[10], i, j, k, faults = 0, hit;
    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &ref[i]);
    for (j = 0; j < nf; j++) fr[j] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < nf; j++)
            if (fr[j] == ref[i]) hit = 1;
        if (!hit) {
            int victim = -1, farthest = -1;
            for (j = 0; j < nf; j++) {
                if (fr[j] == -1) { victim = j; break; }
                int next_use = n + 1;
                for (k = i + 1; k < n; k++)
                    if (ref[k] == fr[j]) { next_use = k; break; }
                if (next_use > farthest) { farthest = next_use; victim = j; }
            }
            fr[victim] = ref[i];
            faults++;
        }
        printf("%d\t", ref[i]);
        for (k = 0; k < nf; k++) fr[k] == -1 ? printf("- ") : printf("%d ", fr[k]);
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults (Optimal) = %d\n", faults);
    return 0;
}
```

INPUT:

```
3
20
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

OUTPUT:

```
Enter number of frames: 3
Enter length of reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref     Frames          Status
7       7 - -           Fault
0       7 0 -           Fault
1       7 0 1           Fault
2       2 0 1           Fault
0       2 0 1           Hit
3       2 0 3           Fault
0       2 0 3           Hit
4       2 4 3           Fault
2       2 4 3           Hit
3       2 4 3           Hit
0       2 0 3           Fault
3       2 0 3           Hit
2       2 0 3           Hit
1       2 0 1           Fault
2       2 0 1           Hit
0       2 0 1           Hit
1       2 0 1           Hit
7       7 0 1           Fault
0       7 0 1           Hit
1       7 0 1           Hit

Total page faults (Optimal) = 9
```

RESULT:

Thus the Optimal page replacement algorithm was implemented successfully.

Exp. No: 70

IMPLEMENTATION OF LEAST FREQUENTLY USED (LFU) PAGE REPLACEMENT ALGORITHM

AIM:

To write a C program to implement the Least Frequently Used (LFU) page replacement algorithm.

PROCEDURE:

1. Read the number of frames and the page reference string.
2. Keep a use count for each frame and the time the page was loaded.
3. On a hit, increase the use count of that page.
4. On a miss, use an empty frame if available; otherwise replace the page with the smallest use count. If counts are equal, replace the page that was loaded earliest (FIFO).
5. Set the count of the new page to 1 and count a page fault.
6. Display the frames after each reference and the total number of page faults.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int nf, n, ref[30], fr[10], i, j, k, faults = 0, hit;
    int cnt[10], loaded[10];

    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &ref[i]);
    for (j = 0; j < nf; j++) { fr[j] = -1; cnt[j] = 0; }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < nf; j++)
            if (fr[j] == ref[i]) { hit = 1; cnt[j]++; }
        if (!hit) {
            int v = 0;
            for (j = 0; j < nf; j++) {
                if (fr[j] == -1) { v = j; break; }
                if (cnt[j] < cnt[v] || (cnt[j] == cnt[v] && loaded[j] < loaded[v])) v =
j;
            }
            fr[v] = ref[i];
            cnt[v] = 1;
            loaded[v] = i;
            faults++;
        }
        printf("%d\t", ref[i]);
        for (k = 0; k < nf; k++) fr[k] == -1 ? printf("- ") : printf("%d ", fr[k]);
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults (LFU) = %d\n", faults);
    return 0;
```

```
}
```

INPUT:

```
3
14
1 2 3 1 2 4 1 5 2 1 6 2 3 1
```

OUTPUT:

```
Enter number of frames: 3
Enter length of reference string: 14
Enter reference string: 1 2 3 1 2 4 1 5 2 1 6 2 3 1

Ref     Frames          Status
1       1 - -           Fault
2       1 2 -           Fault
3       1 2 3           Fault
1       1 2 3           Hit
2       1 2 3           Hit
4       1 2 4           Fault
1       1 2 4           Hit
5       1 2 5           Fault
2       1 2 5           Hit
1       1 2 5           Hit
6       1 2 6           Fault
2       1 2 6           Hit
3       1 2 3           Fault
1       1 2 3           Hit

Total page faults (LFU) = 7
```

RESULT:

Thus the LFU page replacement algorithm was implemented successfully.

Exp. No: 71

PERFORMANCE COMPARISON OF PAGE REPLACEMENT ALGORITHMS

AIM:

To write a C program that compares the FIFO, LRU, Optimal and LFU page replacement algorithms by the number of page faults for different numbers of frames.

PROCEDURE:

1. Read the page reference string.
2. Write one function per algorithm that returns the number of page faults for a given number of frames.
3. Run all four algorithms for 3 and 4 frames.
4. Display the page faults in a table.
5. Observe that Optimal gives the fewest faults and check whether FIFO shows Belady's anomaly (more faults with more frames).

PROGRAM:

```
#include <stdio.h>

int n, ref[30];

int find(int fr[], int nf, int p)
{
    for (int j = 0; j < nf; j++) if (fr[j] == p) return j;
    return -1;
}

int fifo(int nf)
{
    int fr[10], next = 0, f = 0;
    for (int j = 0; j < nf; j++) fr[j] = -1;
    for (int i = 0; i < n; i++)
        if (find(fr, nf, ref[i]) < 0) { fr[next] = ref[i]; next = (next + 1) % nf; f++;
}
    return f;
}

int lru(int nf)
{
    int fr[10], last[10], f = 0;
    for (int j = 0; j < nf; j++) { fr[j] = -1; last[j] = -1; }
    for (int i = 0; i < n; i++) {
        int k = find(fr, nf, ref[i]);
        if (k < 0) {
            k = 0;
            for (int j = 1; j < nf; j++) if (last[j] < last[k]) k = j;
            fr[k] = ref[i];
            f++;
        }
        last[k] = i;
    }
    return f;
}

int optimal(int nf)
```

```
{
    int fr[10], f = 0;
    for (int j = 0; j < nf; j++) fr[j] = -1;
    for (int i = 0; i < n; i++) {
        if (find(fr, nf, ref[i]) >= 0) continue;
        int v = 0, far = -1;
        for (int j = 0; j < nf; j++) {
            int nu = n + 1;
            if (fr[j] == -1) { v = j; break; }
            for (int k = i + 1; k < n; k++) if (ref[k] == fr[j]) { nu = k; break; }
            if (nu > far) { far = nu; v = j; }
        }
        fr[v] = ref[i];
        f++;
    }
    return f;
}

int lfu(int nf)
{
    int fr[10], cnt[10], ld[10], f = 0;
    for (int j = 0; j < nf; j++) { fr[j] = -1; cnt[j] = 0; ld[j] = -1; }
    for (int i = 0; i < n; i++) {
        int k = find(fr, nf, ref[i]);
        if (k >= 0) { cnt[k]++; continue; }
        k = 0;
        for (int j = 0; j < nf; j++) {
            if (fr[j] == -1) { k = j; break; }
            if (cnt[j] < cnt[k] || (cnt[j] == cnt[k] && ld[j] < ld[k])) k = j;
        }
        fr[k] = ref[i]; cnt[k] = 1; ld[k] = i; f++;
    }
    return f;
}

int main()
{
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &ref[i]);

    printf("\nFrames\tFIFO\tLRU\tOptimal\tLFU\n");
    for (int nf = 3; nf <= 4; nf++)
        printf("%d\t%d\t%d\t%d\t%d\n", nf, fifo(nf), lru(nf), optimal(nf), lfu(nf));
    if (fifo(4) > fifo(3))
        printf("\nBelady's anomaly observed in FIFO (more frames gave more faults)\n");
    printf("Optimal gives the minimum number of page faults.\n");
    return 0;
}
```

INPUT:

```
12
1 2 3 4 1 2 5 1 2 3 4 5
```

OUTPUT:

```
Enter length of reference string: 12
Enter reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Frames  FIFO    LRU     Optimal LFU
```

```
3       9       10      7       10
4       10      8       6       8

Belady's anomaly observed in FIFO (more frames gave more faults)
Optimal gives the minimum number of page faults.
```

RESULT:

Thus the page replacement algorithms were compared successfully. Optimal produced the fewest faults and FIFO showed Belady's anomaly for this reference string.

Exp. No: 72

IMPLEMENTATION OF GLOBAL AND LOCAL PAGE REPLACEMENT STRATEGIES

AIM:

To write a C program that compares global and local page replacement strategies for two processes sharing memory.

PROCEDURE:

1. Two processes A and B share 4 frames; each has its own reference string.
2. The references of the two processes are interleaved (A, B, A, B, ...).
3. Local replacement: each process gets a fixed 2 frames and a faulting process replaces only its own oldest page (FIFO).
4. Global replacement: a faulting process may replace the oldest page among all 4 frames, even a page of the other process.
5. Count the page faults of each process under both strategies and display them.

PROGRAM:

```
#include <stdio.h>

int n, refA[20], refB[20];

int in(int pg[], int own[], int cnt, int p, int who)
{
    for (int j = 0; j < cnt; j++) if (pg[j] == p && own[j] == who) return 1;
    return 0;
}

void local_rep(int fa[2])
{
    int fr[2][2] = {{-1, -1}, {-1, -1}}, next[2] = {0, 0};
    fa[0] = fa[1] = 0;
    for (int i = 0; i < n; i++)
        for (int w = 0; w < 2; w++) {
            int p = (w == 0) ? refA[i] : refB[i];
            if (fr[w][0] == p || fr[w][1] == p) continue;
            fr[w][next[w]] = p;
            next[w] = (next[w] + 1) % 2;
            fa[w]++;
        }
}

void global_rep(int fa[2])
{
    int pg[4], own[4], t[4], cnt = 0, time = 0;
    fa[0] = fa[1] = 0;
    for (int i = 0; i < n; i++)
        for (int w = 0; w < 2; w++) {
            int p = (w == 0) ? refA[i] : refB[i];
            time++;
            if (in(pg, own, cnt, p, w)) continue;
            int k;
            if (cnt < 4) k = cnt++;
            else {
                k = 0;
                for (int j = 1; j < 4; j++) if (t[j] < t[k]) k = j;
```

```
            }
            pg[k] = p; own[k] = w; t[k] = time;
            fa[w]++;
        }
}

int main()
{
    int L[2], G[2];
    printf("Enter number of references per process: ");
    scanf("%d", &n);
    printf("Enter reference string of process A: ");
    for (int i = 0; i < n; i++) scanf("%d", &refA[i]);
    printf("Enter reference string of process B: ");
    for (int i = 0; i < n; i++) scanf("%d", &refB[i]);

    local_rep(L);
    global_rep(G);
    printf("\nStrategy\tFaults(A)\tFaults(B)\tTotal\n");
    printf("Local (2+2)\t%d\t\t%d\t\t%d\n", L[0], L[1], L[0] + L[1]);
    printf("Global (4)\t%d\t\t%d\t\t%d\n", G[0], G[1], G[0] + G[1]);
    printf("\nLocal replacement isolates the processes; global replacement lets\n");
    printf("a process with a larger working set take frames from the other.\n");
    return 0;
}
```

INPUT:

```
8
1 2 3 1 2 3 1 2
5 5 5 6 5 6 5 6
```

OUTPUT:

```
Enter number of references per process: 8
Enter reference string of process A: 1 2 3 1 2 3 1 2
Enter reference string of process B: 5 5 5 6 5 6 5 6

Strategy        Faults(A)       Faults(B)       Total
Local (2+2)     8               2               10
Global (4)      5               3               8

Local replacement isolates the processes; global replacement lets
a process with a larger working set take frames from the other.
```

RESULT:

Thus global and local page replacement strategies were implemented and compared successfully.

Exp. No: 73

SIMULATION OF WORKING SET MANAGEMENT AND THRASHING CONTROL

AIM:

To write a C program that computes the working set of each process using a window and controls thrashing by suspending a process when the total demand exceeds the available frames.

PROCEDURE:

1. Read the window size (delta), the number of available frames and the reference strings of three processes.
2. The working set of a process at time t is the set of distinct pages referenced in the last delta references.
3. Compute the working-set size (WSS) of each process at the end of its reference string.
4. Compute the total demand D = sum of all WSS.
5. If D is greater than the available frames, thrashing will occur; suspend the process with the largest working set and recompute D.
6. Display the working sets, D and the decision.

PROGRAM:

```
#include <stdio.h>

int wss(int ref[], int n, int delta, int set[])
{
    int count = 0;
    for (int i = n - delta; i < n; i++) {
        if (i < 0) continue;
        int found = 0;
        for (int j = 0; j < count; j++) if (set[j] == ref[i]) found = 1;
        if (!found) set[count++] = ref[i];
    }
    return count;
}

int main()
{
    int delta, frames, n[3], ref[3][20], w[3], set[20], D = 0, i, j, active[3] = {1, 1,
1};

    printf("Enter window size (delta): ");
    scanf("%d", &delta);
    printf("Enter number of available frames: ");
    scanf("%d", &frames);
    for (i = 0; i < 3; i++) {
        printf("Enter length and references of P%d: ", i + 1);
        scanf("%d", &n[i]);
        for (j = 0; j < n[i]; j++) scanf("%d", &ref[i][j]);
    }
    printf("\n");
    for (i = 0; i < 3; i++) {
        w[i] = wss(ref[i], n[i], delta, set);
        printf("P%d working set = { ", i + 1);
        for (j = 0; j < w[i]; j++) printf("%d ", set[j]);
        printf("}  WSS = %d\n", w[i]);
```

```
        D += w[i];
    }
    printf("\nTotal demand D = %d, available frames = %d\n", D, frames);
    while (D > frames) {
        int big = -1;
        for (i = 0; i < 3; i++)
            if (active[i] && (big == -1 || w[i] > w[big])) big = i;
        printf("D > frames: THRASHING likely. Suspend P%d (WSS = %d)\n", big + 1,
w[big]);
        active[big] = 0;
        D -= w[big];
        printf("New total demand D = %d\n", D);
    }
    printf("D <= frames: the remaining processes can run without thrashing.\n");
    return 0;
}
```

INPUT:

```
4
6
10 1 2 1 3 2 1 4 4 3 4
8 5 6 5 7 8 9 7 6
6 2 2 3 3 2 3
```

OUTPUT:

```
Enter window size (delta): 4
Enter number of available frames: 6
Enter length and references of P1: 10 1 2 1 3 2 1 4 4 3 4
Enter length and references of P2: 8 5 6 5 7 8 9 7 6
Enter length and references of P3: 6 2 2 3 3 2 3

P1 working set = { 4 3 }  WSS = 2
P2 working set = { 8 9 7 6 }  WSS = 4
P3 working set = { 3 2 }  WSS = 2

Total demand D = 8, available frames = 6
D > frames: THRASHING likely. Suspend P2 (WSS = 4)
New total demand D = 4
D <= frames: the remaining processes can run without thrashing.
```

RESULT:

Thus working-set management and thrashing control were simulated successfully.

Exp. No: 74

IMPLEMENTATION OF FILE OPERATIONS USING OPEN(), READ(), WRITE(), AND CLOSE()

AIM:

To write a C program that performs file operations using the system calls open(), read(), write() and close().

PROCEDURE:

1. Read a line of text from the keyboard using read() on file descriptor 0 (standard input).
2. Create the file student.txt using open() with O_CREAT | O_WRONLY | O_TRUNC.
3. Write the text into the file using write() and close it.
4. Open the file again with O_WRONLY | O_APPEND and append one more line, then close it.
5. Open the file in read-only mode, read the whole contents using read() and write them to the screen using write() on file descriptor 1.
6. Close the file.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    char text[100], buf[200];
    char extra[] = "Appended line: Department of CSE\n";
    int fd, n;

    write(1, "Enter a line of text: ", 22);
    n = read(0, text, sizeof(text));

    fd = open("student.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    write(fd, text, n);
    close(fd);
    printf("%d bytes written to student.txt\n", n);

    fd = open("student.txt", O_WRONLY | O_APPEND);
    write(fd, extra, strlen(extra));
    close(fd);
    printf("Line appended to student.txt\n");

    fd = open("student.txt", O_RDONLY);
    n = read(fd, buf, sizeof(buf));
    close(fd);
    printf("Contents of student.txt (%d bytes):\n", n);
    fflush(stdout);
    write(1, buf, n);
    return 0;
}
```

INPUT:

```
Name: Priya  Reg No: 192411001
```

OUTPUT:

```
Enter a line of text: Name: Priya  Reg No: 192411001
31 bytes written to student.txt
Line appended to student.txt
Contents of student.txt (64 bytes):
Name: Priya  Reg No: 192411001
Appended line: Department of CSE
```

RESULT:

Thus file operations using open(), read(), write() and close() were performed successfully.

Exp. No: 75

IMPLEMENTATION OF FILE COPYING USING LOW-LEVEL SYSTEM CALLS

AIM:

To write a C program that copies one file to another using the low-level system calls open(), read(), write() and close().

PROCEDURE:

1. Take the source and destination file names as command-line arguments.
2. Open the source file in read-only mode; display an error if it cannot be opened.
3. Create or truncate the destination file using open() with O_CREAT | O_WRONLY | O_TRUNC.
4. Read a block of up to 64 bytes from the source using read() and write the same number of bytes to the destination using write().
5. Repeat until read() returns 0 (end of file) and count the total bytes copied.
6. Close both files and display the number of bytes copied.

PROGRAM:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    char buf[64];
    int src, dst, n, total = 0;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }
    src = open(argv[1], O_RDONLY);
    if (src < 0) { perror("Source file"); return 1; }
    dst = open(argv[2], O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (dst < 0) { perror("Destination file"); return 1; }

    while ((n = read(src, buf, sizeof(buf))) > 0) {
        write(dst, buf, n);
        total += n;
    }
    close(src);
    close(dst);
    printf("File copied from %s to %s: %d bytes\n", argv[1], argv[2], total);
    return 0;
}
```

OUTPUT:

```
$ ./prog source.txt dest.txt
File copied from source.txt to dest.txt: 86 bytes
$ cat dest.txt
Operating Systems Laboratory
File copy using system calls
```

```
read() and write() are used
```

RESULT:

Thus a file was copied to another file using low-level system calls successfully.

Exp. No: 76

IMPLEMENTATION OF DIRECTORY CREATION, TRAVERSAL, AND DELETION USING SYSTEM CALLS

AIM:

To write a C program that creates a directory, creates files inside it, traverses it and finally deletes it using system calls.

PROCEDURE:

1. Create a directory using mkdir() with permission 0755.
2. Create three files inside the directory using open() and close().
3. Open the directory using opendir() and read each entry using readdir(); display the name and type of each entry.
4. Close the directory using closedir().
5. Delete each file using unlink() and then remove the empty directory using rmdir().
6. Confirm that the directory no longer exists.

PROGRAM:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>

int main()
{
    char *dir = "labdir", path[100];
    char *files[3] = {"a.txt", "b.txt", "c.txt"};
    struct dirent *e;
    DIR *dp;
    int i;

    if (mkdir(dir, 0755) == 0) printf("Directory %s created\n", dir);
    for (i = 0; i < 3; i++) {
        sprintf(path, "%s/%s", dir, files[i]);
        close(open(path, O_CREAT | O_WRONLY, 0644));
        printf("File %s created\n", path);
    }

    printf("\nTraversing %s:\n", dir);
    dp = opendir(dir);
    while ((e = readdir(dp)) != NULL)
        printf("  %-8s %s\n", e->d_type == DT_DIR ? "[DIR]" : "[FILE]", e->d_name);
    closedir(dp);

    printf("\nDeleting:\n");
    for (i = 0; i < 3; i++) {
        sprintf(path, "%s/%s", dir, files[i]);
        if (unlink(path) == 0) printf("  %s deleted\n", path);
    }
    if (rmdir(dir) == 0) printf("  Directory %s removed\n", dir);
    if (access(dir, F_OK) != 0) printf("Verified: %s no longer exists\n", dir);
    return 0;
}
```

OUTPUT:

```
Directory labdir created
File labdir/a.txt created
File labdir/b.txt created
File labdir/c.txt created

Traversing labdir:
  [DIR]    .
  [FILE]   b.txt
  [DIR]    ..
  [FILE]   c.txt
  [FILE]   a.txt

Deleting:
  labdir/a.txt deleted
  labdir/b.txt deleted
  labdir/c.txt deleted
  Directory labdir removed
Verified: labdir no longer exists
```

RESULT:

Thus directory creation, traversal and deletion using system calls were implemented successfully.

Exp. No: 77

IMPLEMENTATION OF FILE METADATA RETRIEVAL AND MODIFICATION USING LINUX SYSTEM CALLS

AIM:

To write a C program that retrieves file metadata and modifies it (permissions, timestamps and size) using Linux system calls.

PROCEDURE:

1. Create a file report.txt with some text.
2. Use stat() to display the size, permissions, owner, last access time and last modification time.
3. Reduce the file size to 10 bytes using truncate().
4. Change the permissions to read-only for everyone (0444) using chmod().
5. Set the access and modification times to 1 January 2025 using utime().
6. Use stat() again and display the modified metadata.

PROGRAM:

```
#include <stdio.h>
#include <time.h>
#include <utime.h>
#include <unistd.h>
#include <sys/stat.h>

void show(char *f)
{
    struct stat st;
    char t[40];
    stat(f, &st);
    printf("  Size        : %ld bytes\n", (long)st.st_size);
    printf("  Permissions : %o\n", st.st_mode & 0777);
    printf("  Owner UID   : %d\n", st.st_uid);
    strftime(t, sizeof(t), "%d-%m-%Y %H:%M:%S", localtime(&st.st_mtime));
    printf("  Modified    : %s\n", t);
    strftime(t, sizeof(t), "%d-%m-%Y %H:%M:%S", localtime(&st.st_atime));
    printf("  Accessed    : %s\n", t);
}

int main()
{
    char *f = "report.txt";
    struct utimbuf ut;
    struct tm tm = {0};

    FILE *fp = fopen(f, "w");
    fprintf(fp, "Quarterly report of the OS lab\n");
    fclose(fp);

    printf("Metadata before modification:\n");
    show(f);

    truncate(f, 10);
    chmod(f, 0444);
    tm.tm_year = 2025 - 1900; tm.tm_mon = 0; tm.tm_mday = 1; tm.tm_hour = 10;
    ut.actime = ut.modtime = mktime(&tm);
```

```
    utime(f, &ut);

    printf("\nMetadata after truncate(10), chmod(0444) and utime():\n");
    show(f);
    return 0;
}
```

OUTPUT:

```
Metadata before modification:
  Size        : 31 bytes
  Permissions : 644
  Owner UID   : 0
  Modified    : 21-10-2026 09:23:25
  Accessed    : 21-10-2026 09:23:25

Metadata after truncate(10), chmod(0444) and utime():
  Size        : 10 bytes
  Permissions : 444
  Owner UID   : 0
  Modified    : 01-01-2025 10:00:00
  Accessed    : 01-01-2025 10:00:00
```

RESULT:

Thus file metadata was retrieved and modified using Linux system calls successfully.

Exp. No: 78

IMPLEMENTATION OF FILE LOCKING FOR CONCURRENT FILE ACCESS

AIM:

To write a C program that demonstrates file locking with fcntl(), so that a second process cannot write to a file while the first process holds a write lock.

PROCEDURE:

1. Create the file shared.txt.
2. Process 1 (parent) opens the file and places a write lock on it using fcntl() with F_SETLK.
3. Process 2 (child) tries to lock the same file without waiting (F_SETLK). The call fails because the file is locked; F_GETLK shows which process holds the lock.
4. Process 1 writes its data, then releases the lock using F_UNLCK.
5. Process 2 tries again, gets the lock, writes its data and releases the lock.
6. Display the final contents of the file.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int lock(int fd, int type, int cmd)
{
    struct flock fl = {0};
    fl.l_type = type;
    fl.l_whence = SEEK_SET;
    return fcntl(fd, cmd, &fl);
}

int main()
{
    int fd = open("shared.txt", O_CREAT | O_RDWR | O_TRUNC, 0644);
    char line[100];
    close(fd);

    fd = open("shared.txt", O_RDWR | O_APPEND);
    lock(fd, F_WRLCK, F_SETLK);
    printf("Process 1 (%d): write lock acquired\n", getpid());
    fflush(stdout);

    if (fork() == 0) {
        int fd2 = open("shared.txt", O_RDWR | O_APPEND);
        struct flock fl = {0};
        fl.l_type = F_WRLCK;
        if (lock(fd2, F_WRLCK, F_SETLK) == -1) {
            fcntl(fd2, F_GETLK, &fl);
            printf("Process 2 (%d): file is locked by process %d, cannot write\n",
getpid(), fl.l_pid);
            fflush(stdout);
        }
        sleep(2);
        if (lock(fd2, F_WRLCK, F_SETLK) == 0) {
            printf("Process 2 (%d): lock acquired, writing\n", getpid());
```

```
            write(fd2, "Written by process 2\n", 21);
            lock(fd2, F_UNLCK, F_SETLK);
        }
        close(fd2);
        return 0;
    }
    sleep(1);
    write(fd, "Written by process 1\n", 21);
    lock(fd, F_UNLCK, F_SETLK);
    printf("Process 1 (%d): data written, lock released\n", getpid());
    fflush(stdout);
    close(fd);
    wait(NULL);

    printf("\nFinal contents of shared.txt:\n");
    FILE *fp = fopen("shared.txt", "r");
    while (fgets(line, sizeof(line), fp)) printf("%s", line);
    fclose(fp);
    return 0;
}
```

OUTPUT:

```
Process 1 (2277): write lock acquired
Process 2 (2278): file is locked by process 2277, cannot write
Process 1 (2277): data written, lock released
Process 2 (2278): lock acquired, writing

Final contents of shared.txt:
Written by process 1
Written by process 2
```

RESULT:

Thus file locking for concurrent file access was implemented successfully.

Exp. No: 79

IMPLEMENTATION OF CONTIGUOUS FILE ALLOCATION

AIM:

To write a C program that simulates the contiguous file allocation method.

PROCEDURE:

1. Represent the disk as an array of 30 blocks, all initially free.
2. Read the number of files and, for each file, its name, starting block and length.
3. Check that all the blocks from start to start + length - 1 are free and within the disk.
4. If they are free, allocate them to the file; otherwise report that the file cannot be allocated.
5. Display the directory (file name, start block, length) and the blocks of each file.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int disk[30] = {0}, n, i, j, start[10], len[10], ok[10];
    char name[10][10];

    printf("Enter number of files: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter file name, starting block and length: ");
        scanf("%s %d %d", name[i], &start[i], &len[i]);
        ok[i] = (start[i] + len[i] <= 30);
        for (j = start[i]; ok[i] && j < start[i] + len[i]; j++)
            if (disk[j]) ok[i] = 0;
        if (ok[i]) {
            for (j = start[i]; j < start[i] + len[i]; j++) disk[j] = 1;
            printf("  %s allocated\n", name[i]);
        } else
            printf("  %s cannot be allocated (blocks not free)\n", name[i]);
    }
    printf("\nDIRECTORY\nFile\tStart\tLength\tBlocks\n");
    for (i = 0; i < n; i++) {
        if (!ok[i]) continue;
        printf("%s\t%d\t%d\t", name[i], start[i], len[i]);
        for (j = start[i]; j < start[i] + len[i]; j++) printf("%d ", j);
        printf("\n");
    }
    return 0;
}
```

INPUT:

```
4
count 0 2
tr 14 3
mail 19 6
list 15 4
```

OUTPUT:

```
Enter number of files: 4
Enter file name, starting block and length: count 0 2
  count allocated
Enter file name, starting block and length: tr 14 3
  tr allocated
Enter file name, starting block and length: mail 19 6
  mail allocated
Enter file name, starting block and length: list 15 4
  list cannot be allocated (blocks not free)

DIRECTORY
File    Start   Length  Blocks
count   0       2       0 1
tr      14      3       14 15 16
mail    19      6       19 20 21 22 23 24
```

RESULT:

Thus the contiguous file allocation method was simulated successfully.

Exp. No: 80

IMPLEMENTATION OF LINKED FILE ALLOCATION

AIM:

To write a C program that simulates the linked file allocation method.

PROCEDURE:

1. Represent the disk as an array of 30 blocks; each block stores a next-block pointer.
2. Read the number of files and, for each file, its name, the number of blocks and the block numbers (which may be scattered).
3. Check that the blocks are free, mark them as used and link each block to the next one; the last block points to -1.
4. Store the start and end blocks of each file in the directory.
5. Display the directory and follow the pointers from the start block to show the chain of each file.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int used[30] = {0}, next[30], n, i, j, k, b[10], start[10], end[10], ok[10];
    char name[10][10];

    printf("Enter number of files: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter file name and number of blocks: ");
        scanf("%s %d", name[i], &k);
        printf("Enter the block numbers: ");
        ok[i] = 1;
        for (j = 0; j < k; j++) {
            scanf("%d", &b[j]);
            if (used[b[j]]) ok[i] = 0;
        }
        if (!ok[i]) { printf("  %s: some blocks already used\n", name[i]); continue; }
        for (j = 0; j < k; j++) {
            used[b[j]] = 1;
            next[b[j]] = (j < k - 1) ? b[j + 1] : -1;
        }
        start[i] = b[0];
        end[i] = b[k - 1];
    }
    printf("\nDIRECTORY\nFile\tStart\tEnd\tChain\n");
    for (i = 0; i < n; i++) {
        if (!ok[i]) continue;
        printf("%s\t%d\t%d\t", name[i], start[i], end[i]);
        for (j = start[i]; j != -1; j = next[j])
            printf("%d%s", j, next[j] != -1 ? " -> " : " -> NULL\n");
    }
    return 0;
}
```

INPUT:

```
2
jeep 5
9 16 1 10 25
book 3
4 21 7
```

OUTPUT:

```
Enter number of files: 2
Enter file name and number of blocks: jeep 5
Enter the block numbers: 9 16 1 10 25
Enter file name and number of blocks: book 3
Enter the block numbers: 4 21 7

DIRECTORY
File    Start   End     Chain
jeep    9       25      9 -> 16 -> 1 -> 10 -> 25 -> NULL
book    4       7       4 -> 21 -> 7 -> NULL
```

RESULT:

Thus the linked file allocation method was simulated successfully.

Exp. No: 81

IMPLEMENTATION OF INDEXED FILE ALLOCATION

AIM:

To write a C program that simulates the indexed file allocation method.

PROCEDURE:

1. Represent the disk as an array of 30 blocks.
2. Read the number of files and, for each file, its name, the index block and the data blocks.
3. Check that the index block and the data blocks are free.
4. Store the list of data block numbers in the index block of the file and mark all the blocks as used.
5. Display the directory (file name and index block) and the contents of each index block.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int used[30] = {0}, n, i, j, idx[10], cnt[10], blocks[10][20], ok[10];
    char name[10][10];

    printf("Enter number of files: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter file name, index block and number of data blocks: ");
        scanf("%s %d %d", name[i], &idx[i], &cnt[i]);
        printf("Enter the data blocks: ");
        ok[i] = !used[idx[i]];
        for (j = 0; j < cnt[i]; j++) {
            scanf("%d", &blocks[i][j]);
            if (used[blocks[i][j]] || blocks[i][j] == idx[i]) ok[i] = 0;
        }
        if (!ok[i]) { printf("  %s: block already in use, not allocated\n", name[i]);
continue; }
        used[idx[i]] = 1;
        for (j = 0; j < cnt[i]; j++) used[blocks[i][j]] = 1;
        printf("  %s allocated\n", name[i]);
    }
    printf("\nDIRECTORY\nFile\tIndex block\n");
    for (i = 0; i < n; i++)
        if (ok[i]) printf("%s\t%d\n", name[i], idx[i]);
    for (i = 0; i < n; i++) {
        if (!ok[i]) continue;
        printf("\nIndex block %d of %s:\n", idx[i], name[i]);
        for (j = 0; j < cnt[i]; j++)
            printf("  entry %d -> block %d\n", j, blocks[i][j]);
    }
    return 0;
}
```

INPUT:

```
3
jeep 19 5
9 16 1 10 25
```

```
notes 4 2
7 12
temp 9 2
3 5
```

OUTPUT:

```
Enter number of files: 3
Enter file name, index block and number of data blocks: jeep 19 5
Enter the data blocks: 9 16 1 10 25
  jeep allocated
Enter file name, index block and number of data blocks: notes 4 2
Enter the data blocks: 7 12
  notes allocated
Enter file name, index block and number of data blocks: temp 9 2
Enter the data blocks: 3 5
  temp: block already in use, not allocated

DIRECTORY
File    Index block
jeep    19
notes   4

Index block 19 of jeep:
  entry 0 -> block 9
  entry 1 -> block 16
  entry 2 -> block 1
  entry 3 -> block 10
  entry 4 -> block 25

Index block 4 of notes:
  entry 0 -> block 7
  entry 1 -> block 12
```

RESULT:

Thus the indexed file allocation method was simulated successfully.

Exp. No: 82

PERFORMANCE COMPARISON OF FILE ALLOCATION METHODS

AIM:

To write a C program that compares contiguous, linked and indexed file allocation by the number of disk accesses needed to read a file sequentially and directly.

PROCEDURE:

1. Read the file size in blocks and the block number to be accessed directly.
2. Contiguous: sequential read needs n accesses; direct access to block k needs 1 access (start + k).
3. Linked: sequential read needs n accesses; direct access to block k needs k + 1 accesses because the chain must be followed.
4. Indexed: sequential read needs n + 1 accesses (index block + data); direct access needs 2 accesses (index block + data block).
5. Display the results in a table along with the external fragmentation and pointer overhead of each method.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, k;
    printf("Enter file size in blocks: ");
    scanf("%d", &n);
    printf("Enter block number to access directly (0 to %d): ", n - 1);
    scanf("%d", &k);

    printf("\nMethod\t\tSequential read\tDirect read of block %d\tExternal
frag.\tOverhead\n", k);
    printf("Contiguous\t%d\t\t%d\t\t\tYes\t\tNone\n", n, 1);
    printf("Linked\t\t%d\t\t%d\t\t\tNo\t\t1 pointer/block\n", n, k + 1);
    printf("Indexed\t\t%d\t\t%d\t\t\tNo\t\t1 index block\n", n + 1, 2);

    printf("\nContiguous is fastest for both kinds of access but suffers from external
fragmentation.\n");
    printf("Linked is good for sequential access but slow for direct access.\n");
    printf("Indexed supports direct access without external fragmentation.\n");
    return 0;
}
```

INPUT:

```
20
15
```

OUTPUT:

```
Enter file size in blocks: 20
Enter block number to access directly (0 to 19): 15
```

```
Method          Sequential read Direct read of block 15 External frag.  Overhead
Contiguous      20              1                       Yes             None
Linked          20              16                      No              1 pointer/block
Indexed         21              2                       No              1 index block

Contiguous is fastest for both kinds of access but suffers from external fragmentation.
Linked is good for sequential access but slow for direct access.
Indexed supports direct access without external fragmentation.
```

RESULT:

Thus the performance of contiguous, linked and indexed file allocation methods was compared successfully.

Exp. No: 83

IMPLEMENTATION OF AN INODE-BASED FILE METADATA RETRIEVAL UTILITY

AIM:

To write a C program that displays inode-based metadata of files (similar to the stat command) and shows that hard links share the same inode.

PROCEDURE:

1. Take one or more file names as command-line arguments.
2. For each file, call lstat() to read its inode information.
3. Display the inode number, file type, permission string (rwx), number of hard links, owner, size, number of blocks and modification time.
4. Create a hard link and a symbolic link to a file and run the program on all three to compare the inode numbers.

PROGRAM:

```
#include <stdio.h>
#include <time.h>
#include <sys/stat.h>

void perm(mode_t m, char *s)
{
    char *r = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        s[i] = (m & (1 << (8 - i))) ? r[i] : '-';
    s[9] = '\0';
}

int main(int argc, char *argv[])
{
    struct stat st;
    char p[10], t[30];
    for (int i = 1; i < argc; i++) {
        if (lstat(argv[i], &st) < 0) { perror(argv[i]); continue; }
        perm(st.st_mode, p);
        strftime(t, sizeof(t), "%d-%m-%Y %H:%M", localtime(&st.st_mtime));
        printf("File   : %s\n", argv[i]);
        printf("Inode  : %lu\n", (unsigned long)st.st_ino);
        printf("Type   : %s\n", S_ISDIR(st.st_mode) ? "Directory" :
                                S_ISLNK(st.st_mode) ? "Symbolic link" : "Regular file");
        printf("Perm   : %s\n", p);
        printf("Links  : %lu\n", (unsigned long)st.st_nlink);
        printf("Owner  : %d\n", st.st_uid);
        printf("Size   : %ld bytes, Blocks: %ld\n", (long)st.st_size,
(long)st.st_blocks);
        printf("Mtime  : %s\n\n", t);
    }
    return 0;
}
```

OUTPUT:

```
$ ./prog data.txt hardlink.txt softlink.txt
File   : data.txt
```

```
Inode  : 886580
Type   : Regular file
Perm   : rw-r--r--
Links  : 2
Owner  : 0
Size   : 16 bytes, Blocks: 8
Mtime  : 21-10-2026 09:23

File   : hardlink.txt
Inode  : 886580
Type   : Regular file
Perm   : rw-r--r--
Links  : 2
Owner  : 0
Size   : 16 bytes, Blocks: 8
Mtime  : 21-10-2026 09:23

File   : softlink.txt
Inode  : 886581
Type   : Symbolic link
Perm   : rwxrwxrwx
Links  : 1
Owner  : 0
Size   : 8 bytes, Blocks: 0
Mtime  : 21-10-2026 09:23
```

RESULT:

Thus the inode-based file metadata retrieval utility was implemented. The hard link has the same inode number as the original file, while the symbolic link has its own inode.

Exp. No: 84

IMPLEMENTATION OF FILE SEARCHING AND INDEXING FOR LARGE DIRECTORIES

AIM:

To write a C program that searches a large directory tree, builds an index of the files and finds the files whose names contain a given keyword.

PROCEDURE:

1. Create a directory tree with a large number of files (done by a setup script).
2. Traverse the tree recursively using opendir(), readdir() and stat(), and store every file path and size in an index (array).
3. Display the number of files indexed and the time taken.
4. Read a search keyword from the user.
5. Search the index using strstr() and display the matching file paths and sizes.
6. Display the number of matches and the search time.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <dirent.h>
#include <time.h>
#include <sys/stat.h>

#define MAX 5000
char path_idx[MAX][600];
long size_idx[MAX];
int count = 0;

void build(char *dir)
{
    DIR *dp = opendir(dir);
    struct dirent *e;
    struct stat st;
    char path[600];
    if (!dp) return;
    while ((e = readdir(dp)) != NULL) {
        if (!strcmp(e->d_name, ".") || !strcmp(e->d_name, "..")) continue;
        snprintf(path, sizeof(path), "%s/%s", dir, e->d_name);
        stat(path, &st);
        if (S_ISDIR(st.st_mode))
            build(path);
        else if (count < MAX) {
            strcpy(path_idx[count], path);
            size_idx[count++] = st.st_size;
        }
    }
    closedir(dp);
}

int main()
{
    char key[50];
    int i, found = 0;
    clock_t t = clock();
    build("data");
```

```
    printf("Indexed %d files in %.4f s\n", count, (double)(clock() - t) /
CLOCKS_PER_SEC);

    printf("Enter keyword to search: ");
    scanf("%s", key);
    t = clock();
    for (i = 0; i < count; i++)
        if (strstr(path_idx[i], key)) {
            printf("  %-35s %ld bytes\n", path_idx[i], size_idx[i]);
            found++;
        }
    printf("%d matching files found in %.6f s\n", found, (double)(clock() - t) /
CLOCKS_PER_SEC);
    return 0;
}
```

INPUT:

```
salary
```

OUTPUT:

```
Indexed 1502 files in 0.0022 s
Enter keyword to search: salary
  data/ece/2025/salary_april.xlsx     13 bytes
  data/cse/2024/salary_march.xlsx     13 bytes
2 matching files found in 0.000085 s
```

RESULT:

Thus file searching and indexing for a large directory was implemented successfully.

Exp. No: 85

IMPLEMENTATION OF FILE INTEGRITY CHECKING USING CHECKSUMS

AIM:

To write a C program that checks file integrity by computing a checksum (CRC-32) of a file and detecting any change in it.

PROCEDURE:

1. Create a file and write some content into it.
2. Read the file byte by byte and compute its CRC-32 checksum.
3. Store the checksum as the reference value.
4. Modify the file (simulate corruption or tampering).
5. Compute the checksum again and compare it with the reference value.
6. If the values differ, report that the file has been modified; otherwise report that it is intact.

PROGRAM:

```
#include <stdio.h>

unsigned int crc32(char *fname)
{
    FILE *fp = fopen(fname, "rb");
    unsigned int crc = 0xFFFFFFFF;
    int c;
    while ((c = fgetc(fp)) != EOF) {
        crc ^= (unsigned char)c;
        for (int k = 0; k < 8; k++)
            crc = (crc >> 1) ^ (0xEDB88320 & -(crc & 1));
    }
    fclose(fp);
    return ~crc;
}

void check(char *f, unsigned int ref)
{
    unsigned int now = crc32(f);
    printf("Current checksum  : %08X -> ", now);
    if (now == ref) printf("File is INTACT\n");
    else            printf("File has been MODIFIED\n");
}

int main()
{
    char *f = "marks.txt";
    FILE *fp = fopen(f, "w");
    fprintf(fp, "Arun 78\nBala 92\nKavi 65\n");
    fclose(fp);

    unsigned int ref = crc32(f);
    printf("Reference checksum: %08X\n", ref);
    check(f, ref);

    fp = fopen(f, "w");
    fprintf(fp, "Arun 78\nBala 99\nKavi 65\n");      /* tampered */
    fclose(fp);
    printf("\nAfter changing Bala's mark from 92 to 99:\n");
```

```
    check(f, ref);
    return 0;
}
```

OUTPUT:

```
Reference checksum: 8B112789
Current checksum  : 8B112789 -> File is INTACT

After changing Bala's mark from 92 to 99:
Current checksum  : 73F1DC7E -> File has been MODIFIED
```

RESULT:

Thus file integrity checking using a CRC-32 checksum was implemented and the modification was detected successfully.

Exp. No: 86

IMPLEMENTATION OF INCREMENTAL FILE BACKUP AND RESTORATION

AIM:

To write a C program that performs a full backup, an incremental backup of only the changed files, and restoration of a deleted file.

PROCEDURE:

1. Create a source directory with three files.
2. Full backup: copy every file from the source directory to the backup directory.
3. Modify one file in the source directory.
4. Incremental backup: for each file, compare the modification time of the source file with that of its backup copy using stat(), and copy only the files that are newer.
5. Simulate data loss by deleting a file from the source directory.
6. Restore the deleted file by copying it back from the backup directory and verify its contents.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>
#include <sys/stat.h>

void copy(char *from, char *to)
{
    FILE *in = fopen(from, "r"), *out = fopen(to, "w");
    int c;
    while ((c = fgetc(in)) != EOF) fputc(c, out);
    fclose(in);
    fclose(out);
}

int backup(int incremental)
{
    DIR *dp = opendir("src");
    struct dirent *e;
    struct stat s, b;
    char sp[300], bp[300];
    int copied = 0;
    while ((e = readdir(dp)) != NULL) {
        if (e->d_name[0] == '.') continue;
        sprintf(sp, "src/%s", e->d_name);
        sprintf(bp, "backup/%s", e->d_name);
        stat(sp, &s);
        if (!incremental || stat(bp, &b) != 0 || s.st_mtime > b.st_mtime) {
            copy(sp, bp);
            printf("  copied %s\n", e->d_name);
            copied++;
        }
    }
    closedir(dp);
    return copied;
}
```

```
int main()
{
    char line[100];
    mkdir("src", 0755);
    mkdir("backup", 0755);
    FILE *fp;
    fp = fopen("src/a.txt", "w"); fprintf(fp, "file A version 1\n"); fclose(fp);
    fp = fopen("src/b.txt", "w"); fprintf(fp, "file B version 1\n"); fclose(fp);
    fp = fopen("src/c.txt", "w"); fprintf(fp, "file C version 1\n"); fclose(fp);

    printf("Full backup:\n");
    printf("  %d files backed up\n", backup(0));

    sleep(1);
    fp = fopen("src/b.txt", "w"); fprintf(fp, "file B version 2\n"); fclose(fp);
    printf("\nb.txt modified. Incremental backup:\n");
    printf("  %d file(s) backed up\n", backup(1));

    unlink("src/c.txt");
    printf("\nc.txt deleted from src (data loss)\n");
    copy("backup/c.txt", "src/c.txt");
    fp = fopen("src/c.txt", "r");
    fgets(line, sizeof(line), fp);
    fclose(fp);
    printf("c.txt restored from backup. Contents: %s", line);
    return 0;
}
```

OUTPUT:

```
Full backup:
  copied b.txt
  copied c.txt
  copied a.txt
  3 files backed up

b.txt modified. Incremental backup:
  copied b.txt
  1 file(s) backed up

c.txt deleted from src (data loss)
c.txt restored from backup. Contents: file C version 1
```

RESULT:

Thus full and incremental file backup and restoration were implemented successfully.

Exp. No: 87

IMPLEMENTATION OF DISK SCHEDULING USING FCFS

AIM:

To write a C program to simulate the First Come First Served (FCFS) disk scheduling algorithm.

PROCEDURE:

1. Read the number of requests, the request queue (cylinder numbers) and the initial head position.
2. Serve the requests in the order in which they arrive.
3. For each request, add the distance |request - head| to the total head movement and move the head to the request.
4. Display the order of service and the total head movement.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, req[20], head, i, total = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (i = 0; i < n; i++) {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal head movement = %d cylinders\n", total);
    return 0;
}
```

INPUT:

```
8
98 183 37 122 14 124 65 67
53
```

OUTPUT:

```
Enter number of requests: 8
Enter the request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Order of service: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total head movement = 640 cylinders
```

RESULT:

Thus the FCFS disk scheduling algorithm was simulated successfully.

Exp. No: 88

IMPLEMENTATION OF DISK SCHEDULING USING SSTF

AIM:

To write a C program to simulate the Shortest Seek Time First (SSTF) disk scheduling algorithm.

PROCEDURE:

1. Read the number of requests, the request queue and the initial head position.
2. Among the pending requests, select the one closest to the current head position.
3. Add the distance to the total head movement, move the head there and mark the request as served.
4. Repeat until all requests are served.
5. Display the order of service and the total head movement.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, req[20], done[20] = {0}, head, i, k, c, total = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (c = 0; c < n; c++) {
        k = -1;
        for (i = 0; i < n; i++)
            if (!done[i] && (k == -1 || abs(req[i] - head) < abs(req[k] - head)))
                k = i;
        total += abs(req[k] - head);
        head = req[k];
        done[k] = 1;
        printf(" -> %d", head);
    }
    printf("\nTotal head movement = %d cylinders\n", total);
    return 0;
}
```

INPUT:

```
8
98 183 37 122 14 124 65 67
53
```

OUTPUT:

```
Enter number of requests: 8
```

```
Enter the request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Order of service: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movement = 236 cylinders
```

RESULT:

Thus the SSTF disk scheduling algorithm was simulated successfully.

Exp. No: 89

IMPLEMENTATION OF DISK SCHEDULING USING SCAN AND C-SCAN

AIM:

To write a C program to simulate the SCAN and C-SCAN disk scheduling algorithms.

PROCEDURE:

1. Read the number of requests, the request queue, the head position and the disk size; the head moves towards higher cylinders.
2. Sort the requests in ascending order.
3. SCAN: serve all requests above the head moving up to the last cylinder (disk size - 1), then reverse and serve the requests below the head.
4. C-SCAN: serve all requests above the head up to the last cylinder, jump back to cylinder 0, and continue serving the remaining requests in the upward direction.
5. Compute the total head movement for each algorithm (the C-SCAN return jump is counted) and display the order of service.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
}

int main()
{
    int n, req[20], head, size, i, pos, total, h;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size (number of cylinders): ");
    scanf("%d", &size);
    sort(req, n);
    for (pos = 0; pos < n && req[pos] < head; pos++);

    /* SCAN */
    total = 0; h = head;
    printf("\nSCAN order  : %d", h);
    for (i = pos; i < n; i++) { total += abs(req[i] - h); h = req[i]; printf(" -> %d",
h); }
    total += (size - 1) - h; h = size - 1; printf(" -> %d", h);
    for (i = pos - 1; i >= 0; i--) { total += abs(req[i] - h); h = req[i]; printf(" ->
%d", h); }
    printf("\nSCAN total head movement   = %d cylinders\n", total);

    /* C-SCAN */
```

```
    total = 0; h = head;
    printf("\nC-SCAN order: %d", h);
    for (i = pos; i < n; i++) { total += abs(req[i] - h); h = req[i]; printf(" -> %d",
h); }
    total += (size - 1) - h; printf(" -> %d", size - 1);
    total += size - 1; h = 0; printf(" -> 0");
    for (i = 0; i < pos; i++) { total += req[i] - h; h = req[i]; printf(" -> %d", h); }
    printf("\nC-SCAN total head movement = %d cylinders\n", total);
    return 0;
}
```

INPUT:

```
8
98 183 37 122 14 124 65 67
53
200
```

OUTPUT:

```
Enter number of requests: 8
Enter the request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size (number of cylinders): 200

SCAN order  : 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
SCAN total head movement   = 331 cylinders

C-SCAN order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
C-SCAN total head movement = 382 cylinders
```

RESULT:

Thus the SCAN and C-SCAN disk scheduling algorithms were simulated successfully.

Exp. No: 90

IMPLEMENTATION OF DISK SCHEDULING USING LOOK AND C-LOOK

AIM:

To write a C program to simulate the LOOK and C-LOOK disk scheduling algorithms.

PROCEDURE:

1. Read the number of requests, the request queue and the head position; the head moves towards higher cylinders.
2. Sort the requests in ascending order.
3. LOOK: serve the requests above the head up to the largest request only (not the end of the disk), then reverse and serve the requests below the head.
4. C-LOOK: serve the requests above the head up to the largest request, jump to the smallest request and continue upward.
5. Compute the total head movement for each algorithm (the C-LOOK jump is counted) and display the order of service.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
}

int main()
{
    int n, req[20], head, i, pos, total, h;

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    sort(req, n);
    for (pos = 0; pos < n && req[pos] < head; pos++);

    /* LOOK */
    total = 0; h = head;
    printf("\nLOOK order  : %d", h);
    for (i = pos; i < n; i++) { total += abs(req[i] - h); h = req[i]; printf(" -> %d",
h); }
    for (i = pos - 1; i >= 0; i--) { total += abs(req[i] - h); h = req[i]; printf(" ->
%d", h); }
    printf("\nLOOK total head movement   = %d cylinders\n", total);

    /* C-LOOK */
    total = 0; h = head;
    printf("\nC-LOOK order: %d", h);
```

```
    for (i = pos; i < n; i++) { total += abs(req[i] - h); h = req[i]; printf(" -> %d",
h); }
    for (i = 0; i < pos; i++) { total += abs(req[i] - h); h = req[i]; printf(" -> %d",
h); }
    printf("\nC-LOOK total head movement = %d cylinders\n", total);
    return 0;
}
```

INPUT:

```
8
98 183 37 122 14 124 65 67
53
```

OUTPUT:

```
Enter number of requests: 8
Enter the request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53

LOOK order  : 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
LOOK total head movement   = 299 cylinders

C-LOOK order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37
C-LOOK total head movement = 322 cylinders
```

RESULT:

Thus the LOOK and C-LOOK disk scheduling algorithms were simulated successfully.

Exp. No: 91

PERFORMANCE COMPARISON OF DISK SCHEDULING ALGORITHMS USING TOTAL HEAD MOVEMENT

AIM:

To write a C program that compares the FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK disk scheduling algorithms using total head movement.

PROCEDURE:

1. Read the request queue, the initial head position and the disk size.
2. Write a function for each algorithm that returns the total head movement.
3. Call all six functions with the same input.
4. Display the total head movement and the average seek distance per request for each algorithm.
5. Identify the algorithm with the least head movement.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int n, req[20], size;

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
}

int fcfs(int h)
{
    int t = 0;
    for (int i = 0; i < n; i++) { t += abs(req[i] - h); h = req[i]; }
    return t;
}

int sstf(int h)
{
    int done[20] = {0}, t = 0;
    for (int c = 0; c < n; c++) {
        int k = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && (k == -1 || abs(req[i] - h) < abs(req[k] - h))) k = i;
        t += abs(req[k] - h); h = req[k]; done[k] = 1;
    }
    return t;
}

/* type: 1 = SCAN, 2 = C-SCAN, 3 = LOOK, 4 = C-LOOK (head moving up) */
int sweep(int h, int type)
{
    int s[20], i, pos, t = 0;
    for (i = 0; i < n; i++) s[i] = req[i];
    sort(s, n);
    for (pos = 0; pos < n && s[pos] < h; pos++);
    for (i = pos; i < n; i++) { t += s[i] - h; h = s[i]; }
```

```
    if (pos == 0) return t;
    if (type == 1 || type == 2) { t += (size - 1) - h; h = size - 1; }
    if (type == 1 || type == 3)
        for (i = pos - 1; i >= 0; i--) { t += h - s[i]; h = s[i]; }
    else {
        if (type == 2) { t += h; h = 0; }
        for (i = 0; i < pos; i++) { t += abs(s[i] - h); h = s[i]; }
    }
    return t;
}

int main()
{
    int head, i, r[6], best = 0;
    char *name[6] = {"FCFS", "SSTF", "SCAN", "C-SCAN", "LOOK", "C-LOOK"};

    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request queue: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &size);

    r[0] = fcfs(head);
    r[1] = sstf(head);
    for (i = 1; i <= 4; i++) r[i + 1] = sweep(head, i);

    printf("\nAlgorithm\tHead movement\tAverage seek\n");
    for (i = 0; i < 6; i++) {
        printf("%-8s\t%d\t\t%.2f\n", name[i], r[i], (float)r[i] / n);
        if (r[i] < r[best]) best = i;
    }
    printf("\nLeast head movement: %s\n", name[best]);
    return 0;
}
```

INPUT:

```
8
98 183 37 122 14 124 65 67
53
200
```

OUTPUT:

```
Enter number of requests: 8
Enter the request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Algorithm       Head movement   Average seek
FCFS            640             80.00
SSTF            236             29.50
SCAN            331             41.38
C-SCAN          382             47.75
LOOK            299             37.38
C-LOOK          322             40.25

Least head movement: SSTF
```

RESULT:

Thus the disk scheduling algorithms were compared using total head movement successfully.

Exp. No: 92

IMPLEMENTATION OF AN ADAPTIVE DISK SCHEDULING ALGORITHM BASED ON REQUEST PATTERNS

AIM:

To write a C program that implements an adaptive disk scheduling algorithm, which studies the pattern of the request queue and selects a suitable algorithm.

PROCEDURE:

1. Read the request queue, the head position and the disk size.
2. Analyse the queue: count the requests and find how many lie above and below the head and how widely they are spread.
3. Rule 1: if the queue is short (3 requests or fewer), use FCFS because there is little to gain from reordering.
4. Rule 2: if most requests (75% or more) lie on one side of the head, use LOOK, which sweeps in that direction.
5. Rule 3: otherwise, for a heavy and spread-out load, use C-LOOK, which gives uniform waiting times; compare it with SSTF and choose the one with less head movement.
6. Display the analysis, the selected algorithm and its total head movement for two different request patterns.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int n, req[20], size;

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
}

int fcfs(int h)
{
    int t = 0;
    for (int i = 0; i < n; i++) { t += abs(req[i] - h); h = req[i]; }
    return t;
}

int sstf(int h)
{
    int done[20] = {0}, t = 0;
    for (int c = 0; c < n; c++) {
        int k = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && (k == -1 || abs(req[i] - h) < abs(req[k] - h))) k = i;
        t += abs(req[k] - h); h = req[k]; done[k] = 1;
    }
    return t;
}
```

```
/* type: 1 = SCAN, 2 = C-SCAN, 3 = LOOK, 4 = C-LOOK (head moving up) */
int sweep(int h, int type)
{
    int s[20], i, pos, t = 0;
    for (i = 0; i < n; i++) s[i] = req[i];
    sort(s, n);
    for (pos = 0; pos < n && s[pos] < h; pos++);
    for (i = pos; i < n; i++) { t += s[i] - h; h = s[i]; }
    if (pos == 0) return t;
    if (type == 1 || type == 2) { t += (size - 1) - h; h = size - 1; }
    if (type == 1 || type == 3)
        for (i = pos - 1; i >= 0; i--) { t += h - s[i]; h = s[i]; }
    else {
        if (type == 2) { t += h; h = 0; }
        for (i = 0; i < pos; i++) { t += abs(s[i] - h); h = s[i]; }
    }
    return t;
}

void adaptive(int head)
{
    int i, above = 0, below = 0, lo = 1000, hi = 0, mv;
    for (i = 0; i < n; i++) {
        if (req[i] >= head) above++; else below++;
        if (req[i] < lo) lo = req[i];
        if (req[i] > hi) hi = req[i];
    }
    printf("Requests = %d, above head = %d, below head = %d, spread = %d\n", n, above,
below, hi - lo);
    if (n <= 3) {
        mv = fcfs(head);
        printf("Light load -> selected FCFS, head movement = %d\n", mv);
    } else if (above * 4 >= n * 3 || below * 4 >= n * 3) {
        mv = sweep(head, 3);
        printf("Requests mostly on one side -> selected LOOK, head movement = %d\n",
mv);
    } else {
        int c = sweep(head, 4), s = sstf(head);
        if (s < c) printf("Heavy mixed load -> selected SSTF (%d) over C-LOOK (%d)\n",
s, c);
        else       printf("Heavy mixed load -> selected C-LOOK (%d) over SSTF (%d)\n",
c, s);
    }
}

int main()
{
    int head, i, sets;
    size = 200;
    printf("Enter number of request patterns: ");
    scanf("%d", &sets);
    while (sets--) {
        printf("\nEnter number of requests: ");
        scanf("%d", &n);
        printf("Enter the request queue: ");
        for (i = 0; i < n; i++) scanf("%d", &req[i]);
        printf("Enter head position: ");
        scanf("%d", &head);
        adaptive(head);
    }
    return 0;
}
```

INPUT:

```
3
3
120 40 90
60
6
110 140 160 125 190 70
100
8
98 183 37 122 14 124 65 67
100
```

OUTPUT:

```
Enter number of request patterns: 3

Enter number of requests: 3
Enter the request queue: 120 40 90
Enter head position: 60
Requests = 3, above head = 2, below head = 1, spread = 80
Light load -> selected FCFS, head movement = 190

Enter number of requests: 6
Enter the request queue: 110 140 160 125 190 70
Enter head position: 100
Requests = 6, above head = 5, below head = 1, spread = 120
Requests mostly on one side -> selected LOOK, head movement = 210

Enter number of requests: 8
Enter the request queue: 98 183 37 122 14 124 65 67
Enter head position: 100
Requests = 8, above head = 3, below head = 5, spread = 169
Heavy mixed load -> selected SSTF (307) over C-LOOK (336)
```

RESULT:

Thus an adaptive disk scheduling algorithm that selects FCFS, LOOK, C-LOOK or SSTF based on the request pattern was implemented successfully.

Exp. No: 93

IMPLEMENTATION OF A FILE CACHE SIMULATOR FOR REPEATED FILE ACCESS

AIM:

To write a C program that simulates a file (block) cache with LRU replacement for repeated file accesses and measures the hit ratio and the time saved.

PROCEDURE:

1. Read the cache size (number of blocks) and the sequence of file block requests.
2. For each request, search the cache. If the block is present it is a cache HIT (time 1 ms).
3. If it is absent it is a MISS: the block is read from disk (time 10 ms) and placed in the cache; if the cache is full, the least recently used block is replaced.
4. Update the last-used time of the block on every access.
5. Display the cache contents after each access, the hit ratio and the total access time with and without the cache.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int size, n, i, j, req[30], cache[10], last[10], used = 0, hits = 0, time_ms = 0;

    printf("Enter cache size (blocks): ");
    scanf("%d", &size);
    printf("Enter number of block requests: ");
    scanf("%d", &n);
    printf("Enter block requests: ");
    for (i = 0; i < n; i++) scanf("%d", &req[i]);

    printf("\nBlock\tResult\tCache\n");
    for (i = 0; i < n; i++) {
        int k = -1, hit = 0;
        for (j = 0; j < used; j++) if (cache[j] == req[i]) k = j;
        if (k != -1) {
            hit = 1;
            hits++;
            time_ms += 1;
        } else {
            time_ms += 10;
            if (used < size) k = used++;
            else {
                k = 0;
                for (j = 1; j < size; j++) if (last[j] < last[k]) k = j;
            }
            cache[k] = req[i];
        }
        last[k] = i;
        printf("%d\t%s\t", req[i], hit ? "HIT" : "MISS");
        for (j = 0; j < used; j++) printf("%d ", cache[j]);
        printf("\n");
    }
    printf("\nHits = %d, Misses = %d, Hit ratio = %.2f\n", hits, n - hits, (float)hits /
n);
    printf("Access time with cache    = %d ms\n", time_ms);
```

```
    printf("Access time without cache = %d ms\n", n * 10);
    return 0;
}
```

INPUT:

```
3
12
1 2 3 1 2 4 1 2 5 1 2 3
```

OUTPUT:

```
Enter cache size (blocks): 3
Enter number of block requests: 12
Enter block requests: 1 2 3 1 2 4 1 2 5 1 2 3

Block   Result  Cache
1       MISS    1
2       MISS    1 2
3       MISS    1 2 3
1       HIT     1 2 3
2       HIT     1 2 3
4       MISS    1 2 4
1       HIT     1 2 4
2       HIT     1 2 4
5       MISS    1 2 5
1       HIT     1 2 5
2       HIT     1 2 5
3       MISS    1 2 3

Hits = 6, Misses = 6, Hit ratio = 0.50
Access time with cache    = 66 ms
Access time without cache = 120 ms
```

RESULT:

Thus the file cache simulator with LRU replacement was implemented and the benefit of caching was measured successfully.

Exp. No: 94

IMPLEMENTATION OF NETWORK FILE SHARING USING NFS IN A LINUX ENVIRONMENT

AIM:

To share a directory over the network using the Network File System (NFS) in Linux and access it from a client machine.

PROCEDURE:

1. On the server, install the NFS server package (nfs-kernel-server).
2. Create the directory to be shared and set its ownership and permissions.
3. Add an entry for the directory and the client network in /etc/exports.
4. Export the directory using exportfs -a and restart the NFS service.
5. On the client, install nfs-common, create a mount point and mount the shared directory using mount -t nfs.
6. Create a file from the client and verify that it is visible on the server; check the mount using df -h.
7. Unmount the share using umount when finished.

COMMANDS:

```
# ---------- On the NFS server (e.g. 192.168.1.10) ----------
sudo apt install -y nfs-kernel-server
sudo mkdir -p /srv/nfs/share
sudo chown nobody:nogroup /srv/nfs/share
sudo chmod 777 /srv/nfs/share
echo "/srv/nfs/share 192.168.1.0/24(rw,sync,no_subtree_check)" | sudo tee -a
/etc/exports
sudo exportfs -a
sudo systemctl restart nfs-kernel-server
sudo exportfs -v
# ---------- On the NFS client ----------
sudo apt install -y nfs-common
showmount -e 192.168.1.10
sudo mkdir -p /mnt/nfs_share
sudo mount -t nfs 192.168.1.10:/srv/nfs/share /mnt/nfs_share
df -h /mnt/nfs_share
echo "Hello from client" > /mnt/nfs_share/client.txt
# ---------- On the server ----------
cat /srv/nfs/share/client.txt
# ---------- On the client ----------
sudo umount /mnt/nfs_share
```

OUTPUT:

Note: These commands need a lab setup (an NFS server and a client machine on the same network). Execute them in the laboratory and record the output.

RESULT:

Thus a directory was shared using NFS and accessed from the client machine successfully.

Exp. No: 95

IMPLEMENTATION OF FILE STORAGE AND RETRIEVAL USING A CLOUD STORAGE SERVICE

AIM:

To store and retrieve files using a cloud storage service (Amazon S3) from the Linux command line.

PROCEDURE:

1. Install the AWS command line interface (AWS CLI).
2. Configure the CLI with the access key, secret key and region of the lab account using aws configure.
3. Create a storage bucket using aws s3 mb.
4. Upload a file to the bucket using aws s3 cp and list the bucket contents using aws s3 ls.
5. Delete the local copy and download the file back from the bucket.
6. Verify the downloaded file, then delete the object and the bucket to clean up.

COMMANDS:

```
sudo apt install -y awscli
aws configure
echo "OS lab cloud storage test" > notes.txt
aws s3 mb s3://oslab-storage-demo
aws s3 cp notes.txt s3://oslab-storage-demo/notes.txt
aws s3 ls s3://oslab-storage-demo
rm notes.txt
aws s3 cp s3://oslab-storage-demo/notes.txt notes.txt
cat notes.txt
aws s3 rm s3://oslab-storage-demo/notes.txt
aws s3 rb s3://oslab-storage-demo
```

OUTPUT:

Note: These commands need a lab setup (an AWS account with S3 access; any S3-compatible service such as MinIO can also be used). Execute them in the laboratory and record the output.

RESULT:

Thus files were stored in and retrieved from a cloud storage service successfully.

Exp. No: 96

IMPLEMENTATION OF FILE TRANSFER BETWEEN LINUX SYSTEMS USING NETWORK PROTOCOLS

AIM:

To write C programs that transfer a file between two Linux systems using the TCP network protocol (sockets).

PROCEDURE:

1. Server: create a TCP socket using socket(), bind it to port 5000 using bind() and wait for a connection using listen() and accept().
2. Client: create a socket and connect to the server's IP address and port using connect().
3. Client: open the file to be sent, read it in blocks and send each block using write() on the socket.
4. Server: receive the blocks using read() and write them into a new file until the client closes the connection.
5. Close the sockets and verify that the received file is identical to the original using cmp or md5sum.
6. To transfer between two machines, run the server on one system and give its IP address to the client on the other; here both are run on the same system using 127.0.0.1.

PROGRAM:

server.c

```
/* server.c - receives a file over TCP */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int sfd, cfd, n, total = 0;
    struct sockaddr_in addr;
    char buf[256];

    sfd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);
    bind(sfd, (struct sockaddr *)&addr, sizeof(addr));
    listen(sfd, 1);
    printf("Server: waiting for connection on port 5000\n");
    fflush(stdout);

    cfd = accept(sfd, NULL, NULL);
    printf("Server: client connected\n");
    FILE *fp = fopen("received.txt", "w");
    while ((n = read(cfd, buf, sizeof(buf))) > 0) {
        fwrite(buf, 1, n, fp);
        total += n;
    }
```

```
    fclose(fp);
    printf("Server: %d bytes received and saved as received.txt\n", total);
    close(cfd);
    close(sfd);
    return 0;
}
```

client.c

```
/* client.c - sends a file over TCP */
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>

int main(int argc, char *argv[])
{
    int fd, n, total = 0;
    struct sockaddr_in addr;
    char buf[256];

    if (argc != 3) { printf("Usage: %s server_ip file\n", argv[0]); return 1; }
    FILE *fp = fopen(argv[2], "r");
    if (!fp) { perror("file"); return 1; }

    fd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);
    inet_pton(AF_INET, argv[1], &addr.sin_addr);
    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect");
        return 1;
    }
    printf("Client: connected to %s\n", argv[1]);
    while ((n = fread(buf, 1, sizeof(buf), fp)) > 0) {
        write(fd, buf, n);
        total += n;
    }
    printf("Client: %d bytes of %s sent\n", total, argv[2]);
    fclose(fp);
    close(fd);
    return 0;
}
```

OUTPUT:

```
$ ./server &
Server: waiting for connection on port 5000
$ ./client 127.0.0.1 marks.csv
Client: connected to 127.0.0.1
Client: 55 bytes of marks.csv sent
Server: client connected
Server: 55 bytes received and saved as received.txt
$ cat received.txt
Roll No,Name,Marks
101,Arun,78
102,Bala,92
103,Kavi,65
$ md5sum marks.csv received.txt
00f0d81524c6f5964f7155df6fb86e27  marks.csv
00f0d81524c6f5964f7155df6fb86e27  received.txt
```

RESULT:

Thus a file was transferred between systems using TCP sockets and the received file was verified to be identical to the original.

Exp. No: 97

AUTOMATION OF LINUX SYSTEM ADMINISTRATION TASKS USING SHELL SCRIPTS AND CRON

AIM:

To automate Linux system administration tasks (disk check, cleanup of old files and log rotation) using a shell script and schedule it using cron.

PROCEDURE:

1. Write a shell script that records the date, disk usage and memory usage in a log file.
2. In the same script, delete temporary files older than 7 days from a temp directory using find -mtime +7 -delete.
3. Rotate the log: if the log file grows beyond a size limit, compress it with a date stamp and start a new one.
4. Give the script execute permission and run it once manually to test it.
5. Schedule the script using crontab -e with the entry 0 2 * * * (run every day at 2:00 AM).
6. Verify the cron entry using crontab -l and check the log file after the scheduled time.

PROGRAM:

admin_task.sh

```
#!/bin/bash
# Daily administration task - scheduled through cron
LOG=admin.log
TMP=tempfiles
MAX_SIZE=2000           # bytes

echo "===== Run at $(date '+%d-%m-%Y %H:%M:%S') =====" >> $LOG
echo "Disk usage of / : $(df -h / | awk 'NR==2 {print $5}')" >> $LOG
echo "Memory used     : $(free -m | awk '/Mem:/ {print $3 " MB of " $2 " MB"}')" >> $LOG

count=$(find $TMP -type f -mtime +7 | wc -l)
find $TMP -type f -mtime +7 -delete
echo "Old temp files deleted: $count" >> $LOG

size=$(stat -c %s $LOG)
if [ $size -gt $MAX_SIZE ]; then
    gzip -c $LOG > ${LOG}_$(date +%Y%m%d).gz
    > $LOG
    echo "Log rotated" >> $LOG
fi
echo "Task completed" >> $LOG
```

crontab_entry.txt

```
# crontab -e  (add the following line)
# m  h  dom mon dow  command
0  2  *   *   *    /home/student/oslab/admin_task.sh
```

OUTPUT:

```
$ ls tempfiles
new.tmp  old_1.tmp  old_2.tmp  old_3.tmp
```

```
$ ./admin_task.sh
$ cat admin.log
===== Run at 21-10-2026 09:23:51 =====
Disk usage of / : 31%
Memory used     : 491 MB of 8029 MB
Old temp files deleted: 3
Task completed
$ ls tempfiles
new.tmp
```

Note: The script was run once manually. After adding the crontab entry, check that it is listed by crontab -l and that admin.log is updated at the scheduled time.

RESULT:

Thus Linux system administration tasks were automated using a shell script and scheduled using cron.

Exp. No: 98

CREATION, CONFIGURATION, AND MANAGEMENT OF CONTAINERS USING DOCKER OR PODMAN

AIM:

To create, configure and manage containers using Docker.

PROCEDURE:

1. Install Docker and start the Docker service; check the installation using docker --version.
2. Pull the nginx image from Docker Hub using docker pull and list the images using docker images.
3. Run a container in the background with port mapping 8080:80 and a name using docker run -d -p --name.
4. List the running containers using docker ps and test the web server using curl.
5. Run a command inside the container using docker exec and view its logs using docker logs.
6. Limit the resources of the container (memory and CPU) using docker update and check the usage with docker stats.
7. Stop and remove the container using docker stop and docker rm.

COMMANDS:

```
sudo apt install -y docker.io
sudo systemctl start docker
docker --version
docker pull nginx:latest
docker images
docker run -d -p 8080:80 --name webserver nginx
docker ps
curl -s http://localhost:8080 | head -4
docker exec webserver cat /etc/os-release | head -2
docker logs webserver | tail -2
docker update --memory 256m --memory-swap 256m --cpus 0.5 webserver
docker stats --no-stream webserver
docker stop webserver
docker rm webserver
docker ps -a
```

OUTPUT:

Note: These commands need a lab setup (a Linux machine with Docker installed and Internet access; Podman accepts the same commands with podman in place of docker). Execute them in the laboratory and record the output.

RESULT:

Thus containers were created, configured and managed using Docker successfully.

Exp. No: 99

CREATION, CONFIGURATION, AND PERFORMANCE EVALUATION OF VIRTUAL MACHINES USING KVM/QEMU OR VMWARE

AIM:

To create and configure a virtual machine using KVM/QEMU and evaluate its performance.

PROCEDURE:

1. Check that the CPU supports hardware virtualisation using egrep -c '(vmx|svm)' /proc/cpuinfo and kvm-ok.
2. Install the KVM, QEMU, libvirt and virt-install packages and start the libvirtd service.
3. Create a virtual machine with 2 vCPUs, 2 GB RAM and a 10 GB disk from an Ubuntu ISO image using virt-install.
4. List and manage the VM using virsh list, virsh dominfo, virsh shutdown and virsh start.
5. Change the memory of the VM using virsh setmaxmem and virsh setmem, and the vCPUs using virsh setvcpus.
6. Evaluate performance: run the same CPU benchmark (sysbench) on the host and inside the VM and compare the results; check VM statistics using virsh domstats.

COMMANDS:

```
egrep -c '(vmx|svm)' /proc/cpuinfo
sudo apt install -y qemu-kvm libvirt-daemon-system virtinst cpu-checker sysbench
kvm-ok
sudo systemctl enable --now libvirtd
sudo virt-install --name osvm --vcpus 2 --memory 2048 --disk size=10 --cdrom ~/ubuntu-
24.04-live-server-amd64.iso --os-variant ubuntu24.04 --graphics vnc
virsh list --all
virsh dominfo osvm
virsh shutdown osvm
virsh setmaxmem osvm 3G --config
virsh setmem osvm 3G --config
virsh setvcpus osvm 3 --config --maximum
virsh setvcpus osvm 3 --config
virsh start osvm
virsh domstats osvm --cpu-total --balloon
# Performance: run on the host, then inside the VM, and compare
sysbench cpu --threads=2 --time=10 run | grep "events per second"
```

OUTPUT:

Note: These commands need a lab setup (a Linux host whose CPU supports virtualisation (Intel VT-x or AMD-V) and an Ubuntu ISO image; VMware Workstation can be used through its graphical wizard instead). Execute them in the laboratory and record the output.

RESULT:

Thus a virtual machine was created and configured using KVM/QEMU and its performance was evaluated and compared with the host.

Exp. No: 100

DEPLOYMENT OF A CONTAINERIZED APPLICATION USING KUBERNETES AND EVALUATION OF RESOURCE UTILIZATION

AIM:

To deploy a containerised application on Kubernetes and evaluate its resource utilisation.

PROCEDURE:

1. Install kubectl and minikube, and start a single-node cluster using minikube start.
2. Write a deployment file (web-deploy.yaml) that runs 2 replicas of the nginx container with CPU and memory requests and limits, and a NodePort service.
3. Deploy the application using kubectl apply -f and check the pods, deployment and service.
4. Access the application through the service URL.
5. Enable the metrics server and view CPU and memory usage of the pods and the node using kubectl top.
6. Scale the deployment to 4 replicas, observe the change in resource utilisation and then delete the deployment.

PROGRAM:

web-deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 250m
            memory: 128Mi
---
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
```

```
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Commands:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
sudo snap install kubectl --classic
minikube start --driver=docker
kubectl get nodes
kubectl apply -f web-deploy.yaml
kubectl get deployments
kubectl get pods -o wide
kubectl get service web-service
curl -s $(minikube service web-service --url) | head -4
minikube addons enable metrics-server
kubectl top nodes
kubectl top pods
kubectl scale deployment web --replicas=4
kubectl get pods
kubectl top pods
kubectl delete -f web-deploy.yaml
minikube stop
```

OUTPUT:

Note: These commands need a lab setup (a Linux machine with Docker, minikube, kubectl and Internet access). Execute them in the laboratory and record the output.

RESULT:

Thus a containerised application was deployed on Kubernetes and its resource utilisation was evaluated successfully.